

COSMIC–ASCEC

*Automated Configurational and Conformational Sampling
with Topological Screening of Molecular Clusters*

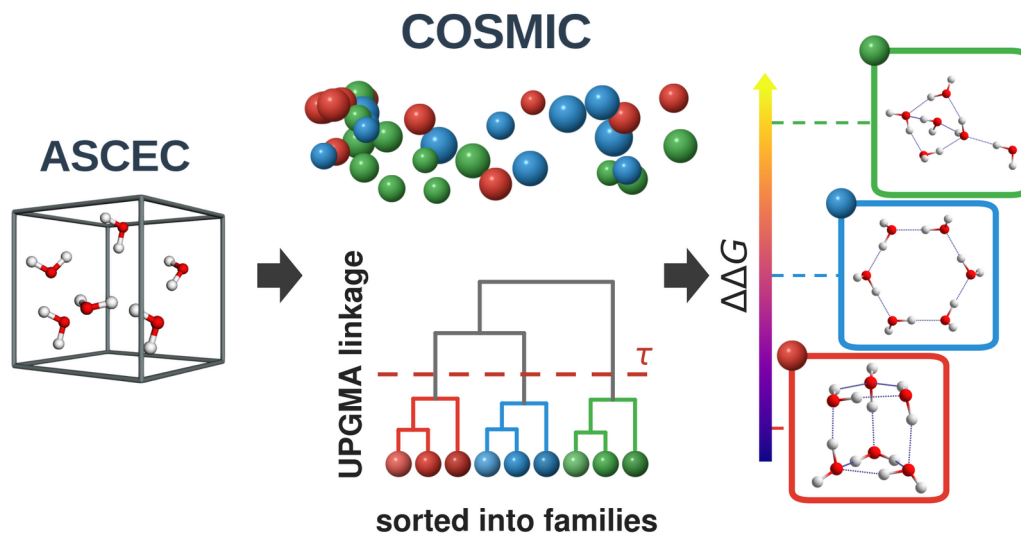
User Manual

Manuel Gómez[†] – Sara Gómez[‡] – Albeiro Restrepo[†]

Química Física Teórica

[†]*Instituto de Química, Universidad de Antioquia UdeA, Medellín, Colombia*

[‡]*Universidad Nacional de Colombia, Departamento de Química, Bogotá, Colombia*



Last Updated: June 2026

Contents

1	About COSMIC-ASCEC	4
1.1	Search Modes	4
1.2	Distribution	4
2	How It Works	5
3	Installation	5
3.1	System Recommendations	5
3.2	Required Software	6
3.3	Step by Step Installation	6
4	Web Input Generator	12
4.1	Features	12
4.2	Accessing the Tool	12
4.3	GUI Modes	12
5	Quick Start: Glycolaldehyde and Water	14
6	Water Hexamer	19
6.1	Molecular Coordinates Generation	19
6.2	Mode Selection	19
6.3	Simulation Box	20
6.4	Annealing Parameters	21
6.5	Geometry Optimization and Refinement	21
6.6	Energy Refinement and COSMIC Screening	22
6.7	Pipeline Review and Execution	22
6.8	Results	24
7	Theoretical Background	28
7.1	Simulated Annealing Algorithm	28
7.2	Monte Carlo Moves	29
7.3	Topological Clustering (COSMIC)	29
8	Input File Format	33
8.1	Block 1: Simulation Parameters and Coordinates	33
8.2	Block 2: The Automated Protocol and Embedded Templates	34
9	Box Size Suggestion	38
9.1	How V_{eff} is Estimated	39
9.2	Choosing the Packing Fraction	39
10	Standalone Functions: Understanding Each Stage	40
10.1	Annealing	40
10.2	Geometry Optimization	42
10.3	Sorting the Results	43
10.4	COSMIC Clustering	44
10.5	Geometry Refinement	45
10.6	What Standalone Mode Does Not Do	46
11	Output Files	47
11.1	Print Levels: miniprint and maxprint	47
11.2	Annealing Output	47
11.3	Calculation and Optimization Output	52
11.4	COSMIC Output	52
12	Workflow Mode	60
12.1	Pipeline Logic	60
12.2	Workflow Input (formic.asc)	60
12.3	Protocol Command Syntax	62

12.4	Execution and Outputs	62
12.5	Review Runs: Rehearsing an Expensive Protocol	64
12.6	Execution Control and Resumption	66
12.7	Failure Recovery Mechanisms	67
13	Results and Validation	69
13.1	Manual Execution: Water Hexamer	69
13.2	Automated Workflow: Formic Acid Dimer	73
13.3	Review Run versus Full Run: the Li ₅ Cluster	77
13.4	Clustering Methodologies: Topology vs. Geometry	78
14	Command Reference	81
14.1	Cheat Sheet	81
14.2	Sampling and Replication	82
14.3	Automated Workflow (Protocol)	82
14.4	Protocol Thresholds	82
14.5	COSMIC Analysis Module	83
15	Quick ORCA 6.1.1 Installation for Linux	83
16	AI Acknowledgement and Reproducibility	85
17	Glossary	85
18	References	86

Part I

Getting Started

1 About COSMIC-ASCEC

Chemistry is not the exclusive domain of the global minimum. A system in thermal equilibrium populates many minima at once, so understanding the chemistry means finding the whole set of structures that contributes to it.

COSMIC-ASCEC is a tool for that purpose. **ASCEC** (*Annealing Simulado Con Energía Cuántica*) explores the potential energy surface; **COSMIC** (COnfigurational Similarity via Motif Identification Clustering) then removes the redundancy from what it finds, so the expensive refinement runs only on structures worth refining. Section 2 describes the stages.

COSMIC compares physicochemical descriptors rather than geometry, so it needs no atom numbering and no superposition, and it reports how far apart two structures really are instead of leaving the judgment to the human eye.

You supply the input file, created using the online generator, what comes back is a characterized surface: the distinct minima, their relative energies, and their Boltzmann populations.

1.1 Search Modes

Every run is one of three modes. They differ in how far the pipeline continues after the first clustering pass, and therefore in what the result supports.

Preliminary runs annealing, optimization and one COSMIC screening, entirely at a low cost semiempirical level. It maps the surface in minutes, which makes it the natural first step with any new system. Because no frequencies are computed, the structures it returns are candidates rather than confirmed minima.

Rigorous adds a geometry refinement at a higher level of theory with a frequency calculation, followed by a second COSMIC screening. The frequencies confirm each structure as a true minimum and supply the thermal corrections, so this mode delivers verified minima with relative free energies and Boltzmann populations. This is the mode for results you intend to publish.

Ultimate adds a high level single point energy refinement on the refined geometries, for example DLPNO-CCSD(T), and a final COSMIC screening. The geometries are those of the rigorous mode, what improves is the accuracy of the relative energies that order them.

Cost rises steeply across the three. A preliminary run of a small system takes minutes, while rigorous and ultimate runs of the same system take hours to days, almost all of it in the refinement stages. Start preliminary, confirm the setup, then rerun in a higher mode.

Review is not a fourth mode but a modifier that applies to any of the three. A review run (`--review`) executes every stage of the chosen protocol, yet carries only the lowest energy structure produced by the annealing, the putative global minimum, into the expensive stages. It is the cheap way to confirm that a rigorous or ultimate protocol is correct before committing to it, and is described in Section 12.5.

1.2 Distribution

COSMIC-ASCEC package is distributed via a public GitHub repository:

- **Repository:** <https://github.com/manuel2gl/qft-cosmic-ascec>

- **License:** GNU General Public License (see LICENSE file for details)
- **Supported OS:** Primarily developed and tested on Linux (Ubuntu, Debian, Mint). Preliminary runs are also tested on Windows 11 using xTB 6.7.1.
- **Contents:**
 - Python scripts for configurational, conformational sampling, and clustering
 - Documentation and brief theoretical description
 - Example input files and workflows
 - Web based input generator:
 - * <https://manuel2gl.github.io/qft-cosmic-ascec/>
- **Requirements:**
 - Python 3.9 or higher (3.11 suggested)
 - ORCA or xTB for quantum chemistry calculations

2 How It Works

The COSMIC-ASCEC pipeline operates in three stages:

1. **Sampling (Annealing):** A simulated annealing algorithm randomly moves, rotates, and deforms the molecules inside a box. At each step, a quantum chemistry package (ORCA or xTB) evaluates the energy. The algorithm gradually “cools down”, favoring low energy structures while still allowing occasional uphill moves to escape local traps. This produces hundreds of candidate configurations.
2. **Clustering (COSMIC):** Most of these candidates are redundant; to overcome this limitation, the COSMIC (COnfigurational Similarity via Motif Identification Clustering) module compares them using a fingerprint constructed from their energies, orbital vacancies, dipole moments, and other properties. Structures with similar fingerprints are grouped into families. Only one representative per family (a **motif**) is retained for further study.
3. **Refinement:** The motifs are refined at a higher level of theory (e.g., DFT) to obtain publication quality geometries, energies, and thermodynamic populations.

3 Installation

3.1 System Recommendations

Although Python scripts are lightweight, quantum chemistry backends are resource intensive:

- **RAM:** Minimum 8GB (16GB+ recommended for DFT calculations).
- **CPU:** Multicore processor (ASCEC can parallelize ORCA jobs).
- **Storage:** SSD recommended. Optimization trajectories can generate large text files.
- **OS:** Linux (Ubuntu/Mint) is the primary supported environment. Windows is supported for preliminary runs using xTB.

3.2 Required Software

3.2.1 Core Python Dependencies

The core functionality is based on standard Python scientific libraries and other software.

Package	Purpose
NumPy	High performance array operations and linear algebra.
SciPy	Optimization routines and statistical functions.
scikit-learn	Algorithms for hierarchical clustering and data preprocessing.
Matplotlib	Generation of energy plots and dendrograms.
cclib	Parsing of legacy ORCA ($\leq 5.0.x$) logs.
OPI	Parsing of modern ORCA (≥ 6.1) outputs.
OpenBabel	File generation and format conversion.
xTB	Highly efficient semiempirical quantum chemical framework.

Table 1: Software required by the ASCEC and COSMIC modules.

***Compatibility Note:** The **COSMIC** module automatically selects the appropriate parser based on the output file version. **cclib** is used for ORCA $\leq 5.0.x$, while **OPI** is required for ORCA 6.1+. *Note: ORCA 6.0 is not supported by either parser; please use v5.0.x or upgrade to v6.1+.* At least one parser must be installed.

3.2.2 Quantum Chemistry Backends

The workflow requires an external electronic structure package to perform QM calculations.

ORCA: ORCA [1] is free for academic use and supports a broad range of high level ab initio and DFT methods. It is fully integrated with both the ASCEC and COSMIC modules. See Section 15 for installation instructions.

xTB: xTB [2] is the default backend for both the annealing and preliminary preoptimization stages due to its speed. It is installed automatically from **conda-forge** by **install.sh** and **win_install.bat**.

3.3 Step by Step Installation

3.3.1 Prerequisites

git is required. The installer makes your copy of COSMIC-ASCEC a real git checkout, which is what lets you update it later with **git pull** and see your own edits with **git status**. Install it before anything else.

On Linux, use your distribution’s package manager:

```
sudo apt install git      # Debian, Ubuntu, Mint
sudo dnf install git      # Fedora, RHEL
sudo pacman -S git        # Arch
```

On Windows, install Git for Windows from <https://git-scm.com/download/win>.

Then confirm it is on your PATH:

```
git --version
```

No administrator rights (for example on a shared cluster)? Run the installer anyway: it falls back to installing git from `conda-forge`, which needs no root. That copy lives inside the conda environment, so it is only on your PATH after `conda activate`, which is why a system wide git is preferable whenever you can get one.

3.3.2 Automatic Installation

We provide a shell script that automates the entire process. It installs Miniconda (if not present), sets up the dependencies (`xTB`, `OpenBabel`, `cclib`, `OPI`), and configures the necessary aliases. The aliases point directly to the environment's Python binary, so no manual `conda activate` is needed to run `ascec` or `cosmic`.

Download the script, run it, delete the download and reload your shell, all in one line. It installs into a Python 3.11 environment named `py11`:

```
cd "$HOME" && wget \
  https://raw.githubusercontent.com/manuel2gl/qft-cosmic-ascec/main/install.sh \
  && bash install.sh && rm -f install.sh && source ~/.bashrc
```

If `wget` is not available, swap the download for `curl`:

```
cd "$HOME" && curl -fLO \
  https://raw.githubusercontent.com/manuel2gl/qft-cosmic-ascec/main/install.sh \
  && bash install.sh && rm -f install.sh && source ~/.bashrc
```

The leading `cd "$HOME"` keeps the download out of any existing checkout, so the `rm` can only ever remove the copy you just fetched. Confirm the install:

```
ascec --version
```

Installing into base instead

If you would rather not have a separate environment, install the latest Miniconda, downgrade Python within `base`, then edit the flag at the top of the script and run it again.

```
micro install.sh
```

```
INSTALL_PY11=FALSE
```

Save the file (`Ctrl+S`) and exit (`Ctrl+Q`).

Uninstall (Linux)

To reverse the installation without touching your Conda base, run the provided uninstaller.

```
bash uninstall.sh
```

This removes the `py11` environment, the `~/software/ascec04` directory, and the `ascec/cosmic` aliases from `.bashrc`. Conda itself is left intact.

Windows Installation

On Windows the same workflow is provided as a `.bat` file (a hybrid batch/PowerShell script). Install Git for Windows first (Section 3.3.1), then download `win_install.bat` from the repository and either double click it or run it from a Command Prompt. The script installs Miniconda if absent, creates the `py11` environment, installs all dependencies (including xTB 6.7.1), and registers `ascec` and `cosmic` as launchers on your user PATH. If Windows blocks execution, right click the file and select *Run as administrator*.

From a Command Prompt, in one line:

```
cd /d "%USERPROFILE%"
curl -fLO ^
  https://raw.githubusercontent.com/manuel2gl/qft-cosmic-ascec/main/win_install.bat
win_install.bat && del win_install.bat
```

Then open a new Command Prompt so the updated PATH is picked up, and check:

```
ascec --version
```

Uninstall (Windows)

Double click `win_uninstall.bat` (or run from Command Prompt) to remove the `py11` environment, the source directory, and the launchers. Miniconda is left untouched.

```
win_uninstall.bat
```

3.3.3 Installing on a Cluster

On a shared machine you usually cannot install software as root, and the site already provides conda as a module. The installer is built for this: it never writes into a shared conda prefix, and every path it uses can be redirected from the command line. Load the site's conda and run the installer as usual:

```
module load anaconda3          # or whatever your site calls it
bash install.sh
```

The installer locates conda by asking it, with `conda info --base`, so any conda already on your PATH is found automatically whatever its layout: Miniconda, Anaconda, miniforge, a module or

a container image. Only when no conda is on `PATH` does it look at the usual prefixes, and only if that also finds nothing does it install Miniconda itself. Set `CONDA_ROOT` solely to override that choice, for instance to select one of several installed condas:

```
CONDA_ROOT=/apps/anaconda3 bash install.sh
```

The environment is then created in your own `envs` directory (normally `~/.conda/envs`) whenever the shared prefix is read only, so nothing you do affects other users. No step needs `sudo`: if git is missing the installer pulls it from `conda-forge` into the environment.

Home directories with a quota. Send both the software and any Miniconda the script installs to a larger filesystem:

```
TARGET_DIR=/scratch/$USER/ascec04 \
  CONDA_INSTALL_DIR=/scratch/$USER/miniconda3 \
  bash install.sh
```

Older site conda. If the module's conda has neither mamba nor the libmamba solver, skip the probing and use the classic solver directly:

```
CONDA_SOLVER=classic CONDA_ROOT="$(conda info --base)" bash install.sh
```

Every knob is an environment variable read at the top of `install.sh`; the defaults apply when you set nothing.

Variable	Purpose
<code>CONDA_ROOT</code>	Select a specific conda prefix. Rarely needed: any conda on <code>PATH</code> is detected automatically.
<code>CONDA_INSTALL_DIR</code>	Where a Miniconda installed <i>by the script</i> should land. Default <code>~/miniconda3</code> .
<code>TARGET_DIR</code>	Where the source is installed. Default <code>~/software/ascec04</code> .
<code>ENV_NAME</code>	Name of the conda environment. Default <code>py11</code> .
<code>PY_VERSION</code>	Python version for that environment. Default <code>3.11</code> .
<code>CONDA_SOLVER</code>	<code>auto</code> , <code>mamba</code> , <code>libmamba</code> or <code>classic</code> .
<code>INSTALL_MAMBA</code>	Whether mamba may be added to the base environment. Never touches a base the script does not own.
<code>MINICONDA_URL</code>	Installer to download; change it on <code>aarch64</code> or <code>ppc64le</code> , or to pin a release.
<code>REPO_URL</code> , <code>REPO_BRANCH</code>	Install a fork or a test branch.
<code>ALLOW_TARBALL</code>	Permit the plain directory install on a machine that truly cannot have git.

Table 2: Environment variables accepted by `install.sh`.

Login shells read `~/.bash_profile` rather than `~/.bashrc`, so the installer writes its alias block to whichever of `.bashrc`, `.zshrc` and `.bash_profile` exist. That is what makes `ascec` resolve inside a batch job.

3.3.4 Manual Installation with Conda

Use this route when you want to see every step, or when the automatic installer cannot run. It does exactly what `install.sh` does.

I. Get the source. Clone the repository. Later, `git pull` in the same directory updates it.

```
mkdir -p "$HOME/software" && git clone \
  https://github.com/manuel2gl/qft-cosmic-ascec.git "$HOME/software/ascec04" \
  && cd "$HOME/software/ascec04" && pwd && ls
```

II. Install Miniconda (skip if you already have conda). Answer `yes` when it offers to initialize conda.

```
cd "$HOME" && wget \
  https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh \
  && bash Miniconda3-latest-Linux-x86_64.sh && source ~/.bashrc
```

III. Create the environment and install the dependencies.

```
conda create -n py11 python=3.11 -y \
  && conda install -n py11 -c conda-forge numpy scipy matplotlib \
  scikit-learn cclib openbabel 'xtb>=6.7' -y \
  && conda run -n py11 pip install orca-pi
```

Both ORCA parsers can coexist: cclib reads ORCA $\leq 5.0.x$ and OPI reads 6.1+. OPI is installed with pip even inside conda.

IV. Add the aliases. Pointing them at the environment's own Python binary means you never need `conda activate` to run either program. This writes the two lines for you, using your real conda path:

```
CB="$(conda info --base)" && printf '%s\n' \
  "# COSMIC ASCEC aliases" \
  "alias ascec='$CB/envs/py11/bin/python $HOME/software/ascec04/ascec-v04.py'" \
  "alias cosmic='$CB/envs/py11/bin/python $HOME/software/ascec04/cosmic-v01.py'" \
  \
  >> ~/.bashrc && source ~/.bashrc
```

V. Verify. No environment activation needed:

```
ascec --version && cosmic --version && obabel -V
```

```
conda run -n py11 python -c \
  "import numpy, scipy, matplotlib, sklearn, cclib, opi; print('All packages OK')"
```

If you installed ORCA as well (Section 15), check it too with `orca -v`. The help screen confirms the command is wired up.

4 Web Input Generator

ASCEC-v04 includes a browser based tool to generate input files automatically, fetch structures from PubChem, and visualize the simulation box in 3D.

4.1 Features

- **PubChem Integration:** Automatically converts SMILES, IUPAC names, or brand names (e.g., "aspirin") into 3D XYZ coordinates using the PUG REST API [3].
Compounds search: <https://pubchem.ncbi.nlm.nih.gov/compound/>
- **3D Visualization:** Uses 3Dmol.js [4] to preview the simulation box, showing the boundaries and initial molecular placement.
- **Protocol Builder:** easy creation of multistage workflows (e.g., `opt`, `cosmic`, `ref`).

4.2 Accessing the Tool

You can access the generator in two ways:

1. **URL:** Visit <https://manuel2gl.github.io/qft-cosmic-ascec/>
2. **Direct:** Run the following command in your terminal:

```
ascec input
```

4.3 GUI Modes

The tool offers two complementary interfaces, selectable from the top navigation bar: **Smart Mode** (the default) and **Advanced Generation**. Both produce the same ASCEC input file; they differ only in how much of the configuration is exposed and how much is automated.

Smart Mode

Smart Mode is a guided, automated workflow designed to get a valid simulation running with minimal input. The typical sequence is:

1. **Build the system.** Enter any PubChem identifier (name, SMILES, CAS, or InChI, e.g. `water`, `64-17-5`, `CCO`) in the *Molecule* $\rightarrow$ *XYZ Converter* to fetch 3D coordinates, and set the number of copies for each species.
2. **Set the simulation box.** Specify the box length directly, or enable *auto* to compute it from a target packing percentage. The molecules and box boundaries are previewed live in the 3D viewer.
3. **Choose the parameters.** A guided form selects sensible defaults for the annealing, preoptimization, refinement, and COSMIC settings, so you only adjust the key choices.
4. **Review and download.** A pipeline summary shows every stage of the workflow before you download the generated input file.

Advanced Generation

Advanced Generation exposes every field of the input file on one page, grouped the way the file itself is laid out. Use it when Smart Mode's defaults are not what you want, or when you need a setting Smart Mode does not offer. Nothing is hidden and nothing is inferred: what you see is what gets written.

The page is organised into the following blocks.

Annealing Theory Level Theory, method and basis set, the QM program and the command used to call it, processor counts for the QM job and for ASCEC itself, how many jobs run concurrently, and the system's charge and multiplicity.

Conformational Search Whether internal degrees of freedom are sampled, the sampling percentage, and the maximum dihedral change.

Quenching and Packing Simulation mode, the linear or geometric route with its initial temperature, factor or step size and number of steps, the Monte Carlo cycles per temperature and their floor, maximum displacement and rotation, the packing percentage or an explicit box length, the number of replicas, and the molecule count.

The three calculation stages One block each for Geometry Optimization, Geometry Refinement and Energy Refinement, each with its own processors, memory, job type and iteration limit, and each with a **live input preview** showing the template exactly as it will be embedded in the `.asc` file. The refinement stages are optional, which is how the same page produces a preliminary, rigorous or ultimate protocol.

COSMIC Screening The threshold, parallel jobs, whether RMSD refinement runs, and any extra flags, set independently for the pass after optimization and the pass after refinement, or locked together with one switch. A weights panel lets individual feature vector components be reweighted; the components are described in Section 7.

Recovery The recovery strategy, the critical percentage that triggers it, and the redo limit, described in the failure recovery section of Part II.

When any stage is set to standalone xTB, two extra controls appear in that block: an **implicit solvent** model with its solvent list, and a free text **extra xTB flags** field taking a comma separated list written without the leading dashes, for example `etemp 400, acc 0.5`. Both are written into the template as `#` directives, and the preview updates as you type. The full flag set is covered in Part II. Anything the controls do not cover can be typed directly into the preview, or edited in the `.asc` file afterwards.

A **Pipeline Overview** at the top of the page lists the stages your current settings produce, in order, so the protocol can be checked before download.

The screenshot displays the COSMIC-ASCEC web application interface. At the top, navigation links include 'Installation', 'Smart Mode' (selected), 'Advanced Generation', 'User Manual', and 'GitHub'. A header bar contains the text 'Automated configurational sampling and topological screening of molecular clusters', 'Química Física Teórica', and 'Universidad de Antioquia, Colombia'.

The main content area is divided into three primary sections:

- Simulation Box:** Features a 'Box Length (Å)' input set to 6.0, a 'Dense' checkbox, and a 'Visual Test' button. A 3D visualization shows a molecular cluster within a cubic box. A 'Box Suggestion — adjust as needed' message is present. On the right, 'Added molecules (3)' are listed: 'Water (2)' and 'Glycolaldeh... (1)'. Below the box are buttons for 'Refresh', 'Download XYZ', 'Download Box XYZ', and an 'Enter simulation name' field.
- Molecule → XYZ Converter:** Includes a text input for 'Enter any PubChem identifier (name, SMILES, CAS, InChI) to generate 3D coordinates.' Below this is a list of molecule buttons: Water, Ethanol, Formic Acid, Alanine, Benzene, Methane, Ammonia, Methanol, Acetone, Acetic Acid, Toluene, Phenol, Pyridine, THF, and Hexane. There are also links for 'PubChem', 'Novoprolabs 3D', and a 'Periodic Table' button.
- Molecule Preview:** Shows a 3D ball-and-stick model of a molecule. Below the model are buttons for 'Add', 'XYZ', 'SMILES', 'Clear (3)', and a menu icon.

The **Sampling Mode** section at the bottom offers three options: 'Preliminary' (Stop after geometry optimization), 'Rigorous' (Stop after geometry refinement), and 'Ultimate' (Stop after energy refinement). A 'MAXPRINT' checkbox is located to the right. At the very bottom, a workflow progress bar shows steps: 'Annealing' → 'Geometry Optimization' → 'COSMIC 1' → 'Geometry Refinement' → 'COSMIC 2' → 'Energy Refinement' → 'COSMIC 3' → 'Results'.

Figure 1: Web Input Generator interface with 3D visualization and parameter forms.

5 Quick Start: Glycolaldehyde and Water

This section presents a minimal example that demonstrates the core functionality of COSMIC-ASCEC through the hydration of glycolaldehyde. It is intended as a black box test that can be run without prior knowledge of the underlying theory or the input file syntax, providing a convenient way to verify the installation and obtain a first overview of the program’s workflow and results.

The system consists of one glycolaldehyde and two water molecules. This is a particularly instructive test case because glycolaldehyde exhibits both **configurational** (intermolecular arrangement) and **conformational** (internal dihedral rotation) degrees of freedom, making it richer than a purely rigid body system.

We begin by generating the initial coordinates with the web input generator. Glycolaldehyde is the simplest member of the diose family, so it can be located in the generator by searching for the name *diose* (Figure 2).

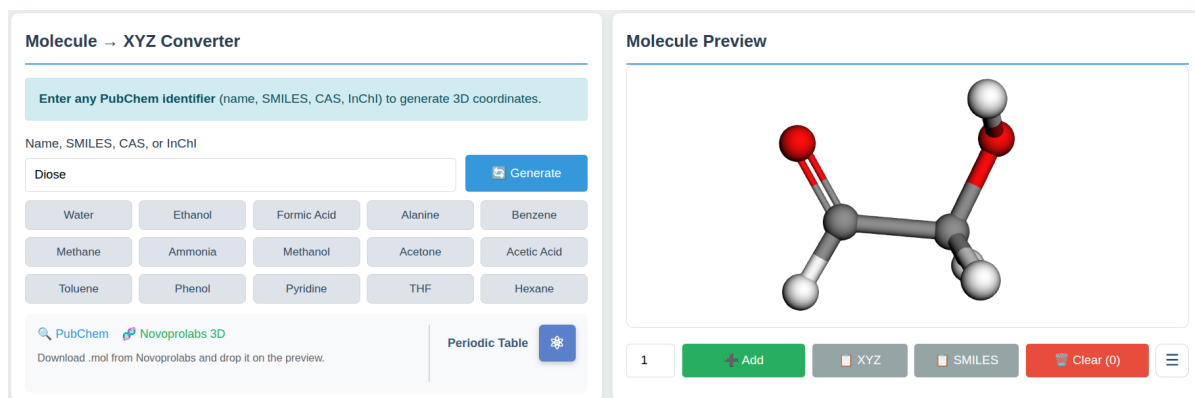


Figure 2: Generating initial coordinates for glycolaldehyde and water using the web interface.

A crucial step in the setup is superposition: both the glycolaldehyde and the water molecules are placed at the exact center of the simulation box, ensuring that the stochastic search starts from an unbiased configuration. The resulting setup should resemble Figure 3, so that the center of mass for each molecule occupies the same region of space before the simulation begins.

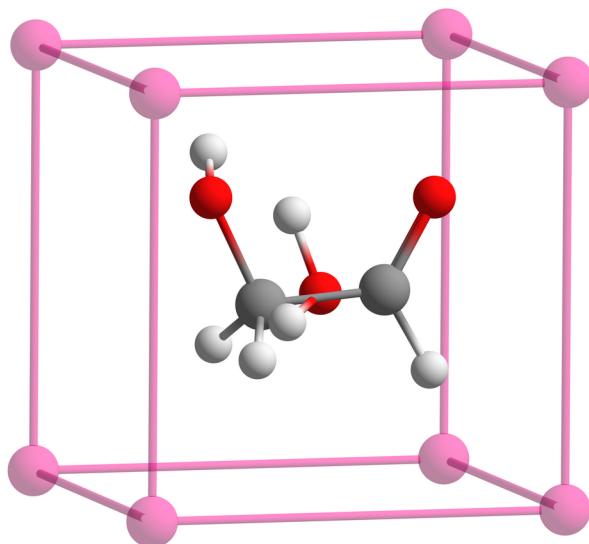


Figure 3: Initial simulation box containing glycolaldehyde and water superposed at the center.

In this example we select a preliminary run (Figure 4), which uses the GFN2-xTB method for both the single annealing stage and the subsequent candidate preoptimization; motifs are then identified by the COSMIC clustering algorithm. This is an **optimization only** workflow (no frequency calculations), which keeps the process fast but does not guarantee that the resulting structures are true local minima.

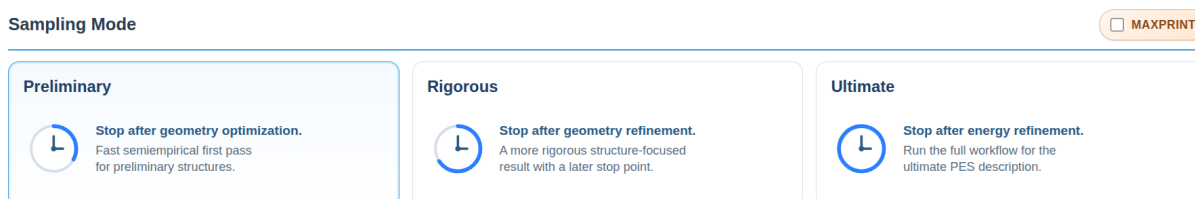


Figure 4: Preliminary sampling mode selection.

We enable the conformational option at the annealing theory level (Figure 5) and then download the .asc input file.

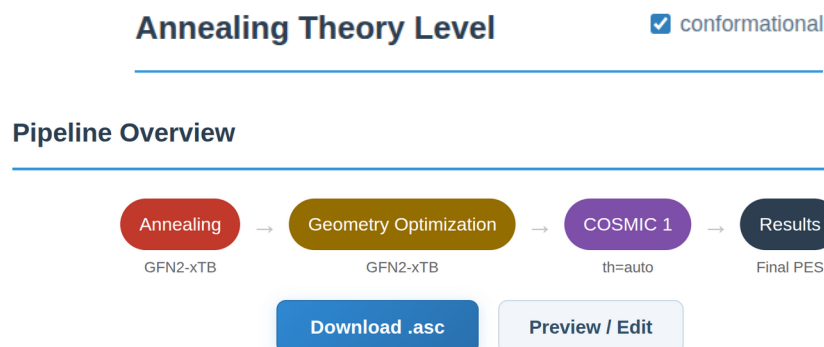


Figure 5: Conformational check and input download.

The input file for this example ships with the repository, together with one input per system and mode used in this manual.

```

/
├─ examples/
│   └─ glyw2/
│       ├── preliminary/glyw2.asc
│       ├── rigorous/glyw2.asc
│       └─ w6/
│           ├── preliminary/w6.asc
│           ├── rigorous/w6.asc
│           └─ ultimate/w6.asc
  
```

To run the simulation, simply execute the program. This calculation is configured to use eight processor cores.

```
cd examples/glyw2/preliminary && ascec glyw2.asc
```

Upon execution, a progress bar is displayed.

```

=== COSMIC ASCEC ===
-----
Progress [.....] 0.0%
-----
[1/3] Annealing ...
  
```

Here, [1/3] indicates that the program is in the first of three main stages: Annealing, Geometry Optimization, and COSMIC screening.

Once the run is complete, the terminal will display a summary message.

```

=== COSMIC ASCEC ===
-----
Progress [#####] 100.0%
-----
[1/3] Annealing ✓
[2/3] geometry_optimization ✓
[3/3] cosmic (80→37) ✓

Workflow finished
Start: 2026-06-23 17:43:40
End: 2026-06-23 17:44:17
Total wall time: 36s 835ms
-----
Warning: No true minima can be assured without frequency calculations.
Protocol summary saved to protocol_summary.txt

✓ Miniprint: 23.8 MB (use --maxprint to keep all files)

```

In approximately 37 seconds, ASCEC autonomously executed a three stage pipeline:

1. **Annealing** (66.3%): The stochastic search explored the configurational and conformational space of the glycolaldehyde–two water complex using GFN2-xTB energies. Starting from a “Big Bang” superposition, it generated **80 accepted configurations** while cooling the system from 300 K down to near 0 K.
2. **Geometry Optimization** (33.7%): All 80 candidate structures were optimized at the GFN2-xTB level, relaxing each configuration into the nearest local energy basin and producing a set of preoptimized structures.
3. **COSMIC** (< 0.1%): The clustering algorithm analyzed the 80 optimized structures using a reduced physicochemical feature vector and identified **37 motifs**, i.e., topologically distinct families of hydrated glycolaldehyde.

A representative subset of these motifs is shown in Figure 6; each of them corresponds to a different interaction pattern between water and glycolaldehyde, with a certain degree of redundancy remaining.

Note the warning in the terminal output. Because this workflow uses **Opt** (optimization only, without frequency calculations), the COSMIC module operates with a **reduced feature vector**. This is sufficient for a rapid preliminary survey of the PES, comparable to the level of analysis performed by many conformational sampling programs. However, without frequency analysis the structures cannot be rigorously confirmed as true local minima, since some may correspond to saddle points. For publication quality results, a complete **Opt+Freq** workflow should be used (as demonstrated in the Water Hexamer example); this yields a fully characterized PES with verified minima and a Boltzmann weighted population distribution. The trade off between speed and rigor is ultimately the user’s decision and depends on the goals of the study.

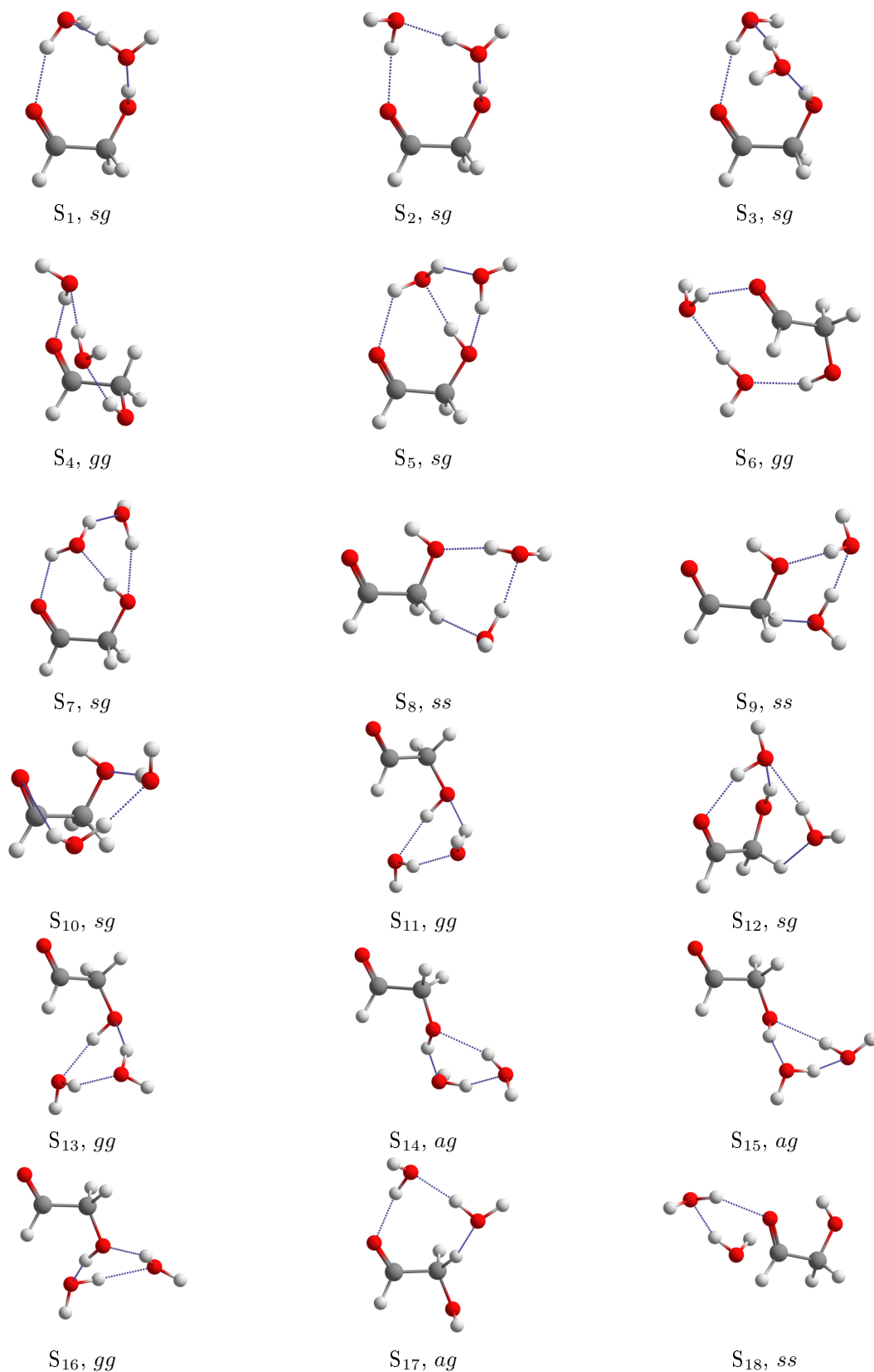


Figure 6: First 18 of the 37 preliminary glycolaldehyde water motifs.

6 Water Hexamer

This section demonstrates the entire COSMIC-ASCEC workflow using the web generator to build the input file from scratch. Unlike the Quick Start example, this pipeline includes a geometry refinement stage, producing a fully characterized PES with verified minima and Boltzmann populations. We use the water hexamer (H_2O)₆ as a well studied benchmark system [5].

6.1 Molecular Coordinates Generation

Open the Web Input Generator at <https://manuel2gl.github.io/qft-cosmic-ascec/> (or run `ascec input` in the terminal). Using Smart Mode, search for "water" in PubChem to fetch its 3D structure, then add six copies to the simulation box. (Figure 7).

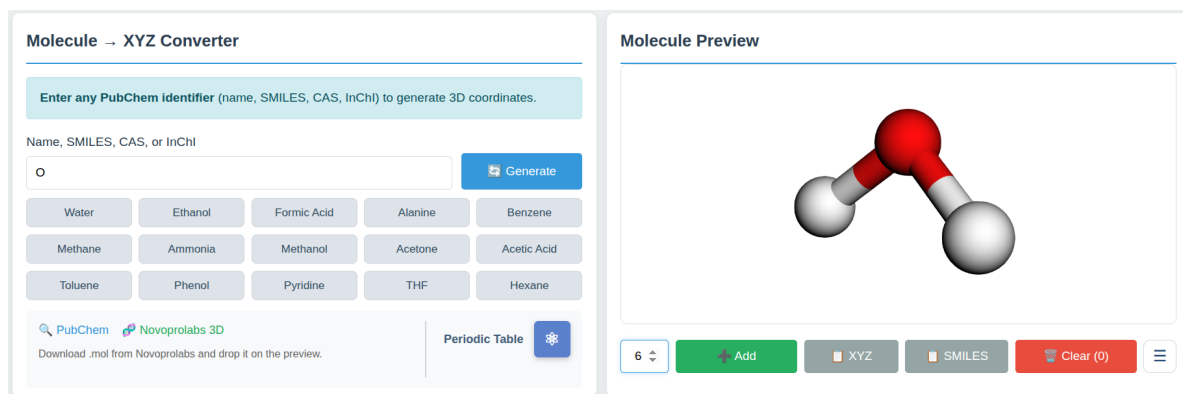


Figure 7: Generating six water molecules using the PubChem integration.

6.2 Mode Selection

The web generator provides several predefined modes. A **preliminary** run uses no replicas and a single low cost level of theory for a quick mapping of the PES. The **rigorous** and **ultimate** modes add a higher level refinement stage, producing a fully resolved PES with any level of theory available on the backend.

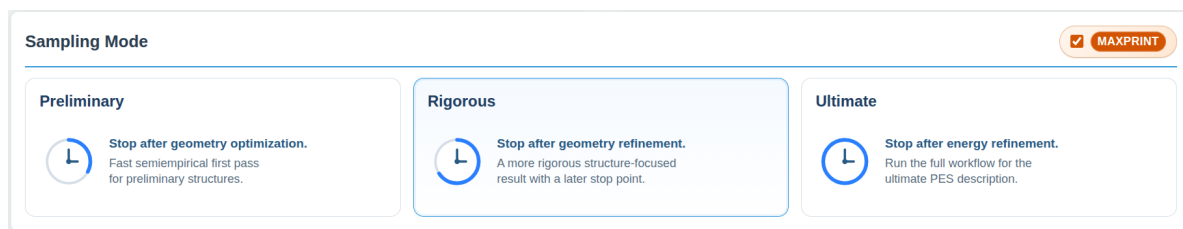


Figure 8: Rigorous mode selection.

As shown in Figure 8, the **maxprint** option is enabled by default in rigorous and ultimate modes: because these workflows involve expensive calculations, the user may prefer to retain all intermediate I/O files for further analysis. If it is disabled, **miniprint** is used instead and only the I/O files of the final ensemble are kept, saving disk space.

6.3 Simulation Box

The simulation box defines the cubic region where molecules are confined during the annealing search. The key parameter is the **effective packing percentage** ϕ , which controls how full the box is: a lower value produces a more dilute (spacious) box, while a higher value makes it denser. The default is 30% for preliminary runs, and 20% for rigorous and ultimate modes, which provides additional free space for the molecules to reorganize during the more exhaustive search. The algorithm that converts ϕ into a box length, together with the underlying geometric reasoning, is described in detail in Section 9.

The web generator (and ASCEC itself) automatically estimates the appropriate box size based on the molecular volumes and, for systems with hydrogen bond donors and acceptors, the additional space required by the H-bond network. For the water hexamer at the 20% effective packing used in rigorous mode, the suggested box length is approximately 7.3 Å (Figure 9). A more detailed box analysis using the standalone `ascec box` command is presented in Section 10.

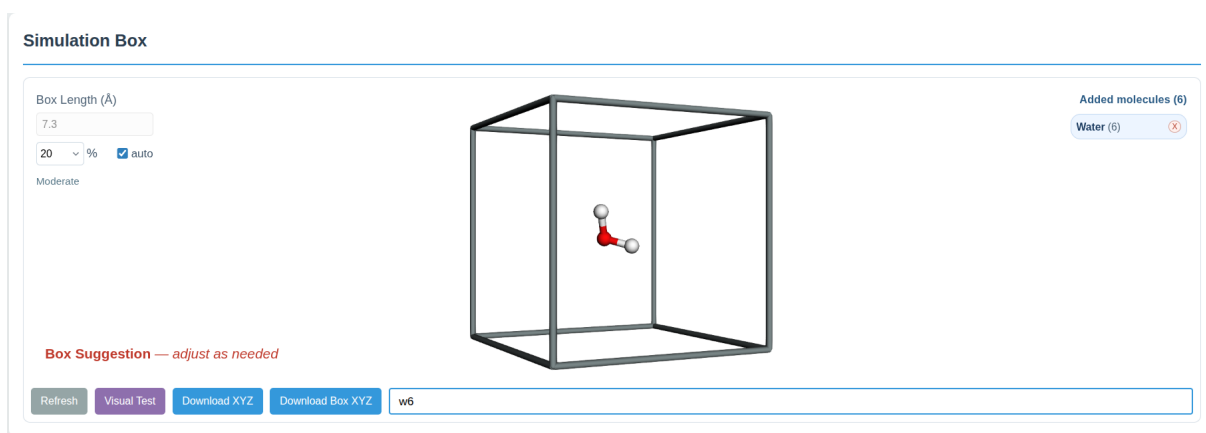


Figure 9: Simulation box setup for the water hexamer at 20% effective packing ($L \approx 7.3$ Å).

A visual test mimics the random rigid body moves performed by the ASCEC module, letting the user check whether the suggested box is adequate. This helps avoid both compressed boxes, which leave no room for cluster formation, and oversized boxes, in which the system cannot reliably evolve during the annealing procedure.

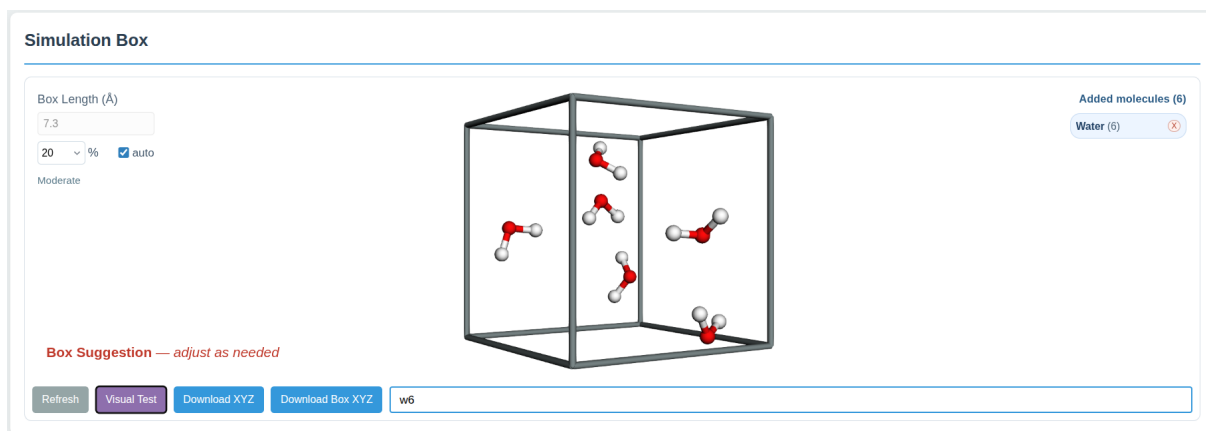


Figure 10: Water hexamer simulation box visual test

6.4 Annealing Parameters

Smart Mode provides a guided interface for selecting the annealing settings (Figure 11).

It supplies sensible defaults that the user can override as needed. The key parameters are:

- **Conformational:** Controls sampling of internal (torsional) degrees of freedom. For a *single* fragment it is enabled automatically with a sampling percentage of 100%. For *multiple* fragments it is optional (off by default); when enabled it defaults to 30%.
- **Annealing method:** GFN2-xTB semiempirical, for fast energy evaluations during the stochastic search.
- **nprocs:** Number of processors used for the QM calculations.
- **Concurrent & Replicas:** Three concurrent (triplicated) annealing runs to improve sampling coverage.
- **Charge & Multiplicity:** The system's overall charge and spin multiplicity; the GUI proposes a multiplicity, but it should always be reviewed.
- **Temperature schedule:** Geometric quenching, which gradually lowers the acceptance probability of high energy states.
- **Box:** With auto box enabled, the packing fraction can be adjusted; with it disabled, a custom box length can be entered.

Annealing Theory Level				Quenching & Box Settings			
☐ conformational				☒ auto box			
Theory	Method	Basis		Route	Initial T (K)	Factor (%)	Steps
Standalone ...	GFN2-xTB	(none)		Geom...	500	5	100
nprocs	Concurrent	Charge	Mult.	Replicas	Packing (%)	Box (Å)	
1	3	1	1	3	20	10.0	

Figure 11: Annealing parameter selection.

6.5 Geometry Optimization and Refinement

A geometry optimization is first carried out with the GFN2-xTB semiempirical method to relax the accepted structures and enable a first COSMIC screening.

The refinement step reoptimizes the unique motifs identified by the first clustering pass at a higher level of theory (Figure 12). For this run we use **wB97X-D3BJ/def2-TZVPP** with a full **Opt+Freq** treatment, which ensures that the final structures and their relative energies are accurate enough for meaningful thermodynamic analysis. Crucially, the frequency calculation confirms each structure as a true local minimum.

A second COSMIC analysis is then applied to the refined ensemble to produce the final set of unique motifs and the Boltzmann population distribution.

Geometry Optimization

Theory	Method	Basis
Standalone ...	GFN2-xTB	(none)
nprocs	Concurrent	MaxIter
1	8	200
Job		
Opt		

Geometry Refinement optional

Theory	Method	Basis
DFT	wB97X-D3BJ	def2-TZVPP
nprocs	Concurrent	MaxIter
12	2	120
Job		
Opt Freq		

Figure 12: Geometry optimization & Refinement theory level.

6.6 Energy Refinement and COSMIC Screening

An optional energy refinement is reserved for the ultimate mode, which is why it is disabled in this rigorous run. The COSMIC screening parameters are also set here: in **automatic** mode the protocol determines the best clustering threshold during the first COSMIC pass (after the geometry optimization stage; see Section 7.3.7 for details) and reuses that same threshold for the final COSMIC stage (Figure 13).

Energy Refinement optional

Theory	Method	Basis
Post-HF	DLPNO-CC...	aug-cc-pVTZ
nprocs	MaxCore (MB)	Job
1	50000	SP (Single ...)

COSMIC Screening same for opt/ref

Opt th	Opt RMSD	Opt -j
automatic	off	4
Ref th	Ref RMSD	Ref -j
opt	off	4

Figure 13: Energy Refinement & COSMIC Screening.

6.7 Pipeline Review and Execution

Before generating the input file, the interface shows the complete execution pipeline (Figure 14). This summary allows you to verify each stage before downloading the **.asc** file.

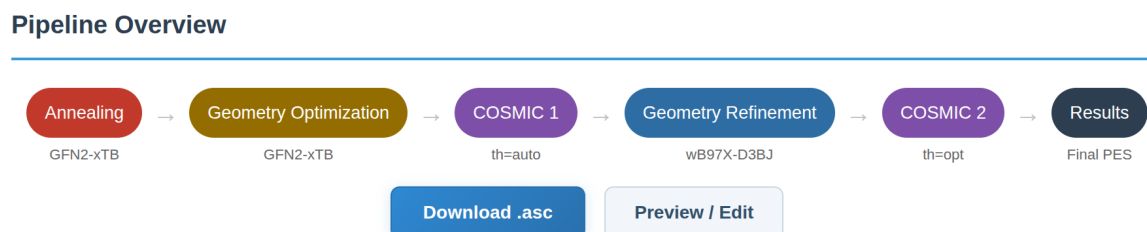


Figure 14: Complete pipeline review showing all workflow stages.

Download the generated `.asc` file and place it in your working directory, adjust the orca input lines as needed.

```
#orca input2
#name
#rescue(HF-3c/freq)
! wB97X-D3BJ def2-TZVPP def2/J RIJCOSX DEFGRID3
! Opt Freq TightOpt TightSCF
%pal
  nprocs 12
end
%geom
  maxiter 120
end
* xyz 0 1
#
*
```

Then execute the protocol:

```
ascec w6.asc &
```

Because the full pipeline can run for hours or days, it is normally left running unattended. There are two ways to detach it from the terminal: launch it in the background with the shell ampersand, (`ascec w6.asc &`), or simply press `Ctrl+D` (or `Ctrl+Z`) while the run is attached.

On either key the program relaunches itself fully detached from the terminal session and prints the background PID; the job keeps running and the shell is freed. Closing the terminal does not stop a detached job.

The `ascec status` command opens an interactive viewer of all ASCEC jobs on the machine (a registry shared across terminals), so a detached run can be inspected at any time:

```
ascec status
```

It lists the running and recent jobs with their ID, PID, input file, and start time. From the prompt you can attach to a job and watch its live progress (`V<id>`), terminate it (`K<id>`), refresh the list (`R`), or quit the viewer (`Q`), quitting the viewer leaves the jobs running.

```
ASCEC STATUS  (2026-06-25 20:28:46)

Running:
ID      PID    INPUT FILE          STARTED
---      -
1      1131777  w6.asc              2026-06-24 16:54:48

-----
[V <id>] Attach/View  [K <id>] Kill      [R] Refresh      [Q] Quit
-----
Choice: v1
```

```

=== COSMIC ASCEC ===
-----
Progress [#####.....] 68.6%
-----
[1/5] annealing ✓
[2/5] geometry_optimization ✓
[3/5] cosmic ✓ (215→39)
[4/5] geometry_refinement 21/39 ...

Attached: job 1 (w6.asc)   updated: 2026-06-25 18:29:05
Ctrl+C or Ctrl+D to detach (job keeps running)
Input realpath: /home/protio/Documentos/w6/w6.asc

```

The automated protocol will sequentially run all stages: Annealing, Geometry Optimization, COSMIC, Geometry Refinement, and Final COSMIC. Depending on the theory levels and number of replicas, the computation may take several hours to a few days.

The run is resumable. Each stage checkpoints its progress to a cache file, so if the workflow is interrupted (or deliberately stopped), simply reissuing the same command, `ascec w6.asc`, picks up where it left off: completed stages are read from the cache and skipped, and execution continues from the first unfinished stage.

If you have another protocol to run, you can queue it behind a job that is already running instead of waiting for it to finish. The `after` keyword takes the PID of the running job (shown in `ascec status`) and holds the new run until that job completes.

```
ascec input2.asc after 1131777
```

```

ASCEC queued in background (PID 3964514) - holding until PID 1131777 finishes,
then it starts automatically.
Track or cancel:  ascec status

```

6.8 Results

The automated protocol funnels the raw stochastic search down to a compact set of fully characterized minima. The three triplicated annealing runs produced **215 accepted configurations**, all of which were preoptimized at the GFN2-xTB level. A first COSMIC pass reduced this ensemble to **39 motifs**, which were then refined at the wB97X-D3BJ/def2-TZVPP level with frequency analysis. A second COSMIC pass on the refined ensemble yielded **22 unique minima**. The complete pipeline ran in roughly **1 day and 3 hours** on a standard workstation, with the DFT refinement stage accounting for $\sim 99.8\%$ of the wall time; the annealing and preoptimization stages together took only a few minutes.

The final clustering applied a topological threshold of $\tau = 2.0$. Its built in validation initially flagged 23.1% of the refined structures as critical (incomplete, or not a true minimum); these were automatically reprocessed, and after the redo passes the critical fraction dropped to 0%, confirming that all 22 representatives correspond to genuine local minima. Figure 15 shows the threshold diagnostic that selects τ and the resulting dendrogram of the 22 clusters.

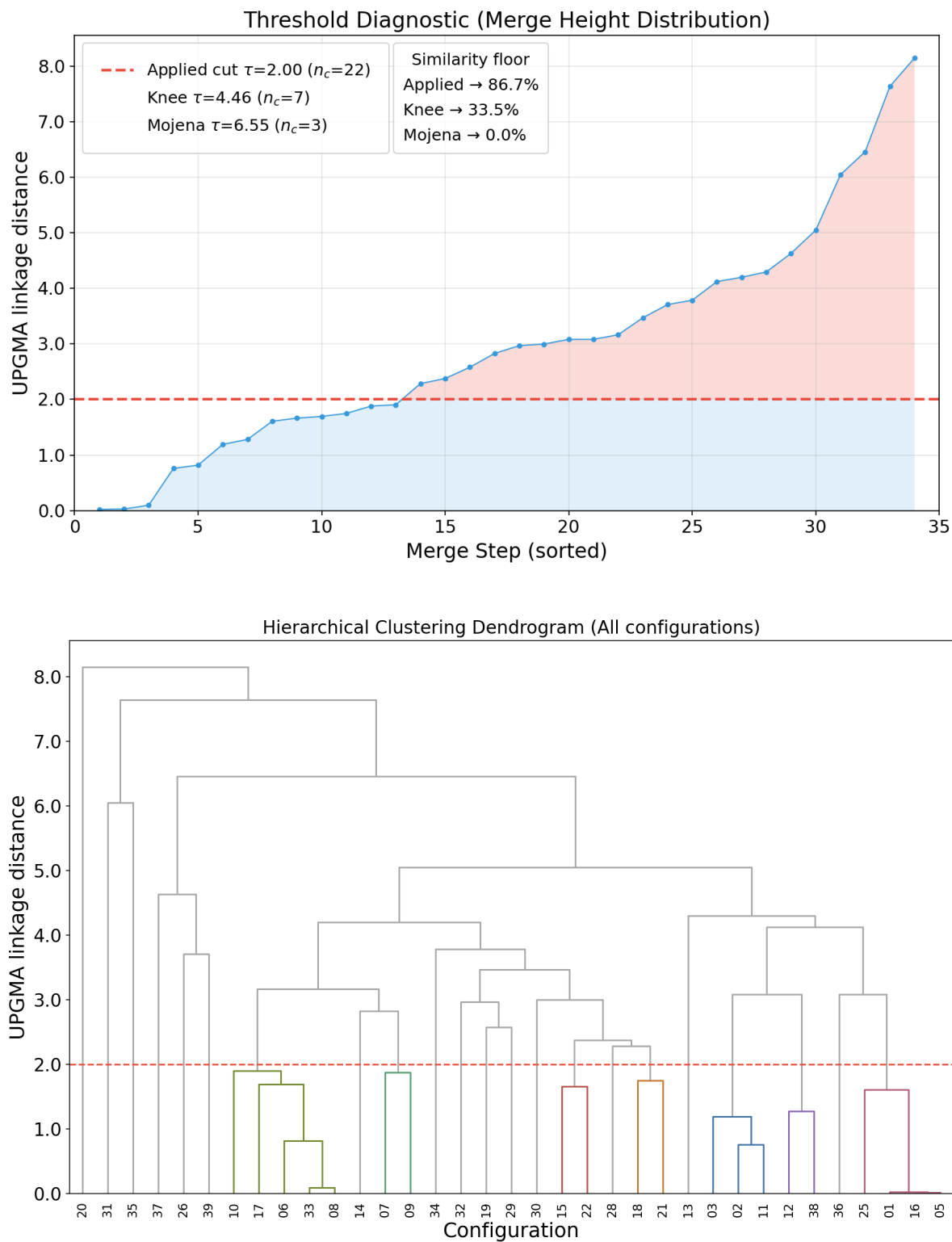


Figure 15: Final COSMIC analysis of the refined ensemble: the threshold diagnostic and the dendrogram grouping all refined structures into 22 unique motifs.

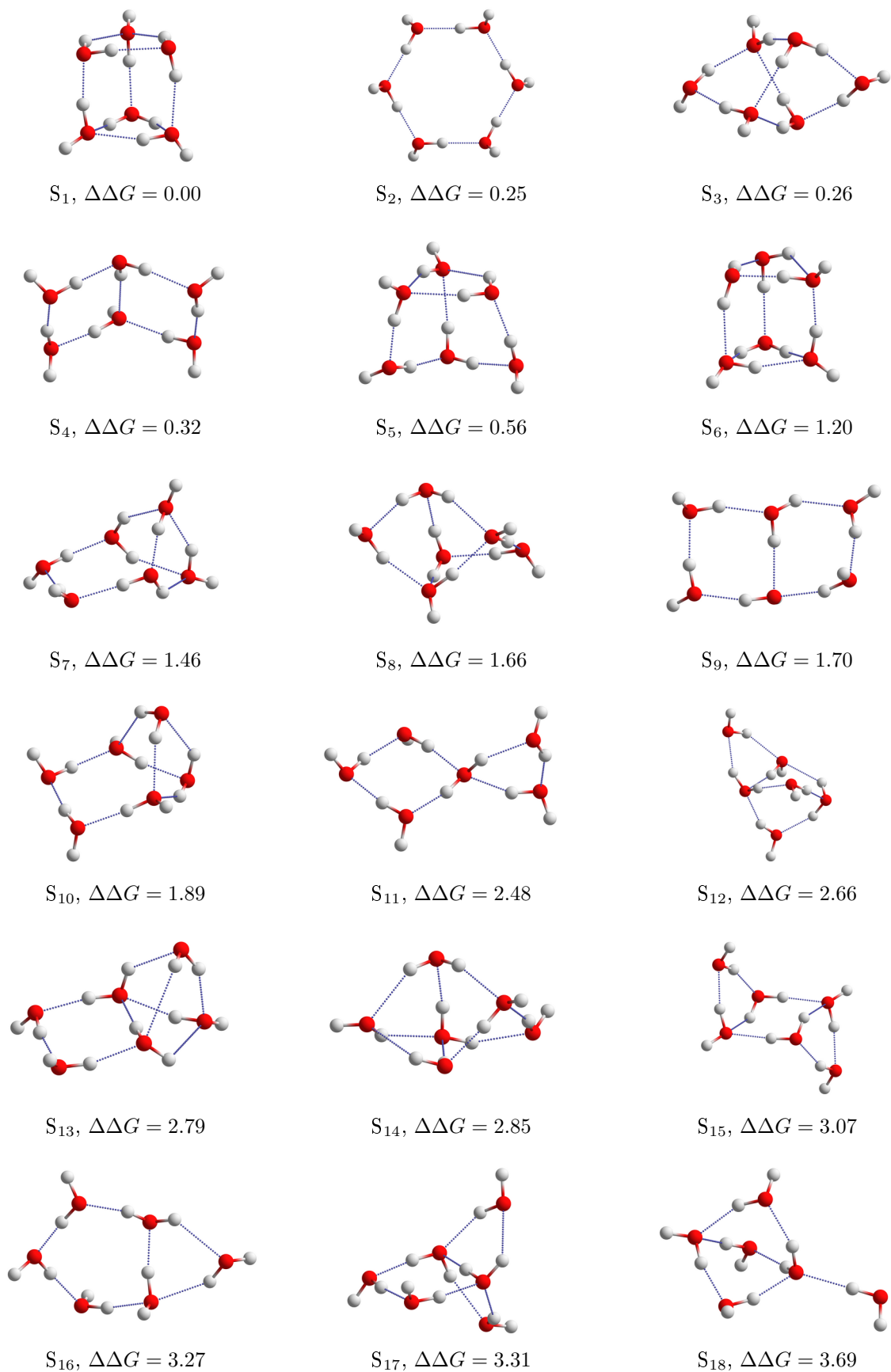


Figure 16: First 18 of the 22 refined water hexamer minima, labeled with their relative Gibbs free energy $\Delta\Delta G$ (kcal/mol) at 298.15 K.

The Boltzmann population (298.15 K) is dominated by a few low lying minima: the five most stable motifs hold $\sim 88\%$ of the population, all within ~ 0.6 kcal/mol of the global minimum. A subset is shown in Figure 16.

The final ensemble is only as reliable as the refinement method: faster methods (semiempirical, GFN2-xTB) give a cheap preliminary survey, but accuracy depends on the high level stage. Against reported DFT and MP2 data for small water clusters [5], the ASCEC protocol recovers the known dominant motifs at a reasonable cost on a standard workstation.

Computational efficiency. The hierarchical design reserves expensive quantum chemical work for the few structures that matter (Figure 17). The ASCEC search and xTB preoptimization explore the space and generate hundreds of candidates for a small fraction of the runtime, the water hexamer’s preliminary survey finishes in ~ 3 minutes, while rigorous DFT geometry optimization and, in the ultimate protocol, DLPNO-CCSD(T) energy refinement dominate ($\gtrsim 99\%$) the wall time.

Since COSMIC collapses the ensemble to a handful of motifs *before* these steps run, costly calculations are minimized while the dominant minima are retained, letting the user trade accuracy against cost (preliminary, rigorous, ultimate) predictably.

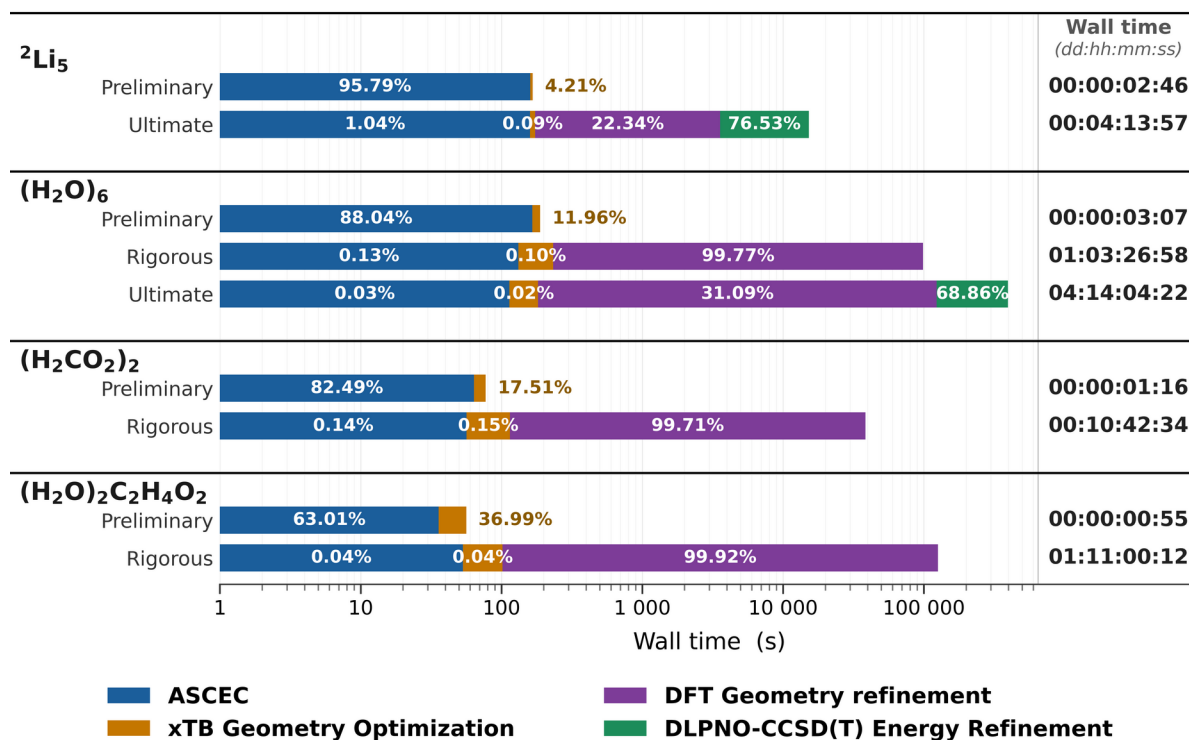


Figure 17: Wall time breakdown of the COSMIC-ASCEC pipeline stages for four benchmark systems, including the water hexamer $(\text{H}_2\text{O})_6$. Total wall time spent in each stage shown in logarithmic time axis.

Part II

Advanced Usage and Reference

7 Theoretical Background

7.1 Simulated Annealing Algorithm

The ASCEC module drives the exploration of the configurational space through a Markov Chain Monte Carlo (MCMC) protocol implemented within a simulated annealing framework [6]. To ensure statistical significance, the algorithm generates thousands of trial configurations. The quantum energy of every generated structure is calculated using efficient methods (supporting any theoretical level available in the external package, e.g., GFN2-xTB, PM3, AM1, HF).

The acceptance of a structural perturbation depends on the change in potential energy, $\Delta E = E_{new} - E_{old}$. If the structural change lowers the energy ($\Delta E < 0$), the move is accepted immediately. In the case of an energy increase ($\Delta E > 0$), the acceptance behavior depends on the chosen criterion:

Standard Metropolis Criterion [7] (Optional) When explicitly requested (via `--standard` flag), the algorithm follows the classical Boltzmann probability. An uphill move is accepted if a random number $\xi \in [0, 1]$ satisfies:

$$\xi < e^{-\frac{\Delta E}{k_B T}} \quad (1)$$

Modified Metropolis Criterion (Default) To mitigate kinetic trapping in local basins and prevent jumps to physically unrealistic high energy configurations ASCEC implements a modified acceptance condition [8]. Unlike the standard probabilistic approach, which compares the Boltzmann factor to a random number, this method evaluates the relative energy cost of the perturbation. The new structure is accepted if:

$$\Phi(\Delta E) < P(\Delta E) \quad (2)$$

where $P(\Delta E) = e^{-\Delta E/k_B T}$ is the temperature dependent Boltzmann probability, and $\Phi(\Delta E)$ represents the relative energy change defined as:

$$\Phi(\Delta E) = \frac{\Delta E}{|E_{new}|} \quad (3)$$

This criterion was found to be more adequate for this class of problems. By removing the stochastic element found in the standard Metropolis method, the algorithm avoids the accidental rejection of chemically viable structures due to random number generation, while still filtering out moves where the relative energy increase is drastic.

Temperature Schedules: The simulation temperature decreases over n_T steps to gradually reduce the acceptance of high energy states. A choice between a linear (decreasing T by a constant amount) and a geometric (decreasing T by a constant percentage) route is available:

- *Linear Quenching:* The temperature is reduced by a fixed amount dT at each step:

$$T_{n+1} = T_n - dT$$

- *Geometric Quenching:* The temperature is scaled down by a cooling factor f (in %):

$$T_{n+1} = T_n \times \left(1 - \frac{f}{100}\right)$$

7.2 Monte Carlo Moves

The configuration space is explored using three perturbation motions:

1. **Translation:** Random displacement of the center of mass.
2. **Rotation:** Random rotation around principal axes.
3. **Conformational (Optional):** Modification of internal dihedral angles.

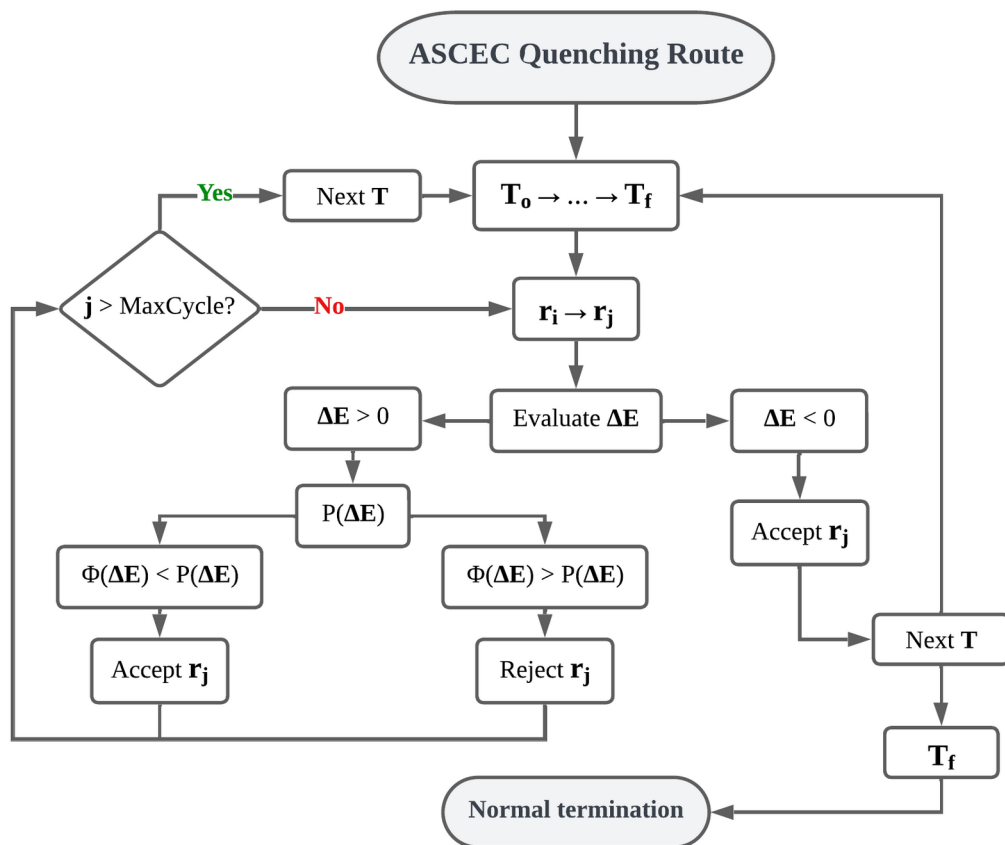


Figure 18: SA procedure using a modified Metropolis criterion.

7.3 Topological Clustering (COSMIC)

While stochastic exploration (ASCEC) is effective at traversing energy barriers, it inherently suffers from a data management limitation: the generation of highly redundant datasets. A typical annealing run yields hundreds of candidate structures, the vast majority of which converge to identical basins of attraction. To address this, the **COSMIC** module functions as a postprocessing discriminator designed to filter, validate, and cluster the raw output into unique chemical motifs.

The scope of this algorithm is to bridge the gap between stochastic chaos and ordered chemical insight. By reducing massive datasets into nonredundant families, it facilitates the manual revision of the Potential Energy Surface (PES) and ensures that only topologically distinct minima are subjected to high level quantum mechanical refinement.

7.3.1 The Physicochemical Feature Vector

Standard clustering methods often rely solely on geometric superposition (RMSD), which depends on atomic indexing [9]. To overcome this, we define the configurational identity of a structure i using a multidimensional scalar feature vector, $\mathbf{x}_i$. This vector aggregates continuous physicochemical descriptors derived from the quantum mechanical data. The **full vector** (up to 15 dimensions) is:

$$\mathbf{x}_i = (E_{elec}, V_{nuc}, G, E_{HOMO}, \Delta E_{HL}, \mu, A, B, C, \nu_{smallest}, \nu_{largest}, n_{HB}, \overline{d_{HB}}, \overline{\theta_{HB}}, \sigma_{d_{HB}}) \quad (4)$$

where the components represent energetics (E_{elec} , G); V_{nuc} : nuclear repulsion energy; E_{HOMO} : energy of the HOMO; ΔE_{HL} : HOMO–LUMO gap; μ : dipole moment; A, B, C : rotational constants; $\nu_{smallest}, \nu_{largest}$: smallest and largest vibrational frequencies; n_{HB} : number of hydrogen bonds, and hydrogen bonding descriptors $\overline{d_{HB}}, \overline{\theta_{HB}}, \sigma_{d_{HB}}$: geometrical parameters defining hydrogen bonds within each structure.

All structures are clustered together in a single pool by default; they are not grouped by hydrogen bond count beforehand. H-bond information is encoded indirectly through $\overline{d_{HB}}$ and $\overline{\theta_{HB}}$, allowing the clustering algorithm to account for H-bond patterns without imposing rigid partitions. The optional `--group-hb` flag restores per H-bond count grouping.

7.3.2 Weighted COSMIC Metric

Since the vector components possess disparate physical units, a per feature Z score standardization is applied $z_i^k = (\gamma_i^k - \bar{\gamma}^k)/\sigma^k$, $Z_i^k = w^k \times z_i^k$. The distance between two structures is then quantified using the Euclidean distance:

$$d(\mathbf{X}'_i, \mathbf{X}'_j) = \sqrt{(\mathbf{X}'_i - \mathbf{X}'_j) \cdot (\mathbf{X}'_i - \mathbf{X}'_j)} = \sqrt{\sum_{k=1}^{n_d} (Z_i^k - Z_j^k)^2} \quad (5)$$

where w^k represents user definable weights used to bias the clustering focus (e.g., prioritizing H-bonding patterns over soft vibrational modes).

7.3.3 Hierarchical Two Step Clustering

To ensure both computational efficiency and high structural fidelity, the clustering process operates via a hierarchical "funnel" protocol involving two distinct steps:

1. Step I: Physicochemical Clustering (Coarse Grained).

The algorithm first performs agglomerative hierarchical clustering using UPGMA (average linkage) based on the standardized z score feature vectors $\mathbf{x}$. This rapidly groups configurations that share similar energetics, electronic states, and general shapes into broad families. The dendrogram is cut at a user defined distance threshold τ to obtain a flat partition. This step drastically reduces the dataset size by merging redundant snapshots without performing expensive geometric alignments.

2. Step II: Optional RMSD Refinement (Fine Grained).

Optionally, a secondary geometric check is applied within each identified cluster. A Root Mean Square Deviation (RMSD) analysis [10] is performed to distinguish stereoisomers or isoenergetic conformers that map to identical scalar vectors (e.g., enantiomers or distinct proton ordering networks with identical energies). This ensures that subtle geometric variations are not lost during the vector based reduction.

7.3.4 Feature Vector Technicalities

Three implementation details govern how the raw physicochemical descriptors of Eq. 4 become the standardized vectors that are actually clustered.

Hydrogen bond descriptors. The components n_{HB} , $\overline{d_{\text{HB}}}$, $\overline{\theta_{\text{HB}}}$ and $\sigma_{d_{\text{HB}}}$ are computed from a strictly geometric hydrogen bond test applied to each structure. For every H covalently bound to a donor D (N, O or F), an acceptor A (also N, O or F) is counted as hydrogen bonded when the $\text{H}\cdots\text{A}$ distance lies in $[1.4, 3.2]$ Å and the $\text{D}-\text{H}\cdots\text{A}$ angle is at least 30° . These are the same criteria used throughout the cluster literature. Note that this detection is deliberately *stricter* than the broad, count based donor/acceptor heuristic used only to *size the box* (Section 9): the box heuristic overcounts on purpose to leave room, whereas the feature vector records the H-bonds a structure actually forms.

Small standard deviations. Z-standardization divides each descriptor by its standard deviation across the pool. If a descriptor is essentially constant, that division would inflate tiny numerical noise into enormous z -scores and dominate the Euclidean distance. COSMIC guards against this by *dropping* any feature column whose population standard deviation falls below a minimum threshold, or whose total spread across the pool is smaller than a user set absolute tolerance. A dead column carries no discriminating information, so removing it entirely (rather than retaining it as noise) keeps the distances finite and meaningful; the dropped features are listed in the verbose log.

Pearson similarity floor. For two weighted, standardized z score vectors the Euclidean distance and the Pearson correlation r between their profiles are linked by $d^2 = 2 N_f (1 - r)$, where N_f is the effective number of active features (the sum of the column variances after weighting). COSMIC inverts this, $r = 1 - d^2/(2N_f)$, to report, for every cluster, the lowest percentage similarity between any member and its representative — the **similarity floor**. A high floor (e.g. $> 90\%$) certifies that a family is internally homogeneous, giving a direct, quantitative check on the clustering quality.

Partial weights (preliminary runs). By default every feature carries equal weight ($w^k = 1$), which keeps COSMIC method agnostic. The flag `--partialweights` instead applies a tuned, reliability based profile that downweights the descriptors most sensitive to method noise and atom indexing — dipole moment (0.6), the hydrogen bond descriptors (0.7), the HOMO energy and highest vibrational frequency (0.85) — while leaving the deterministic geometric quantities (electronic energy, nuclear repulsion, rotational constants) at 1.0, so that geometrically identical structures do not look artificially distinct at a noisy level of theory. The web input generator adds `--partialweights` automatically to the COSMIC step of *preliminary* runs (where the whole pipeline stays at the semiempirical/xTB level); it is a deliberate choice for that specific case, not a blanket setting for every semiempirical calculation. An explicit `--weights` argument always layers on top.

7.3.5 A Dynamic Feature Vector

The feature vector is dynamic, not a fixed template. COSMIC reads whatever properties each quantum chemistry output actually contains and clusters on the **largest feature pool the dataset supports**: electronic and orbital energies, the HOMO–LUMO gap, dipole, nuclear repulsion, rotational constants, hydrogen bond descriptors, and, when the calculation included frequencies, the Gibbs energy and vibrational frequencies. Any descriptor a structure lacks, or that carries no variance across the pool (the small standard deviation rule above), is left out automatically. The motifs are therefore always identified from the richest description available for that dataset, with no need for the user to declare in advance which properties exist.

7.3.6 Quality Control

Beyond clustering, the module serves as a topological filter. It screens for the presence of **one or more imaginary frequencies**, indiscriminately identifying first order saddle points (transition states) and higher order instabilities. If such a structure clusters tightly with a true minimum, it is discarded as a redundant, nonconverged distortion. However, if it forms an isolated cluster, it is flagged as a unique topological feature and output for reoptimization, ensuring that potential basins are not overlooked merely due to incomplete convergence.

7.3.7 Automatic Threshold Selection

By default COSMIC does **not** ask you to pick τ . It is detected per case from the merge heights of the UPGMA [11] tree, because no single fixed value suits every system.

Heights are full distances: no factor of $\frac{1}{2}$. COSMIC builds the tree with SciPy's average linkage routine and cuts it on the *distance* criterion, so the dendrogram height axis and every threshold, τ , the merge heights, and the empirical 2.0, are the full average intercluster distance of Eq. 5. Textbook UPGMA usually plots a node at *half* that value, the branch length from the node down to its leaves; SciPy, and therefore COSMIC, does not. A height read off a COSMIC dendrogram is the distance itself, directly comparable with τ .

Three criteria are available:

- **Knee** [12] (*the default*). Sort the merge heights and plot them against merge number. The curve stays nearly flat while structures belonging to one motif fuse, then climbs steeply once distinct motifs begin to merge; the elbow marks that transition and is the natural place to cut. COSMIC rescales both axes to $[0, 1]$, joins the first and last merges with a secant, and takes the merge lying furthest from it. The cut goes midway between that merge and the next, so the elbow merge itself still counts as within cluster. The knee is trusted only when the tree can support it — at least eight merges, a height range spanning a factor of five or more, and a resulting cluster count between two and $n/2$ — and otherwise COSMIC falls back to the empirical value.
- **Empirical**, $\tau = 2.0$. The fallback when a set is too small for the knee to be reliable, and also an **upper limit**: a detected knee above 2.0 is capped there. *There is no formal justification for this number.* It was determined empirically over many unrelated systems, and across 50 independent preliminary runs on the water hexamer the average knee came to 1.93, close enough that either value delivered much the same surface. Its scale is not arbitrary, though: the distance per component is $\tau/\sqrt{N_f}$, which for a full 15 component vector sits just above 0.5 standard deviations and for the sparsest 5 component one just below 1.0.
- **Mojena** [13]. Computed and drawn on the validation plots as a diagnostic, but not part of the fallback chain: across many test cases it proved too inconsistent to drive the selection.

The threshold actually applied, and where it came from (**knee**, **empirical** or **user**), is reported in the verbose log and on the diagnostic plot. To take the choice back, pass a number to `--threshold`, which is used as given and never capped, or `--th=knee` to keep the automatic detection with the 2.0 ceiling switched off.

8 Input File Format

The ASCEC input file (typically saved with an `.asc` extension) is a comprehensive "master" document. To make it easy to read and edit, it is logically divided into three main blocks: the **Simulation Parameters**, the **Execution Protocol**, and the **Embedded QM Templates**.

Note: The easiest way to generate this file without worrying about formatting is by using the [ASCEC Web Input Generator](#).

8.1 Block 1: Simulation Parameters and Coordinates

The first block of the input file relies on strict positional formatting. It defines the physical conditions of the annealing simulation and the initial coordinates of the molecules.

```
# Input for water pentamer Simulation

1 10          # Line 1: Simulation Mode & Number of Config
              # 1: Annealing; 0: Random configurations
7.0          # Line 2: Simulation Cube Length (Angstroms)
2            # Line 3: Annealing Quenching route
              # (1: Linear, 2: Geometrical)
100.0 10.0 50 # Line 4: Linear Quenching Parameters (Ti, dT, steps)
500.0 5.0 100 # Line 5: Geometric Quenching Parameters (Ti, %, steps)

3000 10      # Line 6: Max MC Cycles per T and floor value
1.0 1.0      # Line 7: Max Displacement (A) & Max Rotation (radians)
0 120        # Line 8: Conformational sampling (%) & Max dihedral (deg)

3 xtb        # Line 9: QM Program Index & alias
              # (1: Gaussian, 2: ORCA, 3: xTB)
GFN2-xTB     # Line 10: Hamiltonian & Basis Set
1 4          # Line 11: nprocs (QM calculations and ASCEC evaluation)
0 1          # Line 12: Charge & Spin Multiplicity

5            # Line 13: Number of Molecules (nmo)

# Lines below: Molecule Definition
# (Num Atoms, Label, then xyz coordinates, separated by *)

*
3
Water1
O   -0.294733  -0.218200  0.154167
H   -0.017333  0.674700  0.408567
H    0.312067  -0.456500  -0.562733
*
3
Water2
O   -0.294733  -0.218200  0.154167
H   -0.017333  0.674700  0.408567
H    0.312067  -0.456500  -0.562733
*
... (Water 3-5)

*
```

Table 3: Detailed description of ASCEC input parameters (Lines 1-13).

Line	Parameters	Description
1	[mode] [nconfigs]	1 : Annealing, 0 : Random Generation. nconfigs is only used if mode is 0.
2	box_length	Length of the cubic simulation box edge in Å.
3	route	Temperature schedule: 1 (Linear) or 2 (Geometric).
4	[Ti] [dT] [steps]	Linear: Start Temp (K), Step size (K), Number of steps.
5	[Ti] [fac] [steps]	Geometric: Start Temp (K), Reduction factor (%), Steps.
6	[max] [floor]	Maximum Monte Carlo cycles per temperature step and floor value as it decreases (0.1) with each step.
7	[disp] [rot]	Max translation distance (Å) and max rotation (radians). Maximum displacement for each mass center, and maximum rotation of the principal axes of each molecule. For monatomic species the rotation has no effect and the value is ignored.
8	[prob] [dih]	Probability of conformational move (%) and max dihedral change (°).
9	[code] [alias]	QM Program ID (1 =Gaussian, 2 =ORCA, 3 =xTB) and the command alias to run it.
10	[method] [basis]	Hamiltonian/Method (e.g., gfn2 , b3lyp) and basis set (e.g., def2-SVP). For semiempirical methods the basis field is ignored; consult the ORCA manual for the correct syntax.
11	[cpu_Q] [cpu_A]	Processors passed to the QM program, and (optional) processors for ASCEC logic and parsing.
12	[chg] [mult]	Total system Charge and Spin Multiplicity.
13	nmol	Number of molecules defined. For atomic clusters each atom must be considered a molecule. If atoms interacting with molecules are considered, then each atom not being part of a molecule must be considered a molecule.

8.2 Block 2: The Automated Protocol and Embedded Templates

Following the molecular coordinates, the file contains the **#Protocol** block and the **embedded QM templates**. The protocol is a sequence of comma separated steps. The example below is a complete ultimate protocol; stopping after the first **cosmic** gives a preliminary run, and after the second a rigorous one. Using dot instead of comma pauses the protocol for review and resume.

```
# Protocol
# (comma = continue, dot = pause; --flags apply to the whole run)

.asc,
--maxprint,
r3 --concurrent=3,
opt --concurrent=8 input1,
cosmic -j4,
ref --concurrent=2 input2,
cosmic --th=opt -j4,
eref input3,
cosmic --th=opt -j4
```

Workflow: Annealing → Opt → COSMIC → Ref → COSMIC → Eref → COSMIC

Each optimization or refinement step names a template (`input1`, `input2`, `input3`) that is **embedded at the bottom** of the same file, using the header and placeholder syntax of Section 8.2.1. The three tiers here are GFN2-xTB (`#xtb`), wB97X-D3BJ/def2-TZVPP with frequencies (`#orca`), and a DLPNO-CCSD(T)/aug-cc-pVTZ single point (`#orca`):

```
#xtb input1
#name
#rescue(GFN2-xTB/freq)
# GFN2-xTB Opt Tight
# nprocs 1
# maxiter 200

#orca input2
#name
#rescue(HF-3c/freq)
! wB97X-D3BJ def2-TZVPP def2/J RIJCOSX DEFGRID3
! Opt Freq TightOpt TightSCF
%pal
  nprocs 12
end
%geom
  maxiter 120
end
* xyz 0 1
#
*

#orca input3
#name
! DLPNO-CCSD(T) aug-cc-pVTZ def2/J AutoAux RIJCOSX TightPNO TightSCF
%scf
  MaxIter 250
end
%pal
  nprocs 1
end
%maxcore 50000
* xyz 0 1
#
*
```

A template is an ordinary QM input file for the target program, with a few `#` placeholders that ASCEC fills in for every structure. Each template, embedded at the bottom of the `.asc` file, begins with a header line that names the program *and* gives a label the protocol refers to:

```
#orca <label>      #gaussian <label>      #xtb <label>
```

A protocol step such as `opt input1` or `ref input2` then selects the block whose label matches; the header program also fixes the file extension written for that stage (`.inp`, `.xtb` or `.com`). The placeholders ASCEC recognizes are shown in Table 4

Placeholder	Meaning
<code>#orca xtb <label></code>	Block header: declares the QM program alias and the label to call
<code>#name</code>	(Optional, ORCA and xTB) replaced by the structure's comment/-filename, so each generated input is traceable.
<code>#</code>	(ORCA) a bare hash, placed between <code>* xyz <chg> <mult></code> and the closing <code>*</code> , marks where the Cartesian coordinates are inserted.
<code>!name</code>	(Optional, Gaussian) the Gaussian spelling of <code>#name</code> : a comment line naming the structure.
<code>!</code>	(Gaussian) a bare exclamation mark on its own line marks where the coordinates are inserted.
<code>#rescue(method[/freq][,num])</code>	Method used to reoptimize failed structures in the redo loop (e.g. <code>#rescue(HF-3c/freq)</code> , <code>#rescue(GFN2-xTB/freq)</code>).
<code># <flag> [args]</code>	(#xtb blocks only) any xTB command line option, passed through as <code>--flag args</code> (e.g. <code># alpb water</code>). See Section 8.2.2.

Table 4: Template placeholders substituted by ASCEC.

8.2.1 Input Templates

I. ORCA template. Written in native ORCA syntax. The bare `#` sits between `* xyz <chg> <mult>` and the closing `*`, exactly where the coordinate block is injected:

```
#orca input1
#name
#rescue(Native-GFN2-xTB/num)
! Opt GFN2-xTB TightSCF NumFreq PAL8
%geom
  maxiter 5000
end
* xyz 0 1
#
*
```

II. xTB template. For standalone xTB the run directives are given as `#`-prefixed lines (method, processors, iterations); ASCEC supplies the geometry automatically, so no `* xyz *` block is needed:

```
#xtb input1
#name
#rescue(GFN2-xTB/freq)
# GFN2-xTB Opt Tight
# nprocs 1
# maxiter 200
```

8.2.2 Advanced: the Full xTB Flag Set

Unlike ORCA or Gaussian, xTB has no keyword line and no `%`-block syntax: *every* setting is a command line flag. An `#xtb` template block is therefore not an input file but a description of

the command ASCEC will build, and each #-prefixed line contributes to it. Two layers are read, in this order.

Named directives. The first line without a recognized keyword is the **method/job line**, and a handful of directives carry names that do not match their flag. These are translated:

Directive	Becomes	Meaning
# GFN2-xTB	--gfn 2	GFN Hamiltonian. Also GFN1-xTB, GFN0-xTB and GFN-FF (→ --gfnff). GFN2 is the default.
# ... Opt <level>	--opt <level>	Geometry optimization. The level is optional and may be Crude, Sloppy, Loose, Lax, Normal, Tight, VTight (or VeryTight) or Extreme.
# ... Freq	--hess	Vibrational analysis. NumFreq is accepted as a synonym: xTB has an analytical Hessian and uses it either way.
# nprocs N	--parallel N	Cores for this job.
# maxiter N	--cycles N	Maximum optimization cycles.
# charge N	--chrg N	Total charge (omitted when neutral).
# uhf N	--uhf N	<i>Unpaired electrons</i> (2 <i>S</i>), not the multiplicity: a triplet is # uhf 2.

Table 5: Named #xtb directives and the xTB flags they produce.

Everything else. Any other # <flag> [args] line is passed through verbatim as --flag args, so the complete xTB command line is reachable from the template without ASCEC needing to know the flag. This is how solvation, electronic temperature, accuracy, and every other xTB option are requested:

```
#xtb input1
#name
#rescue(GFN2-xTB/freq)
# GFN2-xTB Opt Tight
# nprocs 1
# maxiter 200
# uhf 1
# alpb water
# etemp 400
# acc 0.5
```

which ASCEC turns into:

```
xtb <structure>.xyz --gfn 2 --opt tight --parallel 1 --cycles 200 --uhf 1 --alpb water --etemp
400 --acc 0.5
```

Common cases are the implicit solvent models, # alpb <solvent> (ALPB, the current recommendation) and # gbsa <solvent> (GBSA), optionally with a reference state, e.g. # alpb water bar1M; the # cpcmx <solvent> extended conductor like model; and single flag switches such as # json or # lmo, which need no argument.

Four rules govern the passthrough:

- **Reserved names are never passed through.** name, rescue, nprocs, maxiter, charge/chrg, uhf, mult and freq are consumed by Table 5; namespace is reserved because ASCEC in-

jects it to keep each structure’s scratch files apart, and overriding it would break output collection.

- **The protocol wins on resources.** `--parallel` and `--cycles` are reissued from the workflow settings (index line 11 and the stage `--concurrent` bookkeeping), so `# nprocs` in the template is a default, not a guarantee. Charge and multiplicity likewise come from the annealing parameters, which are the single source of truth for the system.
- **Lines carrying shell metacharacters are ignored.** The flags end up inside a generated launcher script, so a directive containing `;`, `|`, `&`, backticks, quotes or redirection is dropped rather than executed.
- **The rescue Hessian inherits the environment.** When `#rescue(GFN2-xTB/freq)` fires on a structure that failed to converge, the extra flags are reapplied to the Hessian run, so the frequencies are computed on the same potential energy surface as the optimization. Job type flags (`--opt`, `--md`, `--gfnff`, ...) are excluded, since the rescue run sets its own job type.

From the web generator. In **Advanced** mode (Section 4) the Geometry Optimization, Geometry Refinement and Energy Refinement blocks each grow an extra row as soon as their level of theory is set to standalone xTB: an **Implicit Solvent** model (ALPB, GBSA or CPCM-X) with its solvent list, and a free text **Extra xTB flags** field that takes a comma separated list without the leading dashes (`etemp 400, acc 0.5`). The generator writes them as the `#` directives above, filtering the reserved names and the unsafe characters listed here, and the template preview updates as you type, so it doubles as a check on what the command line will look like. Anything the controls do not cover can still be typed straight into the preview, or into the `.asc` file afterwards.

Understanding the Execution Pipeline

Upon parsing the `.asc` file and its embedded templates, ASCEC executes the stages in order. It begins with triplicated **Annealing** (`r3 --concurrent=3`) to generate raw candidates, then **preoptimizes** the whole pool with the GFN2-xTB `input1` template (`opt`) and applies the first **COSMIC** screening.

The surviving motifs are **refined** at `wB97X-D3BJ/def2-TZVPP` with frequencies (`ref input2`) and screened again, the `--th=opt` on this second `cosmic` re uses the threshold resolved by the first. Finally an **energy refinement** (`eref input3`) computes DLPNO-CCSD(T) single points on the DFT geometries, and a last **COSMIC** pass delivers the unique minima and their Boltzmann populations. Ending the protocol after the first or second `cosmic` yields the preliminary or rigorous description instead.

***Note:** From this point forward, the advanced sections of this manual explain each of these stages in detail: box sizing, the feature vector and thresholds, the templates, and how to run each function on its own.*

9 Box Size Suggestion

ASCEC anneals inside a hard walled cubic box of side L . Centers of mass are confined within the walls and any move that puts one outside is rejected. The choice of L matters: too small and the molecules are compressed, cannot rotate freely, and pick up artificial steric strain; too large and the search wastes its time in empty space.

One parameter controls it, the **packing fraction** ϕ , given as a percentage. It is the fraction of the box the system effectively occupies, and the box follows from it:

$$L = \left(\frac{V_{\text{eff}}}{\phi} \right)^{1/3} \quad (6)$$

Because the volume scales as L^3 , doubling the number of molecules grows L by only $2^{1/3} \approx 1.26$. A rule linear in the number of molecules would oversize the box badly, and increasingly so with system size.

9.1 How V_{eff} is Estimated

Each atom is treated as a hard sphere of its covalent radius; the spheres of one molecule are assembled in their real geometry and the **convex hull** of that surface gives the molecular volume V_{mol} . No shape is assumed: a single atom gives a sphere, a planar molecule a flattened polyhedron. The molecule’s **extent** E , the largest distance between any two atoms plus their radii, is its effective diameter whatever its orientation.

From these, ASCEC picks one of two routes automatically:

Method	Applies when	V_{eff}	Rationale
A	The system can form hydrogen bonds	$V_{\text{mol}}^{\text{total}} + V_{\text{HB}}^{\text{total}}$	Hydrogen bonds are directional and rigid, holding molecules in open arrangements. The extra volume accounts for the network they impose.
B	No hydrogen bond donors or acceptors	L_{diag}^3	With no directional constraint, the molecular extents laid end to end and read as a cube diagonal give a natural size.

Table 6: The two routes to the effective volume. Method A is preferred whenever it applies.

When Method B overestimates. Laying every molecular extent end to end is a deliberately generous upper bound: it assumes the fragments line up along one diagonal rather than packing together. For a few fragments that is a good approximation and the box is close to what the system needs. As the number of fragments grows, the sum of extents keeps growing linearly while a real cluster packs in three dimensions, so Method B returns a box noticeably larger than necessary. For a system with no hydrogen bonding and many fragments, treat the suggestion as an upper limit: check it against the visual test in the web generator and reduce it, or set L yourself, rather than annealing in empty space.

9.2 Choosing the Packing Fraction

The same ϕ means the same thing for water, methane, argon or a mixed system, because ASCEC computes V_{eff} and selects the method without intervention. The web generator’s presets set it for you: 30% for preliminary runs and 20% for rigorous and ultimate, the looser box giving molecules more room to reorganize during a longer search. Driving the command line directly, these ranges are useful guidance:

ϕ	Box	Use
10–15%	very loose	Isolated clusters; fragments explore independently before associating.
20%	loose	Cluster formation. The default for rigorous and ultimate runs.
30%	moderate	Quick surveys. The default for preliminary runs.
40%+	dense	Condensed phase or liquid like packing; rarely wanted for cluster work.

Table 7: Packing fraction guidance.

The command `ascec input.asc box` prints the analysis and the suggested length for several packing fractions without running anything else, and the web generator previews the box in three dimensions with a test that mimics the random rigid body moves, so an unsuitable box is visible before any time is spent on it.

10 Standalone Functions: Understanding Each Stage

Most users drive COSMIC-ASCEC through the automated pipeline of Section 12, where a single command chains annealing, optimization, clustering and refinement. This section takes that chain apart and runs each stage on its own. That is what you want when you need to inspect an intermediate result, recluster an existing set with different settings, or simply watch what the pipeline does on your behalf.

Two properties of standalone mode shape everything below. First, these commands **write input files and launcher scripts; they do not run the quantum chemistry themselves**. You run the launcher. Second, **sampling is stochastic**: each run starts from its own random seed, so the number of accepted configurations, and the number of motifs that survive clustering, differ from one run to the next. Repeating the example will not reproduce a particular count, and no count is quoted here for that reason.

The worked example is the water hexamer, and everything it needs ships with the software in `examples/w6/standalone/`: the input `w6.asc`, holding six water molecules and the annealing parameters of Block 1 (Section 8.1) with no protocol block and no embedded templates, and one template per calculation stage. Each template is shown where the stage that reads it is described.

Because those templates are separate files here rather than blocks embedded in the `.asc`, it is the **file extension** that tells ASCEC which program to write for: `.inp` for ORCA, `.com` for Gaussian, `.xtb` for standalone xTB. What goes inside them is the same either way, and is described in Section 8.2.1.

10.1 Annealing

Before launching, it is worth checking the box. The `box` subcommand prints the effective volume analysis of Section 9 and a table of recommended edge lengths, without running any quantum chemistry. This analysis depends only on the molecules you supplied, so unlike everything that follows it gives the same answer every time:

```
ascec w6.asc box
```

```
(base) :~/.../examples/w6/standalone$ ascec w6.asc box
```

Box length analysis

1. Molecular volume analysis:

```
-----
Number of molecules to place: 6
Total molecular hull volume: 10.50 Å³
Total H-bond network volume: 67.86 Å³
Effective volume: V_mol + V_HB = 78.36 Å³
```

3. Current box analysis:

```
-----
Cube's length = 5.00 Å
Current effective packing: 62.7%
+ Molecular: 8.4%, H-bond network: 54.3%
Current free volume: 47 Å³ (37.3%)
Largest molecular extent: 2.15 Å
```

Extremely dense - may cause steric clashes

4. Recommendation:

>>> Default suggestion: 7.3 Å (20% packing) <<<

- For isolated clusters: 8.1 Å (15% packing)
- For cluster formation: 7.3 Å (20% packing)
- For network studies: 6.8 Å (25% packing)

Use --box<P> to use a specific packing %
(e.g., --box10) for a 10%

The 5.0 Å box written into `w6.asc` is far too dense for this system, so we let ASCEC rewrite the edge length to a target packing fraction with the `--box` flag, and at the same time replicate the run with `r#` — here `r3`, for a triplicated run:

```
ascec w6.asc r3 --box20
```

```
Using recommended box size: 7.3 Å (20.0% effective packing)
Creating 3 replicated runs in 'annealing/'...
Created launcher script: launcher_ascec.sh
```

To run all simulations sequentially, use:
`./launcher_ascec.sh`

Nothing has been computed yet. The command wrote three input files, one per replica, and a script that runs them in series. Running that script is a separate act:

```
bash launcher_ascec.sh
```

Each replica starts from the “**Big Bang**” configuration: all centers of mass superimposed at the box center, the worst possible, and therefore least biased, starting point. ASCEC makes up to 100 random moves to find a first configuration whose energy is computable without atomic clashes, then anneals from there, writing the accepted structures, the energy trajectory (**tvse**), and the lowest energy structure to each replica folder for later inspection. If all 100 attempts fail, the run stops with diagnostics instead of a trajectory.

The launcher writes no energy plots. Each replica folder ends up with its **tvse_<seed>.dat** trajectory, but the diagrams of Figures 19 and 20 are produced by the replication stage of the automated pipeline, which the launcher bypasses. Generate them afterwards from the data that is already there:

```
ascec diagram
```

This writes **tvse_<seed>.png** beside every trajectory it finds and a combined **tvse_r3.png** for the replica group. The first accepted configurations sit far above the annealed ones and flatten everything after them into a single line; **ascec diagram --scaled** crops the energy axis to drop that initial descent and replots the same data on a readable scale.

```
/
├── annealing/
│   ├── tvse_r3.png
│   ├── w6_1/
│   ├── w6_2/
│   └── w6_3/
├── launcher_ascec.sh
└── w6.asc
```

10.2 Geometry Optimization

The annealing pool is large and highly redundant, so before any expensive refinement every accepted structure is relaxed with the fast semiempirical GFN2-xTB method [2]. This preoptimization must preserve the topology of the PES, letting each structure fall into its nearest local minimum without merging distinct basins or discarding valid low energy isomers.

The template that does it is **opt_xtb.inp**. It reaches GFN2-xTB through ORCA rather than through the xTB binary, which keeps one program in charge of the whole stage:

```
#name
! NATIVE-XTB2 Opt
%geom
  maxiter 100
end
* xyz 0 1
#
*
```

NATIVE-XTB2 is ORCA’s own GFN2-xTB implementation, available from ORCA 6.1 onward and the reason no external xTB installation is needed here. On ORCA 5.x the keyword is **XTB2**

instead, and that route does call an external `xtb` binary. The two `#` placeholders are the ones described in Section 8.2.1: `#name` becomes a comment naming the structure, and the bare `#` between `* xyz 0 1` and the closing `*` is where the coordinates are injected.

The `opt` command scans for the annealing results, fills the template once per accepted structure, and writes the decks plus a launcher:

```
ascec opt opt_xtb.inp
```

With replicas present it first asks whether to treat each replica separately or to combine them into one numbered pool. Combining, answer `c`, is the usual choice: the replicas are independent samples of the same surface, and the clustering wants them in one pool. The decks land in `geometry_optimization/` as `opt_conf_1.inp`, `opt_conf_2.inp` and so on. If ORCA is on your `PATH` the launcher is generated for you, with the ORCA and OpenMPI environment already set; otherwise pass your own launcher as a second argument, `ascec opt opt_xtb.inp launcher_orca.sh`, and its environment block is reused. Without either, the decks are still written and only the launcher is missing.

```
/
├── annealing/
├── geometry_optimization/
│   ├── opt_conf_1.inp
│   ├── ...
│   ├── opt_conf_<N>.inp
│   └── launcher_orca.sh
├── w6.asc
└── opt_xtb.inp
```

```
cd geometry_optimization && bash launcher_orca.sh
```

10.3 Sorting the Results

The calculations leave one output file per structure scattered beside its input. The `sort` command groups each structure into its own subfolder, writes `combined_results.xyz` with the optimized geometries, records a wall time summary, and collects the output files into the `cosmic/` folder where the clustering will read them:

```
ascec sort
```

Use `--nosum` to skip the summary, or `--justsum` to write the summary without moving anything. The collected folder is named after the program and the number of structures in it, so an ORCA pool of `<N>` structures becomes `orca_out_<N>`:

```
/
├── annealing/
├── geometry_optimization/
├── cosmic/
│   ├── orca_out_<N>/
│   │   ├── opt_conf_1.out
│   │   ├── ...
│   │   └── opt_conf_<N>.out
├── w6.asc
```

10.4 COSMIC Clustering

With the outputs collected, the clustering runs from the `cosmic/` folder `sort` just created — the level *above* `orca_out_<N>/`, not inside it:

```
cd cosmic && cosmic -j4
```

COSMIC parses each output into the physicochemical feature vector of Equation 4, standardizes it, builds the UPGMA tree and cuts it at the resolved threshold, keeping the lowest energy structure of each cluster as its representative. The representatives get a folder of their own, `candidates_<NN>/`; `clustering_summary.txt`, the dendrograms and the per cluster data land beside it in `cosmic/`. Two arguments control the run:

- **--th (threshold):** the clustering granularity, the maximum distance in feature space for two structures to count as the same motif. The default, `--th=auto`, detects it per case; Section 7.3.7 explains how, and why the detected value is capped. Pass a number instead, as in `--th=2`, to set it yourself.
- **-j (jobs):** cores used to parse the output files. Omitted, COSMIC uses all of them.

A lower threshold creates tighter, more homogeneous clusters; a higher one yields broader families. Table 8 gives the ranges worth reaching for when you override the automatic choice.

Granularity	Value (τ)	Typical Use Case
Strict / Tight	0.5 - 1.5	Small/Rigid Systems. Distinguishes between specific rotamers or isoenergetic conformers with slight dihedral changes.
Standard	2.0 - 2.5	General Purpose. Default for molecular clusters (e.g., water hexamers). Balances distinct topology against vibrational noise.
Loose / Broad	> 3.0	Large/Flexible Systems. Groups broad structural families (e.g., folding patterns) while ignoring local side chain fluctuations.

Table 8: Recommended clustering threshold (τ) values.

Adding `--rmsd` enables the optional Step II geometric check inside each family. What the feature vector contains, how it is weighted and how the threshold is chosen are all covered in Section 7.3.

What the survivors are called. Every clustering pass promotes its input one rung up a three level ladder, and the name records the stage a structure came out of, so nothing has to be traced by the directory it was found in. This first pass reads unlabelled optimization output, so it emits the lowest rung:

Pass	Reads	Writes	What the set is
First	opt_conf_*.out	candidates_<NN>/	The distinct motifs present in the preoptimized pool.
Second	candidate*_opt.out	motifs_<NN>/	The same set after refinement at the higher level.
Third	motif*_opt.out	u_motifs_<NN>/	Unique motifs, after a final single point energy refinement.

Table 9: The three rungs of representatives. Each folder holds <label>_01.xyz, <label>_02.xyz and so on, ordered by energy, with the count in the folder name. The third rung needs an energy refinement stage, which exists in protocol mode only.

Caching. Parsing hundreds of quantum chemistry logs is COSMIC’s main bottleneck, so the parsed descriptors are cached to a `data_cache_XXXXXX.pkl` file on the first run. Reclustering at a different threshold then reuses the cache instantly, which is what makes standalone reclustering cheap; pass `--reprocess-files` (or delete the cache) whenever the underlying outputs change.

```

/
├── cosmic/
│   ├── orca_out_<N>/
│   ├── candidates_<NN>/
│   │   ├── candidate_01.xyz
│   │   ├── ...
│   │   └── all_candidates_combined.xyz
│   ├── dendrogram_images/
│   ├── extracted_clusters/
│   ├── extracted_data/
│   ├── clustering_summary.txt
│   └── data_cache_<hash>.pkl
└── w6.asc

```

10.5 Geometry Refinement

The representatives that survived clustering are the only structures worth spending real quantum chemistry on. The `ref` stage is the refinement counterpart of `opt`: it reads them and writes one high level input each. Run it from the `cosmic/` folder holding `candidates_<NN>/`, with the B3LYP template `ref_b3lyp.inp`:

```

#name
! B3LYP 6-31G
! Opt Freq
%pal
  nprocs 8
end
%geom
  maxiter 80
end
* xyz 0 1
#
*

```

```
ascec ref ref_b3lyp.inp
```

The decks are written to `geometry_refinement/`, each named after the representative it came from: `candidate_01_opt.inp`, `candidate_02_opt.inp` and so on, so a structure can be traced back through every stage by its name alone. As before, a launcher is generated when ORCA is on the PATH, and as before, running it is your step.

The same command accepts a Gaussian template instead. Handing it `ref_b3lyp.com` writes `.com` decks rather than `.inp` ones, with `!name` carrying the structure name and a bare `!` marking where the coordinates go, which are simply the Gaussian spellings of the two ORCA placeholders. One practical difference: launcher generation is automatic only for ORCA, so a Gaussian stage needs a launcher passed as the second argument.

```
!name
%nprocshared=8
%mem=4GB
#p B3LYP/6-31G Opt Freq

Water hexamer standalone refinement

0 1
!
```

Once the refinement finishes, the cycle closes as it did before: `ascec sort` to collect the outputs, which this time gathers them into `cosmic_2/`, then a second clustering pass over them:

```
cd cosmic_2 && cosmic --th=opt -j4
```

`--th=opt` reuses the threshold the first pass resolved, so the refined set is partitioned on the same footing as the preoptimized one instead of having a new threshold detected from a much smaller sample. This pass promotes the set one rung, writing `motifs_<NN>/` together with the Boltzmann populations of the refined structures. For most work that is where the chain ends; the third rung of Table 9 needs an energy refinement stage, which exists in protocol mode only.

10.6 What Standalone Mode Does Not Do

Every command in this section writes files. None of them runs a quantum chemistry program, and none of them judges what came back. That division is deliberate, and it is the whole of the difference between standalone and protocol mode:

- **Manual Calculations**
- **No failure is retried**

If you want any of that, use the protocol of Section 12. It runs these same stages in this same order, and adds between them the validation and repair that this section does without.

Standalone mode exists for the opposite purpose. Each command stops at a point where you can look: at the box analysis before committing to a run, at the trajectories before optimizing, at a generated deck before submitting a few hundred of them, at one clustering before trying another threshold on the same cached descriptors. It is the mode to reach for while you are learning how the method behaves or showing someone else, and the mode to leave behind once you want results you can trust without watching them being made.

11 Output Files

A successful run of ASCEC with the annealing option (“1” in Line 1) with candidate structure clustering using the COSMIC script for this example protocol will generate several files.

11.1 Print Levels: miniprint and maxprint

Every workflow runs at one of two print levels. **Miniprint** (the default) cleans up the bulky per structure QM I/O at the end of the run, keeping only what is needed to interpret and reproduce the results; **maxprint** (`-maxprint`, the default for rigorous and ultimate modes) keeps every intermediate file from every stage. Miniprint typically shrinks a finished project from gigabytes to tens of megabytes. A review run (`--review`, Section 12.5) always implies `--maxprint`, since its purpose is to be inspected.

Under miniprint each completed stage keeps its combined geometries (`combined_results.xyz/.mol`) and energy table (`*_summary.txt`), the full COSMIC analysis of each screening (dendrograms, `clustering_summary.txt`, `extracted_clusters/`, `extracted_data/`, and the `motifs_N/` or `umotifs_N/` representatives), the final ensemble, and `protocol_summary.txt`. The individual QM `.out` files, however, are retained *only for the motifs of the final, most expensive stage* — the DFT (or CCSD(T)) calculations a user may want to inspect for frequencies, orbitals or thermochemistry. A two step GFN2→DFT run therefore reduces to:

```
project/
├── annealing/
├── optimization/
├── similarity/
├── refinement/
├── similarity_2/
├── geom_ref_out/
│   ├── umotif_01/
│   └── umotif_02/
├── final_ensemble.xyz
├── final_ensemble.mol
└── protocol_summary.txt
```

Here `optimization/` and `refinement/` keep only coordinates and summaries, while `geom_ref_out/` holds the complete ORCA files for each final motif. A run whose final stage carries no frequencies keeps a single `similarity/` stage and writes `possible_final_ensemble.*` instead; an ultimate GFN2→DFT→DLPNO-CCSD(T) run adds `refinement_2/` and `similarity_3/`, retaining `.out` files for both the geometry- and energy refinement motifs. Unique motif numbering is carried across stages, so a given `umotif_NN` denotes the same structure everywhere.

11.2 Annealing Output

11.2.1 Main Log File (w6.out)

This file serves as the primary log, recording the complete history of the annealing protocol. To verify that the simulation completed successfully without runtime errors, users should inspect the final lines of the file (e.g., via the command `tail w6.out`). A successful run is confirmed by the presence of the termination flag:

```
** Normal annealing termination **
```

```
tail -n 15 w6_1.out
```

```

=====

** Normal annealing termination **
Total Wall time: 0 days 0 h 30 min 52 s 880 ms
Energy was evaluated 2919 times

Energy evolution in tvse_115889.dat
Configurations accepted by Max.-Boltz. statistics = 72
Accepted lower energy configurations = 55
Accepted configurations in result_115889.xyz = 127
Lowest energy configuration in rless_115889.out
Lowest energy = -0.51829172 u.a. (Config. 127)

=====

```

11.2.2 Reproducibility and Metadata

To ensure scientific rigor, ASCEC generates essential metadata for data validation and traceability. Each annealing run is initialized with a unique six digit random seed, which is used to seed all internal pseudorandom number generators (PRNGs). This mechanism ensures the stochastic independence of parallel runs, preventing statistical bias in the exploration of the potential energy surface. The seed is logged in the output file and appended to the names of key output files (e.g., `result_957775.xyz`, `tvse_957775.dat`), enabling clear identification and organization of results. Rerunning ASCEC with the same seed and input parameters will reproduce the same sequence of proposed molecular moves. However, exact reproduction of the full annealing trajectory additionally requires an identical software environment — the same Python and NumPy versions, as well as the same version, compilation, and parallelization settings of the external quantum chemistry program (e.g., ORCA or xTB) — since numerical differences in energy evaluations, however small, can alter accept/reject decisions and cause trajectories to diverge.

Setting the seed. By default the six digit seed is drawn at random, but it can be fixed from the environment to reproduce a run. `COSMIC_ASCEC_SEED` sets a single seed; `COSMIC_ASCEC_SEEDS` takes a comma separated list — one seed per replica — so an entire replicated protocol is reproducible from a single command:

```
COSMIC_ASCEC_SEED=878186 ascec glyw2.asc protocol
```

```
COSMIC_ASCEC_SEEDS=871246,112412,261557 ascec li5.asc protocol
```

The number of seeds in `COSMIC_ASCEC_SEEDS` must equal the number of replicas requested (three, for the `li5` example above); a mismatch is reported as an error. The same environment caveat applies: fixing the seed reproduces the sequence of proposed moves, but a bit for bit trajectory also needs the identical software stack described above.

11.2.3 Simulation Generated Data

I. Global Minimum Candidate (`rless_XXXXXX.out`)

This file contains the Cartesian coordinates corresponding to the lowest energy configuration visited during the entire annealing run. The suffix `XXXXXX` corresponds to the unique random seed of the run.

```
# Configuration: 141 | Energy = -71.65131472 u.a.
3
water1
O      3.114949    -2.572173    0.846802
H      3.740289    -3.084485    1.381054
H      2.849619    -3.185249    0.144893
3
water2
O      -1.728986     1.116733   -0.116378
H      -2.554571     0.817409   -0.525959
(...)
```

II. Energy Evolution Log (tvse_XXXXXX.dat)

This dataset records the thermodynamic history of the simulation. It is formatted in three columns: **Step Number**, **Temperature (K)**, and **Potential Energy (Hartree)**. Data is recorded exclusively for accepted Monte Carlo steps. This file is the source for generating the energy profile plots (see Figure 19) and the aggregate replica comparison (see Figure 20). These diagrams are automatically produced by the ASCEC script. To regenerate them manually, use the `diagram` command with the `--scaled` flag to autoscale the Y-axis.

```
ascec diagram --scaled
```

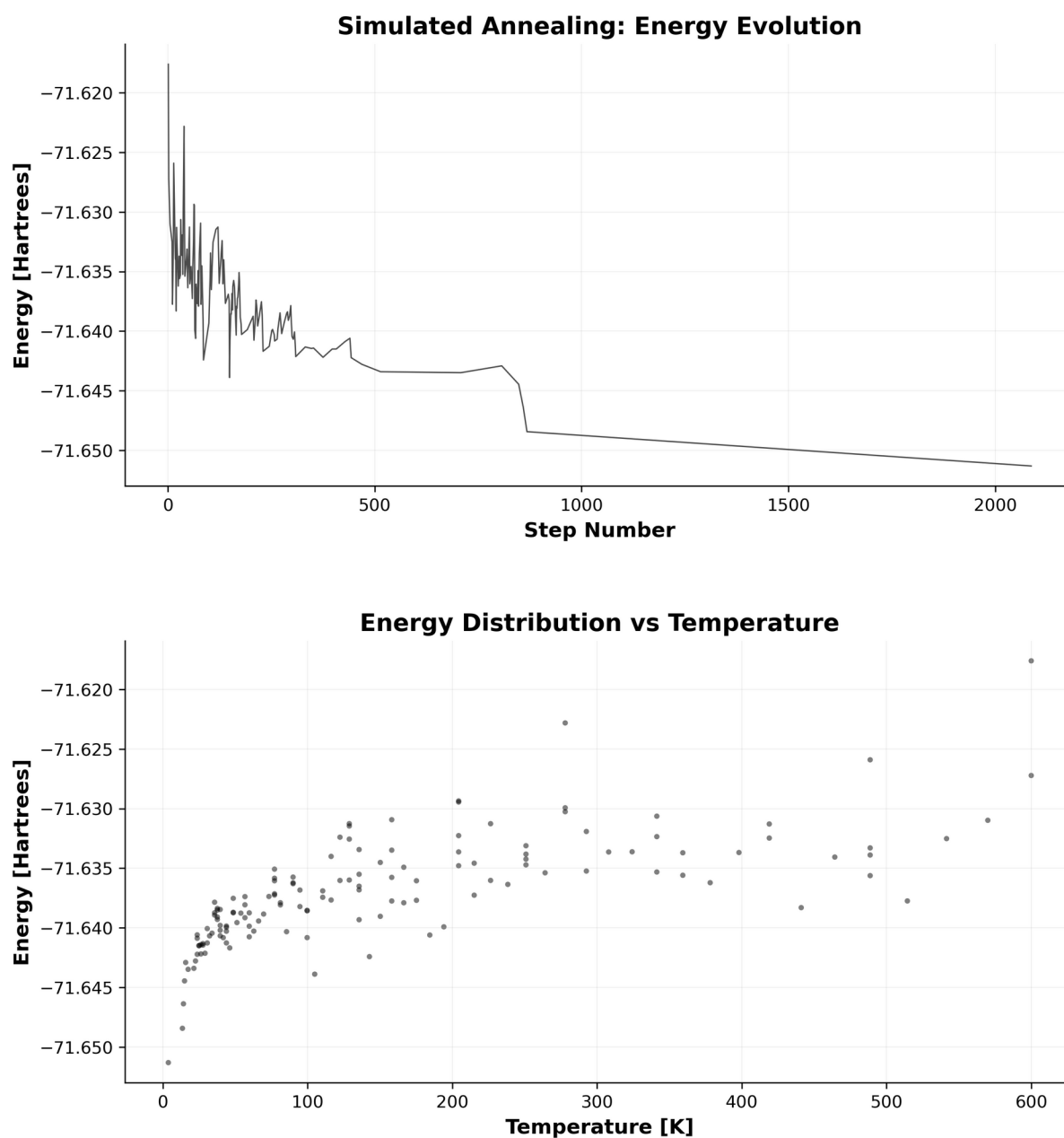


Figure 19: Single run energy evolution profile (Temperature vs. Energy) generated by ASCEC.

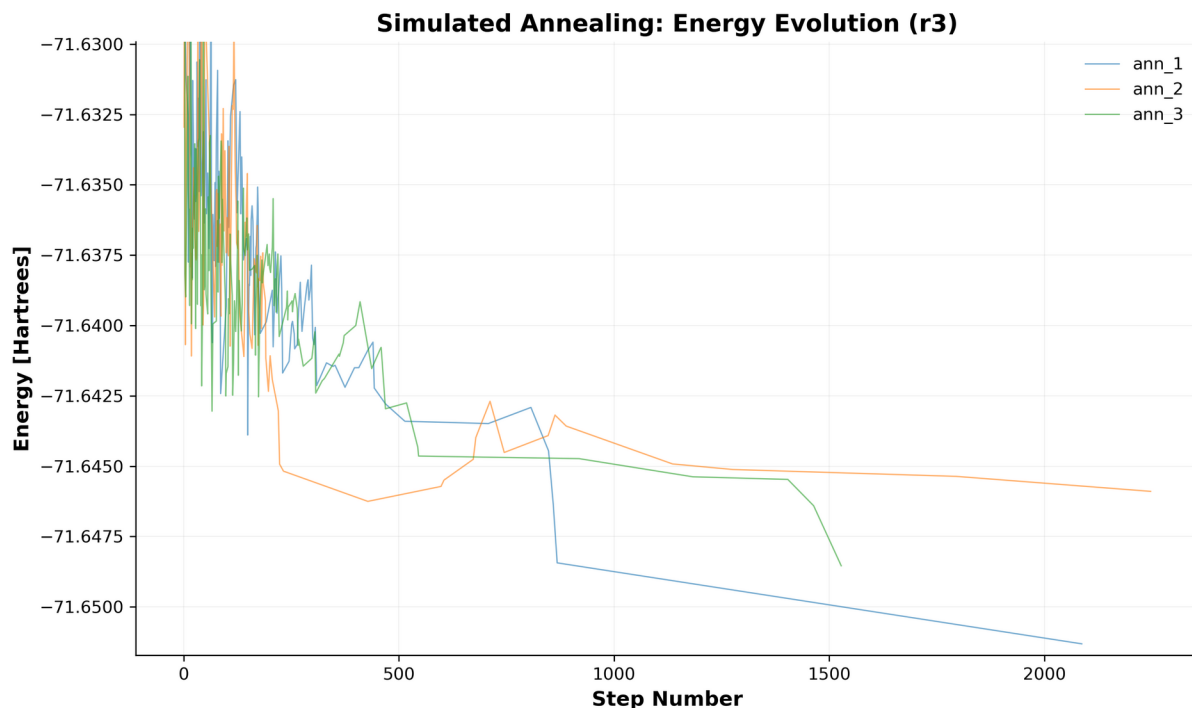


Figure 20: Combined energy profiles comparing multiple replica executions.

III. Structural Trajectory (result_XXXXXX.xyz | .mol)

These files contain the Cartesian coordinates of the entire ensemble of accepted structures. They serve as the full trajectory of the simulation, formatted for immediate visualization and animation using standard molecular editors (e.g., Avogadro2, Molekel, IQmol, or GaussView).

```

18
Configuration: 1 | E = -71.61760217 a.u. | T = 600.0 K
O      7.521687      8.123473      2.987348
H      7.729738      7.604909      3.779022
(...)
18
Configuration: 2 | E = -71.62722332 a.u. | T = 600.0 K
O      7.649736      8.853069      2.485145
H      8.312217      8.157098      2.610376
18
Configuration: 3 | E = -71.63098312 a.u. | T = 570.0 K
O      8.358924      7.910310      1.934682
H      9.138323      7.334893      1.954008

```

IV. Simulation Box Visualization (resultbox_XXXXXX.xyz | .mol)

Unless the `-nobox` flag is used, the program generates auxiliary files containing the simulation box vertices as dummy atoms. This allows for the visual verification of the confinement boundaries relative to the molecular system.

Compatibility Note:

- **Avogadro2:** Supports both `.xyz` and `.mol` formats for box visualization.
- **IQmol & GaussView:** Require the `.mol` file to correctly render the dummy atoms.

11.3 Calculation and Optimization Output

Once the process of calculating the generated input configurations and their sorting has been completed, the working directory (e.g., `calculation/` or `optimization/`) will typically contain the following key components:

- **Master Trajectories** (`combined_results.xyz` | `.mol`): Single master files containing the aggregated Cartesian coordinates for all processed configurations. These are generated to allow for the immediate visualization of the entire ensemble.

```

18
Coordinates from ORCA-job opt_conf_1 E -30.494666183120
O          4.89755049315488      6.57402652599668      3.11667076076455
H          4.49984221242737      6.65106539506547      4.01165830268415
(...)
18
Coordinates from ORCA-job opt_conf_2 E -30.489323286040
O          5.03567346983380      6.71360166724240      4.08623792011938
H          4.16925086054156      7.14879619512198      4.15686651181911
H          4.91370051289682      6.02838117620930      3.40385389606381

```

- **Job Subdirectories:** Dedicated folders containing the specific input/output files for each.
 - *In Calculation:* Naming depends on the input generation strategy:
 - * **Option ‘c’ (Combine):** `opt_conf_#` (e.g., `opt_conf_1`). Numbering is sequential across the merged dataset.
 - * **Option ‘a’ (Separate):** `opt#_conf_#` (e.g., `opt1_conf_1`). Explicitly identifies the source replica (e.g., Configuration 1 from Replica 1).
 - *In Optimization:* Named `motif_#` (e.g., `motif_01`).
- **Execution Summary** (`*_summary.txt`): A text file detailing the computational duration (wall time) and final energies for each calculation.
- **Launcher Script:** The shell script used to execute the batch jobs (e.g., `launcher_orca.sh`).

Exclusively in the **Calculation** directory, if the ****Combine**** strategy (Option c) was selected during setup, a file named `combined_r#` (e.g., `combined_r3`) is generated. This holds the raw merged data from the annealing replicas that were used to generate the initial input files.

11.4 COSMIC Output

Once the clustering process concludes, the results are stored in a dedicated directory. Note that the directory creation is driven by the `sort` command, which automatically increments the suffix (e.g., `cosmic_2`) to prevent overwriting. A typical output structure contains:

```

/
├─ dendrogram_images/
├─ extracted_clusters/
├─ extracted_data/
├─ motifs_53/
└─ orca_out_416/

```

```

|_ skipped_structures/
|_ boltzmann_distribution.txt
|_ clustering_summary.txt
|_ data_cache_105638.pkl

```

11.4.1 Dendrograms

The dendrogram is the main visualization tool for understanding the topology of the sampled set. It represents a hierarchical tree in which the vertical axis (Y) indicates the **Euclidean distance** (dissimilarity) between configurations.

By default, a single dendrogram (**dendrogram.png**) is generated for the entire dataset. The optional **--group-hb** flag produces per hydrogen bond family dendrograms (**dendrogram_H{N}.png**), which can be useful for visual inspection of specific interaction networks. The examples below use **--group-hb** to illustrate family specific trees.

Understanding the Clustering Logic (Figure 21) The structure of the dendrogram directly informs the choice of the clustering threshold (τ). As illustrated in the specific case of an H-Bond=10 family (using **--group-hb**), where three clusters were identified:

1. **Redundancy Identification:** Configurations **30** and **31** merge at a low height ($d < 1.0$). This implies they are geometrically and electronically equivalent.
2. **Cluster Differentiation (Intermediate):** Configuration **51** joins the (30+31) pair at a distance of $d \approx 3.5$. With a standard threshold of $\tau = 2.0$, this structure falls **above** the cutoff. Consequently, it is **not** grouped with the pair and is identified as a distinct motif.
3. **Global Separation:** Configuration **48** joins the ensemble at a much higher distance ($d \approx 5.8$). Since $5.8 \gg \tau$, it is recognized as a completely distinct basin of attraction, indicating significant structural and electronic differences.

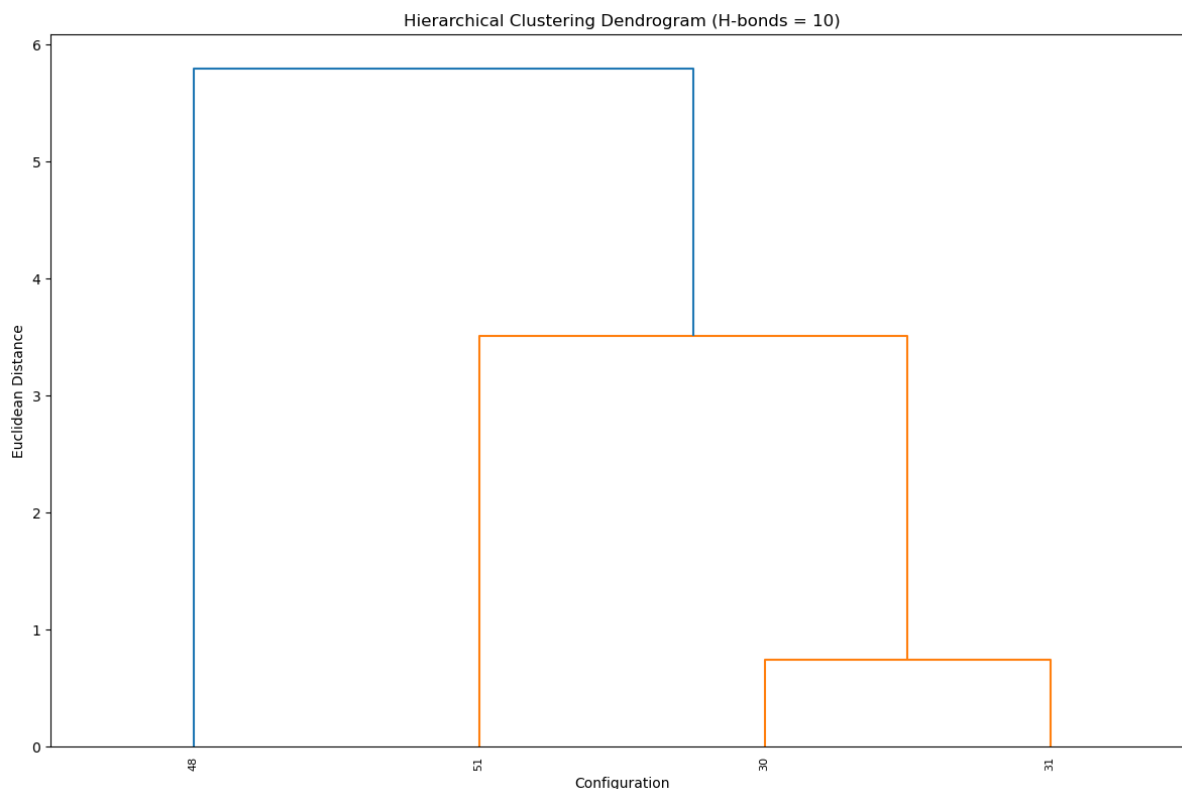


Figure 21: Detailed view of a dendrogram branch for the H-bonds=10 family (generated with `--group-hb`). The vertical height of horizontal nodes indicates the dissimilarity between branches.

Family Specific Trees (`--group-hb`): When the `--group-hb` flag is active, the COSMIC module splits the dataset based on the number of detected Hydrogen Bonds (HB), generating a separate dendrogram for each subset. This allows granular inspection of specific interaction networks. Without this flag, all structures appear in a single unified dendrogram.

Preoptimization Analysis (GFN2-xTB) Figure 22 displays the clustering results for the H-Bond=9 family following the initial semiempirical preoptimization (using `--group-hb`).

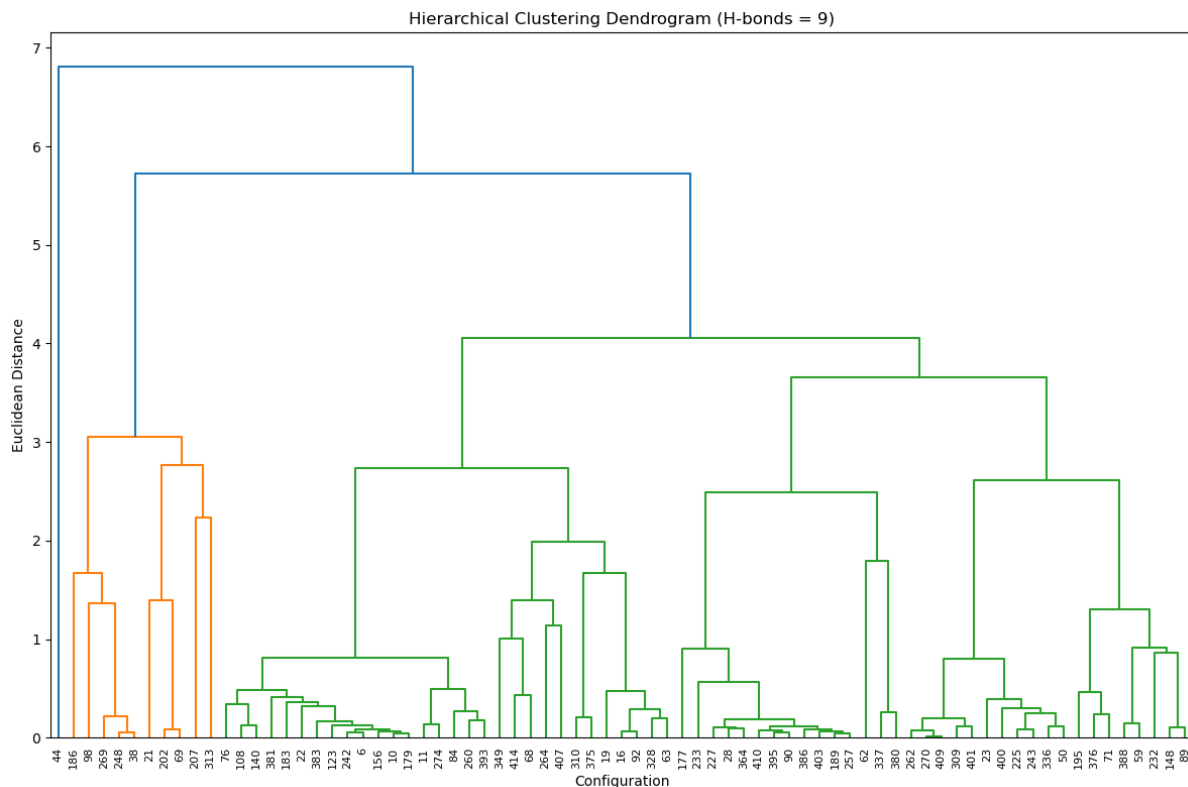


Figure 22: Dendrogram for the H-Bond=9 family following GFN2-xTB preoptimization (generated with `--group-hb`).

As shown in Figure 22, the tree exhibits a high density of leaves with deep, complex branching. The presence of distinct major families at large Euclidean distances indicates that the annealing process successfully located fundamentally different structural basins. However, the dense grouping at the bottom of the clusters suggests a high degree of redundancy.

Hierarchical Reduction Strategy:

In this specific example, the preprocessing step successfully reduced the ensemble from **416 raw candidates** to **53 motifs**. These representatives were subsequently reoptimized using a higher level of theory (B97-3c), followed by a secondary clustering analysis which yielded **20 unique motifs**. It is crucial to note that the final number of motifs diverges from the intermediate count, as the topology of the Potential Energy Surface (PES) is sensitive to the electronic structure method employed; distinct semiempirical minima may merge into a single basin upon DFT refinement. Furthermore, for this example, we did not correct the structures that the script flagged as critical and requiring manual inspection, so this final figure could be slightly higher.

- Impact of critical structures on total dataset: 8/416 configurations (1.9%)
- Impact of critical structures on total dataset: 4/53 configurations (7.5%)

Final Optimization Analysis (DFT/B97-3c) Figure 23 presents the topological classification of the H-Bond=7 family after high level refinement.

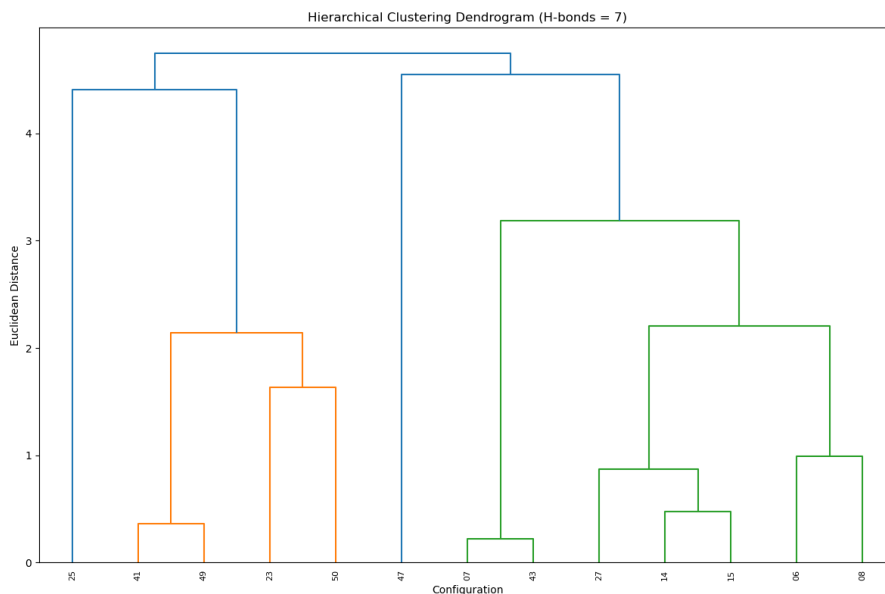


Figure 23: Dendrogram for the H-Bond=7 family following B97-3c final optimization (generated with `--group-hb`).

As shown in Figure 23, the topology is significantly more structured than the preoptimization stage. The clear vertical separation between nodes confirms that the high level optimization has relaxed the structures into well defined, distinct minima, facilitating the selection of a robust threshold for the final Boltzmann population analysis.

Motifs Dendrogram Figure 24 presents the topological classification of the motifs found.

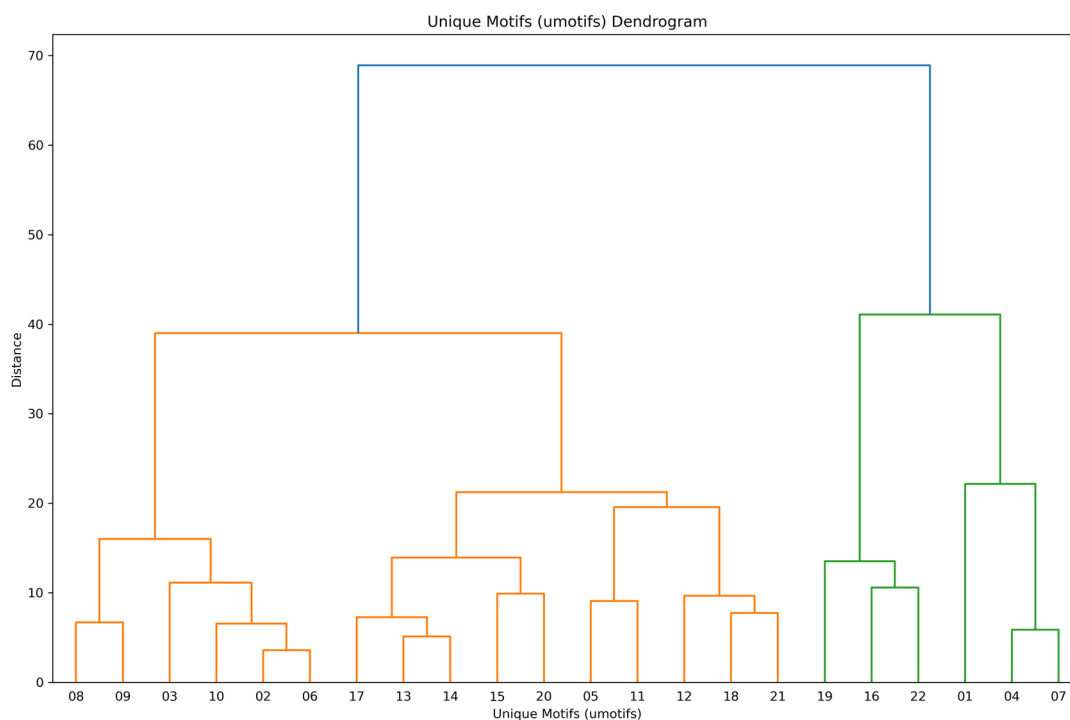


Figure 24: Dendrogram for the 22/23 unique motifs found after final optimization using B97-3c.

11.4.2 Clustering Summary Log

The script generates a comprehensive summary file (`clustering_summary.txt`) containing global statistics and detailed cluster breakdowns.

```
Clustering Results for: orca_out_416
COSMIC threshold (distance): 2.0
Total configurations processed: 416
Total files skipped: 23 (5.5%)
Critical skipped files: 8 (1.9%)
Total number of final clusters: 53
```

Example of Non Converged Output When certain structures did not converge, the summary report indicates “N/A” for Gibbs energy. This indicates that the associated ORCA jobs reached the maximum number of optimization cycles without converging, so the thermochemical calculation step was not performed. Even so, the script was able to analyze the last Cartesian coordinates and attempted to cluster them.

```
All configurations

Configurations: 53
Number of clusters: 23

Cluster 22 (3 configurations):
Files:
- motif_25_opt.out (Gibbs Energy: N/A)
- motif_26_opt.out (Gibbs Energy: N/A)
- motif_31_opt.out (Gibbs Energy: N/A)

Cluster 23 (1 configurations):
Files:
- motif_42_opt.out (Gibbs Energy: N/A)
```

Valid Cluster Analysis For successfully converged systems, the summary provides precise thermodynamic data. Below is an example showing a clear separation between a singleton cluster (Cluster 6) and a grouping of two configurations (Cluster 5).

```
Cluster 5 (2 configurations):
Files:
- motif_23_opt.out (Gibbs Energy: -458.357020 Hartree (-287623.37 kcal/mol))
- motif_50_opt.out (Gibbs Energy: -458.356211 Hartree (-287622.87 kcal/mol))

Cluster 6 (1 configurations):
Files:
- motif_25_opt.out (Gibbs Energy: -458.356863 Hartree (-287623.27 kcal/mol))
```

Note: When using `--group-hb`, the summary groups results under “Hydrogen bonds: N” headers instead of “All configurations”.

11.4.3 Error Handling and Diagnostics

The COSMIC module implements a comprehensive diagnostic protocol to ensure ensemble integrity. It classifies files based on termination status, convergence, and vibrational topology:

1. Anomaly Classification The algorithm detects three specific types of problematic structures, which can be found in the output files:

- **Execution Failures:** Files lacking the "Normal Termination" flag (e.g., due to crash).
- **Non Converged Geometries:** Files that did not reach a stationary point within the maximum optimization cycles despite normal termination.
- **Topological Instabilities:** Structures containing **one or more imaginary frequencies**. This includes first order saddle points (Transition States), higher order instabilities (Hilltops), or soft modes (e.g., unrelaxed methyl rotations).

2. Filtering Logic To balance data retention with redundancy reduction, anomalies are processed via clustering:

- **Skipped Files:** Files that cannot be parsed or failed execution are automatically skipped. Additionally, structures with imaginary frequencies are skipped if they cluster tightly ($d < \text{threshold}$) with a known true minimum, as these are considered "noisy" versions of a successfully identified basin.
 - **Flagged as Critical:** Any structure with imaginary frequencies that forms an **isolated cluster** (distinct from valid minima) is preserved and flagged as **Critical**. These are stored separately as they potentially represent:
 - * Unique Transition States (TS).
 - * Distinct local minimum that failed to converge purely due to numerical issues.
 - * Valid basin with a flat potential surface (soft modes).

Action Required: These files are exported for review and generally require manual inspection or reoptimization.

```

/
├─ skipped_structures/
│   └─ clustered_with_minima/
│       ├── motif_26_opt.out
│       ├── motif_26_opt.xyz
│       └─ need_recalculation/
│           ├── combined_need_recalc.xyz
│           ├── motif_11_opt.out
│           ├── motif_11_opt.xyz
│           └─ skipped_summary.txt

```

11.4.4 Generated Cluster Data

Beyond the summary, the script populates two key directories for detailed inspection:

1. **extracted_clusters/:** Contains subfolders for every identified cluster (e.g., **cluster_3_11** denotes Cluster #3 containing 11 configurations).

Inside, the individual or combined (**.xyz** | **.mol**) files allow for visual superposition and verification of the cluster.

2. **extracted_data/**: Contains statistical data files (e.g., **cluster_15_4.dat**). These text files list the raw physicochemical descriptors (dipole moments, rotational constants, energies) for every member of the group, enabling quantitative analysis of the cluster's homogeneity, along with weighting and tolerances.

Cluster (represented by: cluster_15_4) (4 configurations)

```
- opt_conf_239.out
- opt_conf_287.out
- opt_conf_322.out
- opt_conf_53.out
```

```
Deviation Analysis (Max-Min / |Mean|):
Gibbs Free Energy (Hartree) %Dev: 0.00%
HOMO-LUMO Gap (eV) %Dev: 0.04%
Dipole Moment (Debye) %Dev: 2.80%
Radius of Gyration (Å) %Dev: 0.21%
Rotational Constant A (GHz) %Dev: 0.49%
Rotational Constant B (GHz) %Dev: 0.49%
Rotational Constant C (GHz) %Dev: 0.62%
Last Vibrational Frequency (cm-1) %Dev: 0.02%
Number of Hydrogen Bonds %Dev: 0.00%
```

```
Clustering Weights Applied:
Gibbs Free Energy: 1.00
```

```
Clustering Absolute Tolerances Applied:
Gibbs Free Energy: 0.000005
```

11.4.5 Boltzmann Distribution

After optimization and clustering, the **boltzmann_distribution.txt** file is generated, ranking unique motifs by their Gibbs Free Energy at 298.15 K. While this reference temperature can be modified via the **-T=#** flag, the thermochemical properties must be recomputed accordingly to maintain accuracy. As illustrated below, the global minimum (UMotif 01) and the first local minimum (UMotif 02) are nearly isoenergetic, comprising more than 50% of the population.

```
Population by Energy Minimum
(assuming non-degeneracy, gi = 1)
```

```
Unique Motif (umotif) Assignment Summary
(sorted by Boltzmann population)
```

```
cluster_15 (umotif_01)
From structure: motif_03_opt
Gibbs Energy: -458.360815 Hartree (-287625.75 kcal/mol, -12472.63 eV)
Population: 26.77 %
```

```
cluster_1 (umotif_02)
From structure: motif_06_opt
Gibbs Energy: -458.360808 Hartree (-287625.75 kcal/mol, -12472.63 eV)
Population: 26.56 %
```

12 Workflow Mode

The automated protocol enables the execution of complex, multistage pipelines through a single directive. Specifically configured for the **ORCA** package. Unlike standalone execution—which requires user intervention to filter and resubmit files—the Workflow Mode autonomously manages the transition between annealing, optimization, and clustering. It includes built in logic to correct problematic structures (e.g., imaginary frequencies), significantly reducing the manual overhead required to locate global minima.

12.1 Pipeline Logic

Figure 25 delineates the execution logic of the automated pipeline for a standard two step refinement protocol. It illustrates the iterative progression from initial annealing to final refinement, including the automated feedback loops for correcting critical failures.

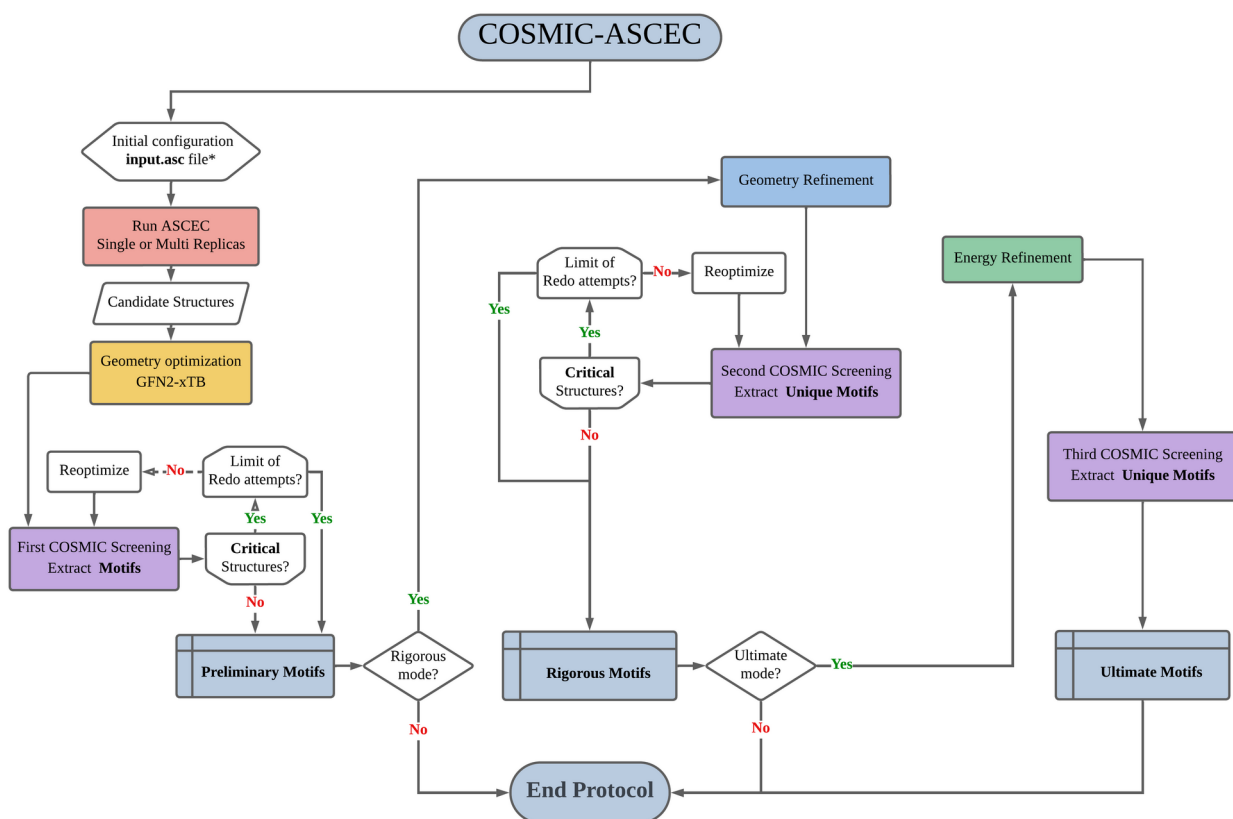


Figure 25: Schematic representation of the Two Step Refinement Protocol.

12.2 Workflow Input (formic.asc)

To illustrate this functionality we reproduce the paper’s rigorous formic acid dimer run. Each monomer can adopt a *syn* or *anti* arrangement of its O–H and C–H bonds, so a dimer combines them as *aa*, *as* or *ss*. Starting from two superimposed *ss* (*syn*) monomers, the workflow runs the annealing and preoptimization with the fast GFN2-xTB method (standalone xTB), then refines the COSMIC filtered motifs at wB97X-D3BJ/def2-TZVPP with ORCA. The complete input below — parameters, coordinates, protocol, and the two embedded QM templates (Section 8.2.1) — is the only file required.

```
# Input for cis_formic Simulation
```

```

1 10          # Line 1: Simulation Mode & Number of Config
6.7          # Line 2: Simulation Cube Length (Angstroms)
2           # Line 3: Annealing Quenching route
100.0 10.0 50 # Line 4: Linear Quenching Parameters
500.0 5.0 100 # Line 5: Geometric Quenching Parameters
3000 10      # Line 6: Max Monte Carlo Cycles per T and floor value
1.0 1.0      # Line 7: Maximum Displacement (A) & Rotation (radians)
60 120       # Line 8: Conformational sampling (%) & Max dihedral (deg)
3 xtb        # Line 9: QM Program Index & alias (1:Gaussian 2:ORCA 3:xtb)
GFN2-xtb     # Line 10: Hamiltonian & Basis Set
1 4          # Line 11: nprocs (QM calculations and ASCEC evaluation)
0 1          # Line 12: Charge & Spin Multiplicity
2           # Line 13: Number of Molecules (nmo)

```

Lines below: Molecule Definition

```

*
5
cis1
C      0.340460   -0.288340   0.000000
O      1.570760   -0.288340   0.000000
O     -0.288840    0.913960   0.000000
H     -0.369640   -1.132440   0.000000
H     -1.252740    0.795160   0.000000
*
5
cis2
C      0.340460   -0.288340   0.000000
O      1.570760   -0.288340   0.000000
O     -0.288840    0.913960   0.000000
H     -0.369640   -1.132440   0.000000
H     -1.252740    0.795160   0.000000
*

```

Protocol

(comma = continue to next stage, dot = pause at breakpoint)

```

.asc,
--maxprint,
r5 --box20 --concurrent=5,
opt --concurrent=8 input1,
cosmic -j4,
ref --concurrent=2 input2,
cosmic --th=opt -j4

```

Embedded QM templates (Section labelled placeholders)

```

#xtb input1
#name
#rescue(GFN2-xtb/freq)
# GFN2-xtb Opt Tight
# nprocs 1
# maxiter 200

```

```

#orca input2
#name
#rescue(HF-3c/freq)
! wB97X-D3BJ def2-TZVPP def2/J RIJCOSX DEFGRID3

```

```

! Opt Freq TightOpt TightSCF
%pal
  nprocs 12
end
%geom
  maxiter 80
end
* xyz 0 1
#
*
```

12.3 Protocol Command Syntax

The specific commands used in the `Protocol` section of the input file are detailed below.

Command / Flag	Description
<code>.asc</code>	Workflow Initiator. Designates the file as a multistage workflow script.
<code>-maxprint</code>	Retain I/O. Keeps every intermediate file from all stages (rigorous/ultimate default), instead of the space saving <code>miniprint</code> .
<code>-review</code>	Review run. Executes every stage of the protocol, but carries only the lowest energy annealing structure (the putative global minimum) into <code>opt</code> , <code>ref</code> and <code>eref</code> . Rehearses an expensive protocol at the cost of one structure. Implies <code>-maxprint</code> . See Section 12.5.
<code>r5 -box20 -concurrent=5</code>	Annealing. Runs five ($r = 5$) independent replicas concurrently. <code>-box20</code> sizes the box for 20% effective packing (rigorous default).
<code>opt -concurrent=8 input1</code>	Preoptimization. Optimizes every accepted structure with the embedded <code>input1</code> template (GFN2-xTB), 8 jobs at a time.
<code>cosmic -j4</code>	First screening. Clusters the preoptimized pool with the automatic (knee) threshold on 4 cores, keeping one motif per family.
<code>ref -concurrent=2 input2</code>	Refinement. Reoptimizes the motifs at high level with the embedded <code>input2</code> template (wB97X-D3BJ/def2-TZVPP Opt+Freq), 2 jobs at a time.
<code>cosmic -th=opt -j4</code>	Final analysis. Reuses the threshold resolved by the first screening (<code>-th=opt</code>) to cluster the refined structures into the unique minima and their Boltzmann ensemble.

Table 10: Commands and flags used in the rigorous formic acid workflow. The convergence tolerance flags (`-critical`, `-skipped`, `-retry`, `-redo`) are optional; their defaults, described in the Command Reference, drove the automatic error correction in this run.

12.4 Execution and Outputs

The automated protocol is initiated via the following command:

```
ascec formic.asc protocol
```

12.4.1 Prerequisites and File Structure

Prior to execution, specific configuration and template files must be present in the working directory. These templates must adhere to the syntax defined in Section 8.2.1, utilizing specific placeholders for dynamic data injection:

- **#name:** (Optional) Replaced by the specific filename to ensure traceability in output logs.
- **#:** Indicates the insertion point for the Cartesian coordinate block.
- **###:** Designates the location within the launcher script where serial execution commands will be injected.

Upon initialization, the system parses these files and generates a structured directory tree for each stage of the workflow.

Annealing → Calculation → COSMIC → Optimization → COSMIC(2)

Required Input Files	Generated Output Structure
<pre> / ├─ formic.asc ├─ launcher_orca.sh ├─ opt_wb97x.inp └─ preopt_gfn2.inp </pre>	<pre> / ├─ annealing/ ├─ calculation/ ├─ optimization/ ├─ cosmic/ ├─ cosmic_2/ ├─ protocol_425319.pkl └─ protocol_summary.txt </pre>

12.4.2 Terminal Output

When the workflow is initiated, the system parses the protocol and begins the first stage:

```

Protocol mode activated
Creating new protocol cache: protocol_670892.pkl (for cis_formic.asc)
Resuming workflow

Workflow: annealing → geometry_optimization → cosmic → geometry_refinement →
cosmic

-----
[1/5] Annealing
-----
Using recommended box size: 6.7 Å (20.0% effective packing)
Creating 5 replicated runs in 'annealing/'...
Created: formic_1/formic_1.asc
Created: formic_2/formic_2.asc
Created: formic_3/formic_3.asc

```

```
Created: formic_4/formic_4.asc
Created: formic_5/formic_5.asc
Running 5 annealing simulation(s)

formic_1...
```

12.5 Review Runs: Rehearsing an Expensive Protocol

A rigorous or ultimate protocol can run for hours or days, and almost all of that time is spent in the **opt**, **ref** and **eref** stages, which execute once per surviving structure. This makes the protocol as a whole awkward to test: a mistake in an ORCA template, a wrong multiplicity, a refinement that never converges or a misspelled COSMIC flag is not discovered until a large amount of computer time has already been spent.

A **review run** solves this. Adding **--review** to the protocol executes *every* stage exactly as written, but carries only one structure through the expensive ones: the lowest energy configuration found by the annealing, that is, the **putative global minimum**. The whole pipeline is therefore rehearsed end to end for roughly the cost of a single structure, and the run still returns something useful, namely an optimized and refined putative global minimum.

Orthogonal to the search modes. **--review** is not a fourth mode. Like **--maxprint** it is a global modifier that can be applied to a preliminary, rigorous or ultimate protocol (Section 1.1), and it is written on its own line directly under the **.asc**, marker. It implies **--maxprint**: a review run exists in order to be inspected, so nothing is cleaned up at the end (Section 11.1).

Example: the Li₅ cluster. The **li5** example shipped with the distribution is an open shell lithium pentamer (charge 0, multiplicity 2) whose ultimate protocol chains three quantum chemistry stages and three COSMIC screenings. Adding one line turns it into a review run:

```
# Protocol
# (comma = continue, dot = pause; --flags apply to the whole run)

.asc,
--review,
r1,
opt --concurrent=8 input1,
cosmic -j4,
ref --concurrent=2 input2,
cosmic --th=opt -j4,
eref input3,
cosmic --th=opt -j4
```

and it is launched exactly like any other protocol:

```
ascec li5.asc protocol
```

The stages then behave as follows:

- **Annealing runs in full.** Whatever **r<N>** asks for is executed with the requested concurrency, and each replica writes its complete set of accepted configurations. Nothing about the search itself is reduced. The web generator sets a single replica when *review* is selected, since only one structure continues past the annealing and further replicas would be

searched and then discarded; if you write more by hand the protocol honours them and the selection below is taken across all of them.

- **One structure is selected.** The lowest energy accepted configuration across *all* replicas becomes the stage’s combined input (`combined_r<N>.xyz`), which is precisely what that file means: the structures handed to the optimization. A review run therefore introduces *no new file names* — the run directory looks exactly like any other run, and every stage keeps its usual naming (`opt_conf_<N>`, then `candidate_01`, `motif_01` and `u_motif_01` as each COSMIC pass reports). Only the counts differ, since one structure survives. The complete annealing record is untouched in `annealing/*/result_*.xyz`.
- **Every later stage sees exactly one structure.** `opt`, `ref` and `eref` each run a single quantum chemistry job, and each COSMIC pass reports one input and one motif. Concurrency flags such as `--concurrent=8` are left in place and simply have nothing to parallelize, so the same file runs unchanged once the flag is removed.

The whole protocol then runs unattended. For the Li_5 cluster the seven stages complete in under an hour:

```

=== COSMIC ASCEC ===
-----
Progress [#####] 100.0%
-----
[1/7] annealing ✓
[2/7] geometry_optimization ✓
[3/7] cosmic (1→1) ✓
[4/7] geometry_refinement ✓
[5/7] cosmic_2 (1→1) ✓
[6/7] energy_refinement ✓
[7/7] cosmic_3 (1) ✓

Workflow finished
Start: 2026-09-23 11:35:35
End:   2026-09-23 12:31:08
Total wall time: 55m 32s
-----
Protocol summary saved to protocol_summary.txt

```

Every stage reports one structure, as expected. The timing breakdown in `protocol_summary.txt` is the part worth reading, because it is what the full run scales with:

Stage	Wall time	% total	Reading
Annealing	3m 01s	5.4%	Fixed cost; does not grow with the number of motifs.
Optimization	3.8s	0.1%	Semiempirical, negligible per structure.
Refinement	7m 01s	12.6%	Cost of <i>one</i> wB97X-D3BJ/def2-TZVPP Opt+Freq.
Energy refinement	45m 26s	81.8%	Cost of <i>one</i> DLPNO-CCSD(T1)/aug-cc-pVTZ point.

Table 11: Li_5 review run: where the time goes for a single structure.

Four fifths of the run is a single coupled cluster energy. That number, multiplied by the motif count the full run is expected to produce and divided by the concurrency, is the estimate to budget against before launching it. Section 13.3 carries this through for the Li_5 full run.

What a review run tells you. That the input file parses, that every embedded template produces a valid quantum chemistry input, that the requested methods and basis sets exist on the backend, that the charge and multiplicity are accepted, that each stage hands its output to the next, and roughly how long one structure takes in each stage, which is the number the full run scales with.

What it does not tell you. The selected structure is the lowest one *at the annealing level of theory*; it may well not remain the global minimum after optimization and refinement, which is precisely why the full run exists. The motif counts are also degenerate by construction: one structure always produces one motif, and that number says nothing about the diversity of the potential energy surface. A review run validates a protocol, it does not sample a surface, and its summary file is labelled accordingly.

Promoting to the full run. Once the review run has completed cleanly, delete the `--review` line and launch the same file again. Nothing else changes.

12.6 Execution Control and Resumption

The workflow allows for granular control over the execution pipeline via delimiters and caching.

12.6.1 Stage Delimiters

- **Continuous Execution (,):** A comma at the end of a protocol line indicates that the script should proceed immediately to the next stage upon completion.
- **Breakpoints (.):** A period (or dot) acts as a breakpoint. The workflow will complete the current stage and then pause, allowing the user to manually inspect intermediate files (e.g., to review **Critical** structures).

12.6.2 Resuming the Workflow

The system maintains a cache file (e.g., `protocol_425319.pkl`) to track progress. If a workflow is paused via a breakpoint or interrupted, it can be resumed from the last successful stage using the same command:

```
ascec formic.asc protocol
```

This ensures that expensive calculations are not repeated unnecessarily.

Resuming the Pipeline

Users can direct the workflow to start from a specific stage index:

```
1.Anneling → 2.Calculation → 3.COSMIC → 4.Optimization → 5.COSMIC2
```

Force Restart: To start over from the beginning of the Calculation stage (2):

```
ascec formic.asc protocol 2
```

Resume Incomplete: To resume the Calculation stage (2) preserving valid progress:

```
ascec formic.asc protocol 2 -i
```

12.7 Failure Recovery Mechanisms

The workflow includes two fault tolerance mechanisms for the rough potential energy surfaces of difficult systems.

12.7.1 Retry Logic (Execution Reliability)

This handles execution failures where a calculation does not reach "Normal Termination" (crashes, nonzero exit codes) by either restarting from scratch or resuming from the last recorded coordinates. It mitigates transient errors (filesystem disruptions, operational noise) but cannot fix systematic input syntax errors, so the QM templates must be prevalidated for the system under study.

```
-----
[2/5] Calculation
-----
Executing calculations...

Running: opt_conf_1.inp... ✓ (2nd Attempt)
Running: opt_conf_2.inp... ✓
...
Running: opt_conf_495.inp... ✓
Running: opt_conf_496.inp... ✓

Status: 496/496 calculations completed
All calculations completed successfully
```

12.7.2 Redo Logic (Convergence and Topology)

The final ensemble must consist exclusively of **true local minima**. To ensure no optimized structure retains imaginary frequencies, the Redo logic automatically intervenes when topological instabilities are detected:

1. **Multiple Imaginary Frequencies:** Treated as numerical convergence artifacts. The system attempts to resolve these by resubmitting the optimization using the final coordinates of the previous run.
2. **Single Imaginary Frequency:** Treated as first order saddle points (Transition States) or soft vibrational modes. The algorithm **perturbs the geometry along the imaginary vibrational mode** to push the structure toward a stable basin before reoptimizing.

Computational Cost Warning: This repeated refinement is expensive; it is meant to resolve soft modes and saddle points, not to compensate for an inappropriate method.

```
-----
[4/5] Optimization
-----
Stage redo enabled: max 10 attempts, target skipped ≤ 0.0%

Executing optimization calculations...
Running: motif_01_opt.inp... ✓
...
```

```

Running: motif_35_opt.inp... ✓

Status: 35/35 optimizations completed
Optimization calculations completed

-----
[5/5] COSMIC
-----
Using cosmic script: cosmic-v01.py

Working directory: cosmic_2

Processed 2 structures with imaginary frequencies:
- 0 clustered with true minima (can be ignored)
- 2 may represent missing motifs (need recalculation)
  motif_25_opt.out - 1 imaginary freq(s)
  motif_12_opt.out - 2 imaginary freq(s)

Finished processing folder: orca_out_35
All selected molecular analyses complete!

Results: 5.7% critical, 5.7% skipped
→ Threshold exceeded (skipped 5.7% > 0.0%)

```

Clarification of Metrics: The **Critical** set is a specific subset of the **Skipped** set. While all structures containing imaginary frequencies are automatically classified as **Skipped**, the **Critical** designation is reserved exclusively for those that form **isolated motifs**.

```

-----
Redo attempt 2/10

Processing redo structures from: cosmic_2/skipped_structures
Found 2 structure(s) to retry

motif_12_opt: 2 imaginary freqs, using final geometry ✓
motif_25_opt: 1 imaginary freq, displacing along mode ✓

Regenerated 2 input file(s) with new geometries
Resuming: Using existing optimization directory (Redo Mode)

Executing optimization calculations...
Resuming: 33/35 optimizations already completed
Running: motif_12_opt.inp... ✓
Running: motif_25_opt.inp... ✓

Status: 35/35 optimizations completed

Using cosmic script: cosmic-v01.py
python ../../cosmic-v01.py --th=2 -j4 --update-cache /tmp/tmpk17rmd.txt

```

13 Results and Validation

This section presents validation case studies demonstrating the efficacy and robustness of the COSMIC ASCEC protocols. We evaluate the software across three distinct dimensions:

1. **Manual Execution:** A hierarchical Two Step protocol using the Water Hexamer (H_2O)₆ [5].
2. **Automated Workflow:** A demonstration of the fully automated pipeline utilizing the *cis*-Formic Acid Dimer [14].
3. **Clustering Algorithms:** An assessment of standard topological clustering versus RMSD refined hierarchical clustering.

13.1 Manual Execution: Water Hexamer

To validate the modular utility of the software, the Water Hexamer system (`w6.asc`) was analyzed using a manual execution strategy. This case study benchmarks the tradeoff between computational cost and configurational space coverage.

13.1.1 Hierarchical Two Step Protocol (ORCA)

This protocol employed a cost effective preoptimization filter using the semiempirical GFN2-xTB method, followed by high level DFT refinement. The workflow processed an initial stochastic pool of **416 raw annealing candidates**.

1. Geometric Reduction (GFN2-xTB): The COSMIC module reduced the raw input of 416 structures to ****53 representative configurations**** based on topological distinctness. Figure 26 illustrates the ensemble members of a specific cluster (`cluster_1_24`), while Figure 27 displays its representative structure.

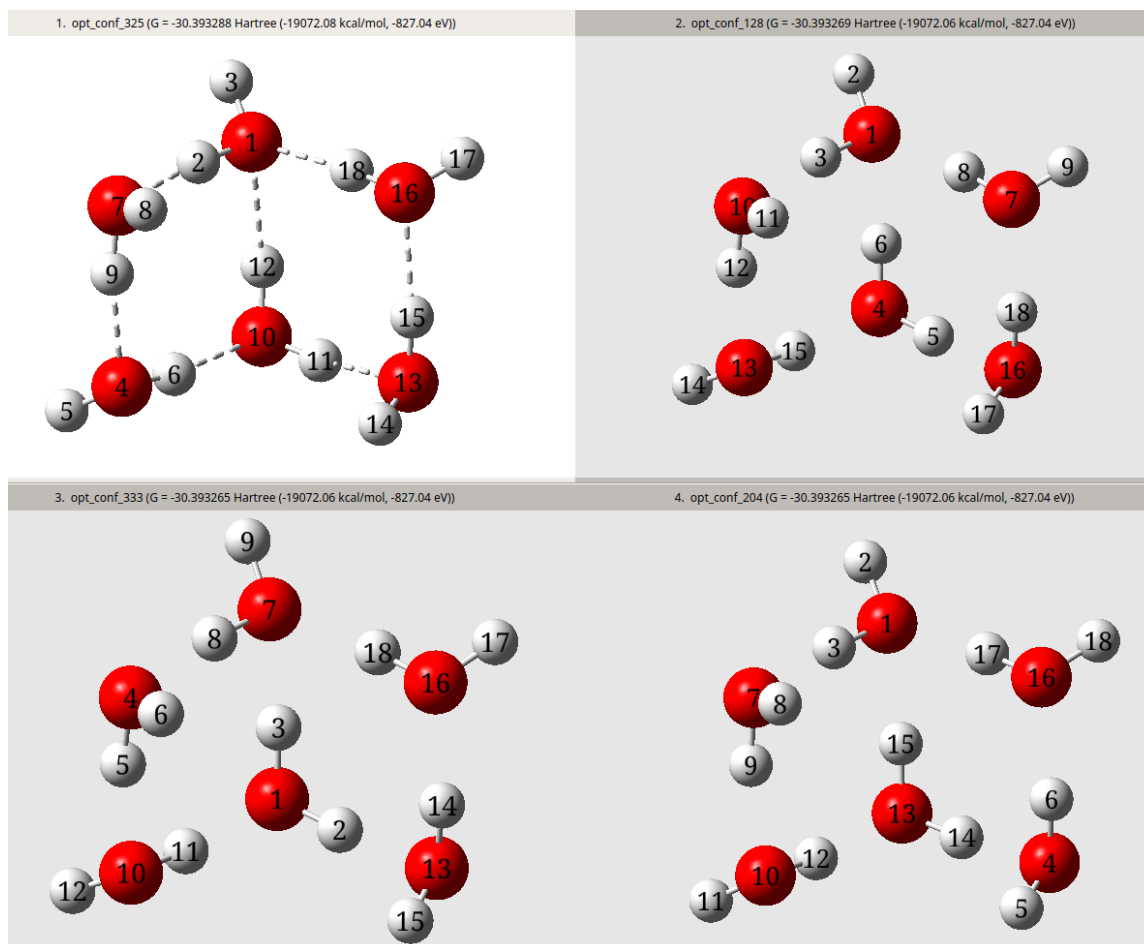


Figure 26: Visual inspection of the `cluster_1_24` ensemble members.

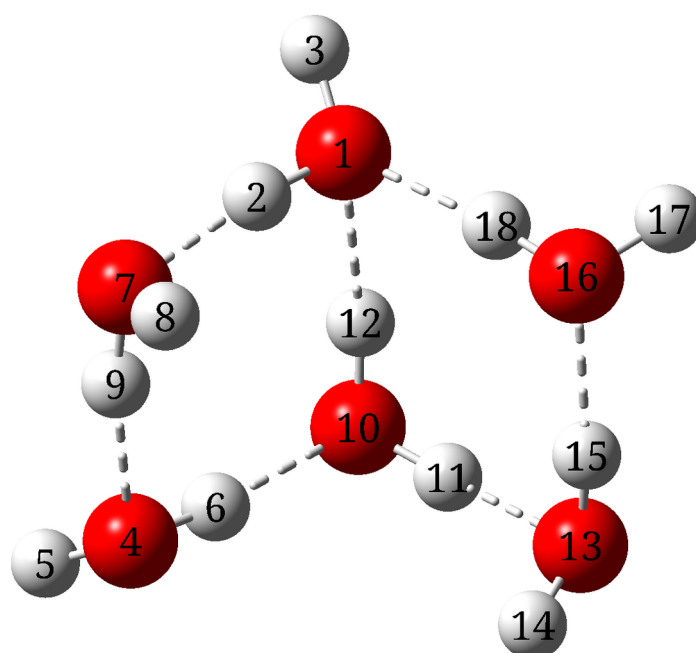


Figure 27: Representative configuration for `cluster_1_24`.

2. High Level Refinement (B97-3c): The 53 representatives underwent full DFT optimization (B97-3c). A final COSMIC check (Threshold $\tau = 2.0$) consolidated these into ****20 Unique Motifs****. Some of them can be seen in Figure 28. In addition, Figure 29 highlights the three most statistically significant structures.

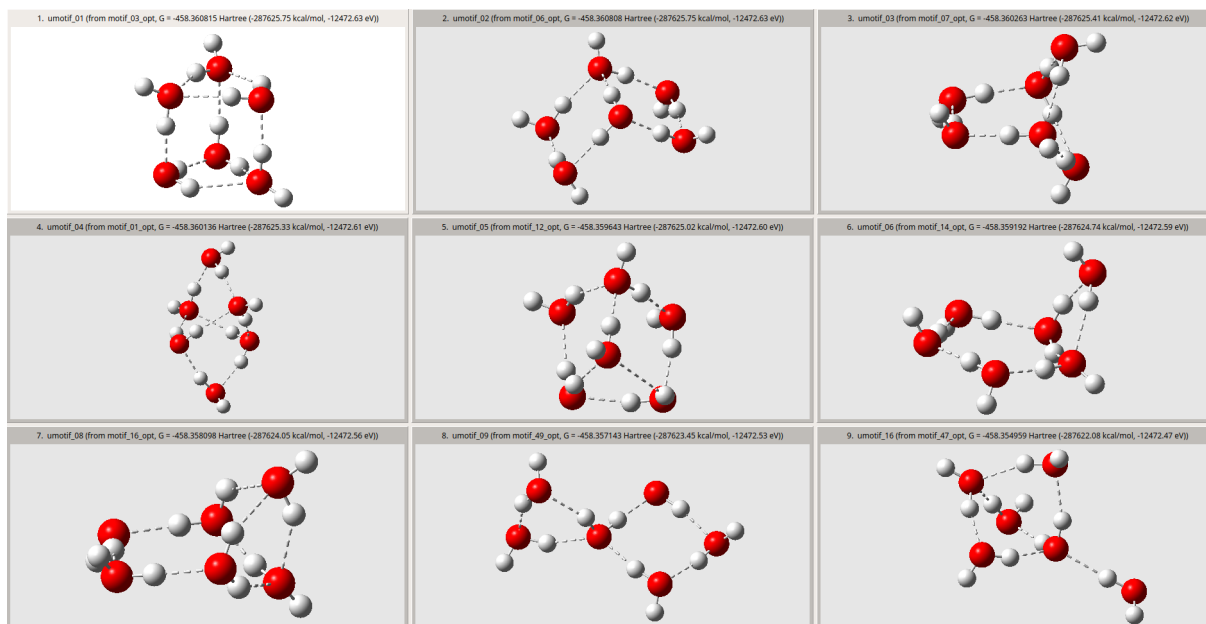


Figure 28: Subset of unique motifs identified for $(H_2O)_6$ system via the Two Step protocol.

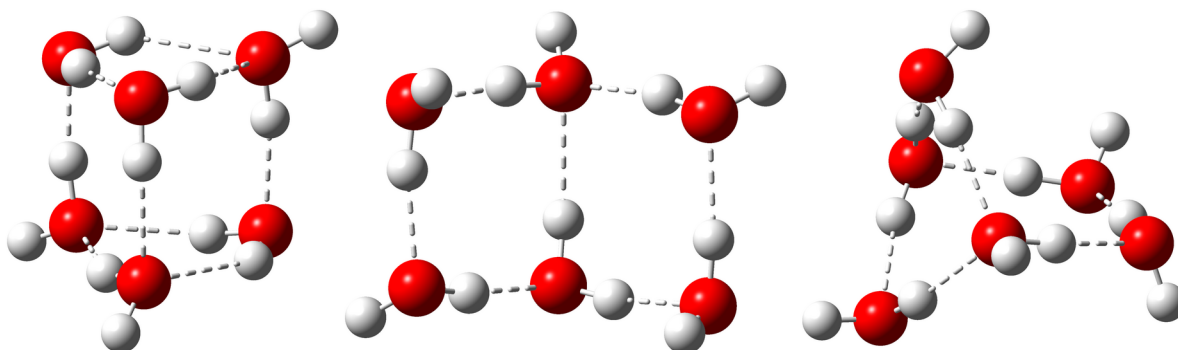


Figure 29: The three dominant water hexamer motifs identified by ASCEC.

Boltzmann Distribution: Thermodynamic analysis at 298.15 K indicates that the top three motifs account for over **68%** of the total ensemble population. The global minimum (Prism like, UMotif 01) and the first local minimum (Book like, UMotif 02) are effectively isoenergetic.

Unique Motif (UMotif) Assignment Summary
(Sorted by Boltzmann population - 298.15 K)

UMotif_01 (cluster_15)
Gibbs Energy: -458.360815 Hartree (-287625.75 kcal/mol)
Population: 26.35 %

```
UMotif_02 (cluster_1)
Gibbs Energy: -458.360808 Hartree (-287625.75 kcal/mol)
Population: 26.14 %

UMotif_03 (cluster_2)
Gibbs Energy: -458.360263 Hartree (-287625.41 kcal/mol)
Population: 14.69 %
```

13.1.2 Direct Approach (Gaussian 09) and Comparative Analysis

The Gaussian 09 validation employed a direct optimization strategy at the wb97XD/6-31+G(d) level of theory, skipping the semiempirical prescreening. Comparing the Hierarchical and Direct method approaches reveals critical insights regarding Potential Energy Surface (PES) sampling and computational efficiency.

1. Topological Accuracy and Semiempirical Bias Both methods successfully identified the dominant low energy isomers (Prism, Cage, Book). However, a detailed inspection of the full ensembles reveals that the semiempirical pretreatment (GFN2-xTB) exhibits a specific topological bias that must be taken into account.

GFN2-xTB tends to overstabilize cooperative hydrogen bonding networks, prioritizing structures where water molecules act as both donors and acceptors. Consequently, configurations containing "pure acceptor" water molecules (which do not donate hydrogen) are occasionally filtered out during the preoptimization step. Figure 30 illustrates such a structure, which was found by the direct approach but excluded by the two step protocol.

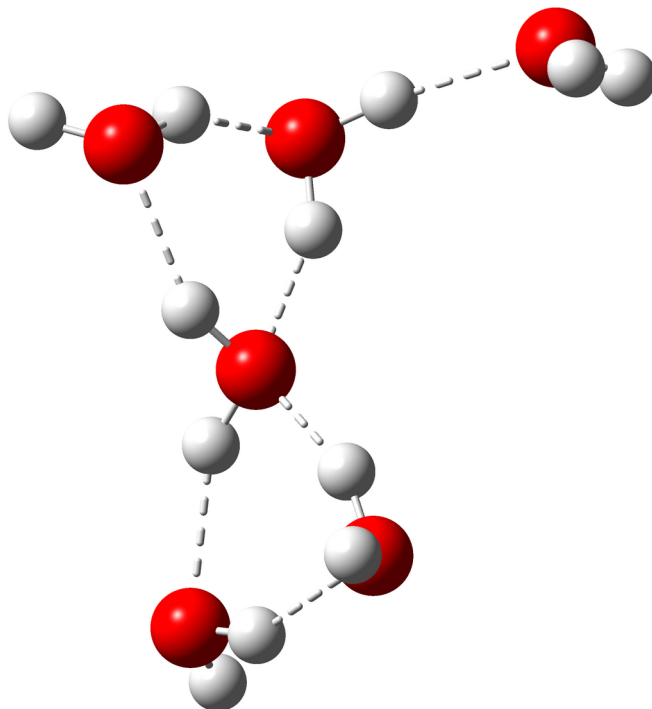


Figure 30: A specific water hexamer configuration identified only by the Direct Approach (Gaussian 09), characterized by a pure acceptor water molecule.

Despite this topological divergence, the thermodynamic impact is negligible. The missing structure possess high relative energies and contribute $\approx 0\%$ to the Boltzmann distribution at 298.15 K. Furthermore, comparisons with high level benchmarks (Figure 31) confirm that the ASCEC Two Step protocol structures closely replicate the MP2/6-311++G** geometries reported by [5], in cases where the presence of pure acceptor water molecules is **not mandatory**.

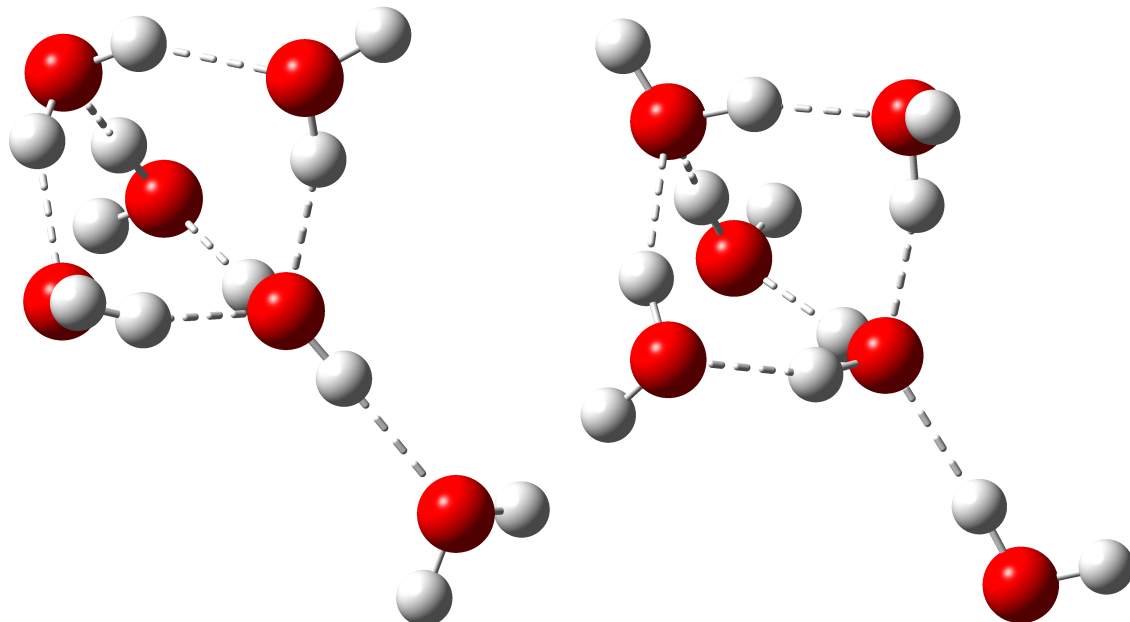


Figure 31: Structural comparison of the MP2/6-311++G** reference geometry (Left) [5] and the ASCEC derived GFN2-xTB//B97-3c structure (Right).

2. Computational Efficiency The decisive advantage of the hierarchical two step protocol is cost. Because the raw annealing pool is first collapsed to a handful of representatives by the GFN2-xTB preoptimization and the first COSMIC pass, only those few structures reach the expensive DFT stage. The direct approach instead sends every candidate straight to DFT, so it pays the high level cost hundreds of times over for a final ensemble that, as shown above, is essentially identical in its thermodynamically relevant minima. The two step route therefore delivers the same PES at a small fraction of the wall time.

13.2 Automated Workflow: Formic Acid Dimer

To demonstrate the autonomy of the ASCEC pipeline, the Formic Acid Dimer system was processed using the **Workflow Mode**, following the rigorous protocol of the accompanying paper. Unlike the manual execution, this protocol autonomously managed error correction, imaginary frequency elimination, and convergence criteria to recover the full *aa/as/ss* dimer family, and to locate the cyclic global minimum, starting from two superimposed *ss* (*syn*) monomers (Figure 32).

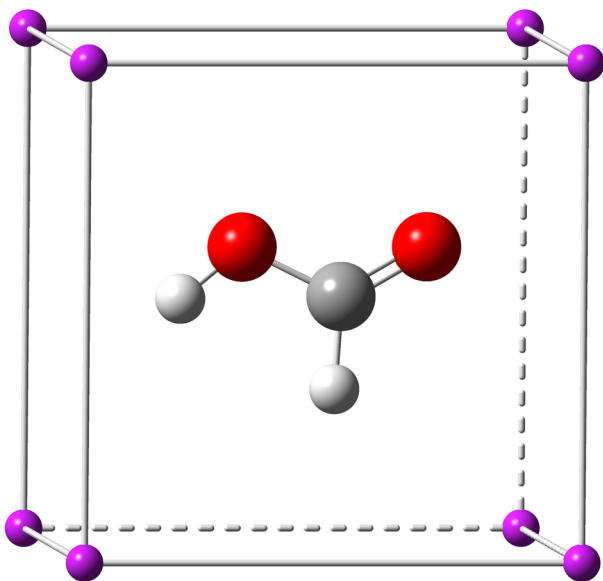


Figure 32: Initial simulation box containing *cis*-Formic Acid monomers.

13.2.1 Protocol and Computational Efficiency

The rigorous run used five independent annealing replicas, preoptimized every accepted structure at GFN2-xTB (standalone xTB), and refined the COSMIC filtered motifs at wB97X-D3BJ/def2-TZVPP with full frequency analysis (ORCA). Starting from the **6.7 Å** box, the five replicas accepted **574 candidates**, and the complete pipeline finished in **10 h 43 min**.

Consistent with every other benchmark, the stochastic search is computationally negligible. The recorded `protocol_summary.txt` splits the wall time as annealing **0.1%**, GFN2-xTB pre-optimization **0.2%**, and DFT refinement **99.7%** (Figure 17): almost the entire cost is the high level stage acting on the handful of structures COSMIC retained.

13.2.2 Automated Error Correction and Robustness

The two COSMIC screenings behaved very differently, which is instructive. After the GFN2-xTB preoptimization the first screening reduced the 574 structures to **46 motifs** with **0% critical** and **0% skipped**: at the semiempirical level every retained structure was already a clean minimum, so no redo was triggered.

The picture changed once the 46 motifs were refined at wB97X-D3BJ/def2-TZVPP. The final screening initially found **11 of them (23.9%)** carrying imaginary frequencies — soft modes and near saddle points that the tighter DFT surface exposes. Because the protocol requires a clean final ensemble (`critical` $\leq 0\%$), the **Redo** loop engaged automatically:

- a structure with a *single* imaginary frequency is displaced along that mode and reoptimized;
- a structure with *several* imaginary frequencies is reoptimized from its last geometry, treating them as convergence artifacts.

Over **three redo attempts** all 11 flagged structures were recovered as genuine minima, bringing the critical fraction to **0%** (Figure 33). Structures that survive every redo attempt are usually not minima at all but dissociated arrangements — fragments driven far apart during the search (> 7.5 Å, Figure 34) — which the algorithm correctly refuses to count; in this run none remained.

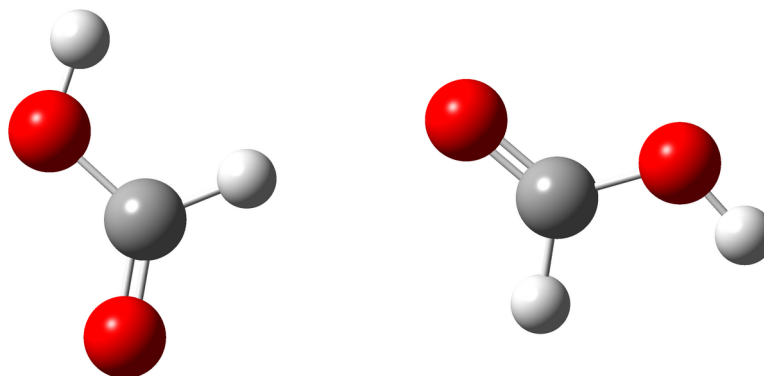


Figure 33: A structure flagged with an imaginary frequency and automatically recovered as a valid minimum by the Redo loop.

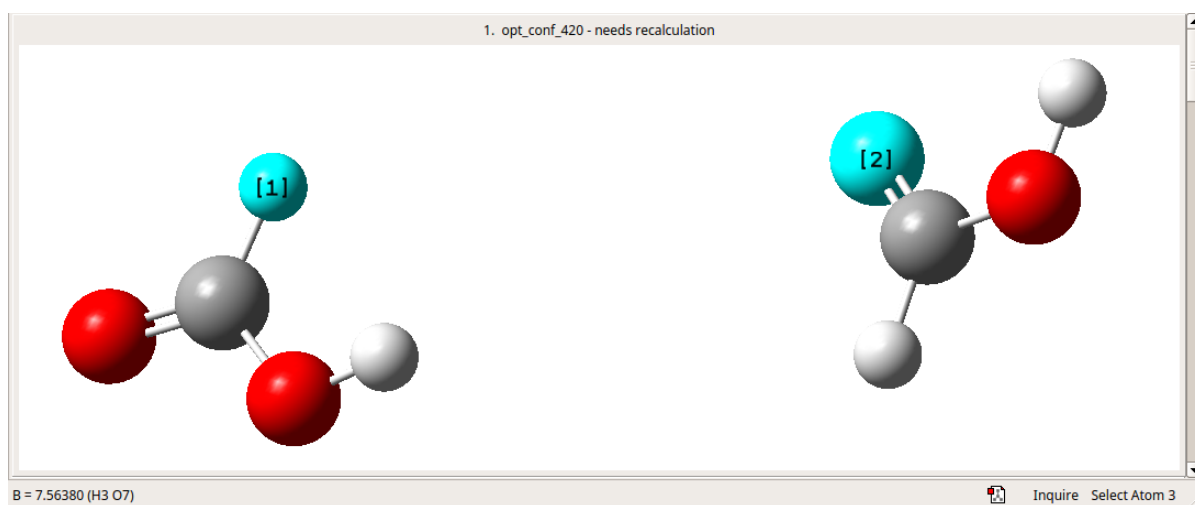


Figure 34: A dissociated arrangement ($r > 7.5 \text{ \AA}$) of the kind flagged as *critical* when it cannot be recovered; such structures are excluded from the final ensemble.

13.2.3 Thermodynamic Analysis

The final COSMIC pass reused the automatic (knee) threshold resolved by the first screening — this is what `-th=opt` means — and consolidated the 46 refined structures into **21 unique motifs** (7 *aa*, 10 *as*, 4 *ss*; one visual redundancy remains), of which 18 coincide with the reference MP2 calculations of the literature (Figure 35). The Boltzmann distribution at 298.15 K is overwhelmingly dominated by the cyclic *aa* dimer: its double hydrogen bond holds **99.99%** of the population, and the next motif already lies **5.46 kcal/mol** higher, so every remaining conformer is effectively inaccessible at room temperature ($RT \approx 0.59 \text{ kcal/mol}$).

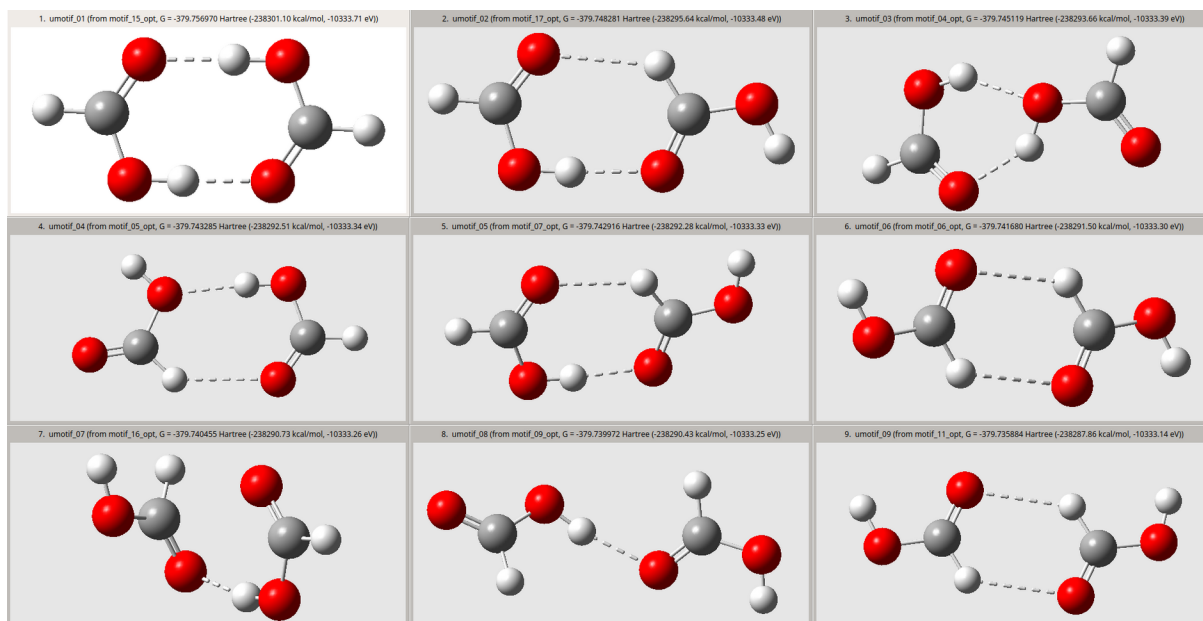


Figure 35: Subset of unique motifs identified for the formic acid system.

The global minimum (UMotif 01, Figure 36) corresponds to the cyclic *aa* dimer with a double hydrogen bond. Remarkably, although the input used two *ss* (*syn*) monomers, the search recovered the complete *aa/as/ss* family and correctly identified the cyclic *aa* structure as the global minimum, demonstrating the protocol's ability to overcome the monomer isomerization barrier during the stochastic search.

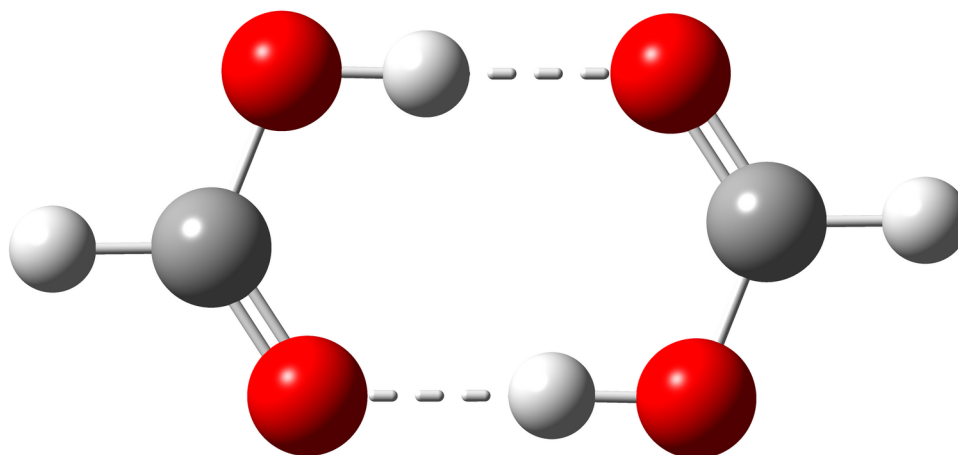


Figure 36: Global minimum: The cyclic Formic Acid Dimer.

Unique Motif (umotif) Assignment Summary
(sorted by Boltzmann population)

```
cluster_1 (umotif_01)
From structure: motif_21_opt
Gibbs Energy: -379.756973 Hartree (-238301.10 kcal/mol, -10333.71 eV)
Relative Gibbs Energy: 0.000000 Hartree (0.00 kcal/mol)
Population: 99.99 %
```

a $\Delta\Delta G = 0.00$ b $\Delta\Delta G = 2.59$ c $\Delta\Delta G = 27.06$ d $\Delta\Delta G = 27.93$

Figure 37: The four unique motifs of Li_5 from the full ultimate run, with relative composite Gibbs energies in kcal/mol at $\tau = 0.70$.

How well did the review run predict it? The per structure costs transfer almost exactly. The review run measured 6 m 57 s for one geometry refinement against the full run's 7 m 31 s mean (92%), and 45 m 23 s for one energy refinement against 48 m 34 s (93%). What the review

run cannot supply is the number of motifs; that estimate has to come from the chemistry, or from a cheap preliminary run. Given it, the arithmetic is straightforward: 12 refinements two at a time predicts 42 minutes against 57 actual, and 4 energy refinements one at a time predicts 3.0 hours against 3.2 actual. The shortfall is scheduling, not method — jobs of uneven length do not pack perfectly into a fixed number of slots.

The useful conclusion is not the total but its shape. The review run showed that 82% of the cost was a single coupled cluster energy, and that held: energy refinement was 76.5% of the full run. A protocol whose review run is dominated by one stage will be dominated by that stage at scale, and that is where a wrong keyword or an unaffordable `%maxcore` is worth catching first.

And the structure? The review run’s single refined structure came out at a composite Gibbs energy of -37.406576 Hartree. The full run’s global minimum, the motif carrying 98.74% of the population, is -37.406589 Hartree. The difference is 0.008 kcal/mol, and every interatomic distance agrees to within 0.005 Å. For Li_5 the putative global minimum picked from the annealing survived optimization and refinement intact.

This is a result, not a rule. It holds here because the annealing sampled a small surface densely and the lowest semiempirical structure was also the lowest at coupled cluster level. Neither is guaranteed: on a larger or more flexible system the ordering can and does change between levels of theory, which is the entire reason the full run exists. A review run is a rehearsal of the protocol, and its structure is a first guess — occasionally, as here, a very good one.

13.4 Clustering Methodologies: Topology vs. Geometry

This section evaluates the impact of the hierarchical clustering protocols on ensemble generation. We contrast the coarse grained Physicochemical Feature Vector (Step I) approach against an fine grained RMSD refinement (Step II) hierarchical strategy to determine the optimal use cases for either and to decide how to choose between them.

13.4.1 Case Study A: Resolving Spatial Isomerism (Adenine–Thymine)

The limits of purely vector based clustering (Step I) are evident in systems where distinct stereoisomers possess identical internal connectivity and energies. In the Adenine–Thymine dimer, the standard algorithm (Threshold $\tau = 2.0$) grouped spatially distinct configurations into a single family (`cluster_17`). While physicochemical properties are conserved, the spatial orientation differs significantly (Figure 38).

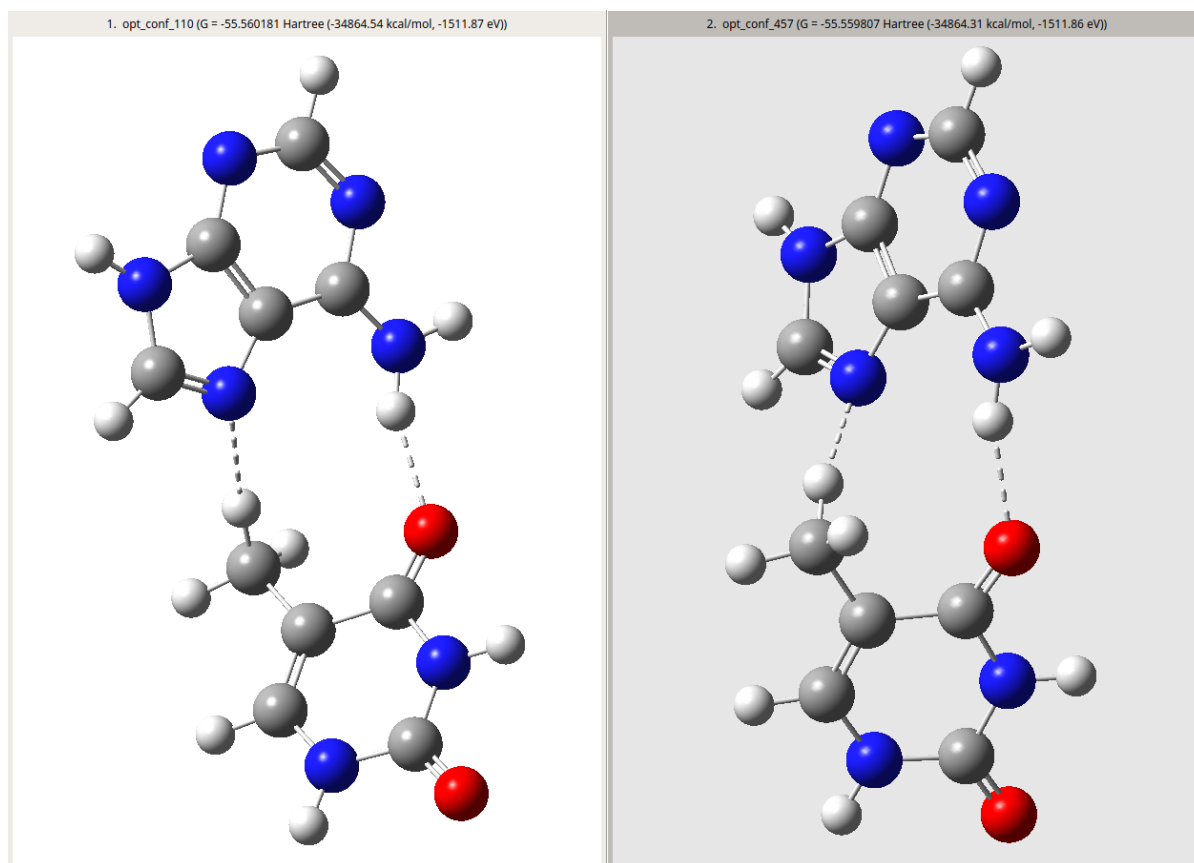


Figure 38: Two distinct spatial arrangements of the Adenine–Thymine dimer initially grouped into a single topological cluster.

By enabling the **RMSD Refinement (Step II)**, the algorithm successfully distinguished these isomers. The secondary geometric check decomposed **cluster_17** into two unique entities (**cluster_140** and **cluster_141**), despite their nearly identical Gibbs energies.

```
Cluster 140 (1 configurations) | RMSD Validated from Cluster 17:
Files:
- opt_conf_110.out (Gibbs Energy: -55.560181 Hartree (-34864.54 kcal/mol))

Cluster 141 (1 configurations) | RMSD Validated from Cluster 17:
Files:
- opt_conf_457.out (Gibbs Energy: -55.559807 Hartree (-34864.31 kcal/mol))
```

A pairwise comparison confirms the geometric distinctness, yielding a heavy atom RMSD of $> 6.6 \text{ \AA}$, indicating a rotation of the complex that the scalar feature vector could not detect.

```
cosmic --compare opt_conf_110.out opt_conf_457.out
```

```
RMSD Analysis (Heavy Atoms):
Pairwise RMSD values between configurations:
opt_conf_110.out vs opt_conf_457.out: 6.655 \AA
```

13.4.2 Case Study B: Handling Symmetry and Indexing (Arsenate Anion)

While RMSD refinement offers geometric precision, it is computationally expensive and sensitive to atom indexing, which can lead to "overclustering" in high symmetry systems. This is illustrated by the monohydrated dihydrogen arsenate anion ($H_2AsO_4^- \cdot H_2O$) (Figure 39).

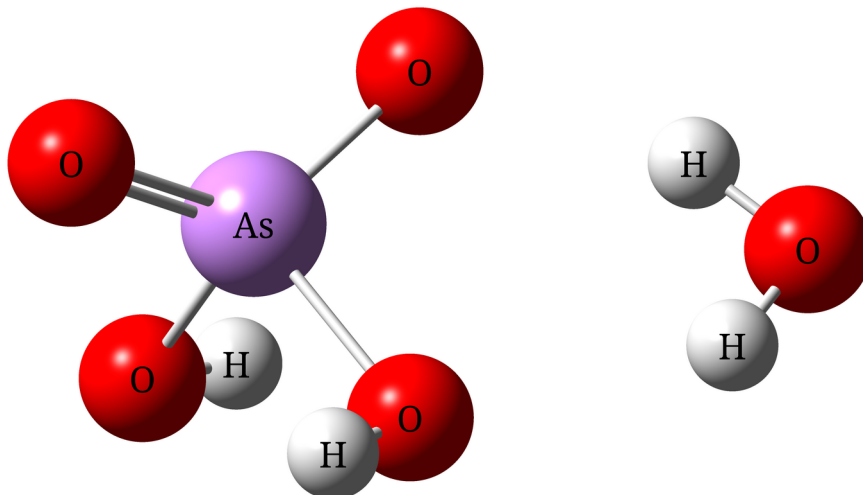


Figure 39: The monohydrated dihydrogen arsenate anion system.

The standard topological clustering (Step I) correctly identified a unified motif (`cluster_2_20`) containing 20 chemically identical structures (Figure 40).

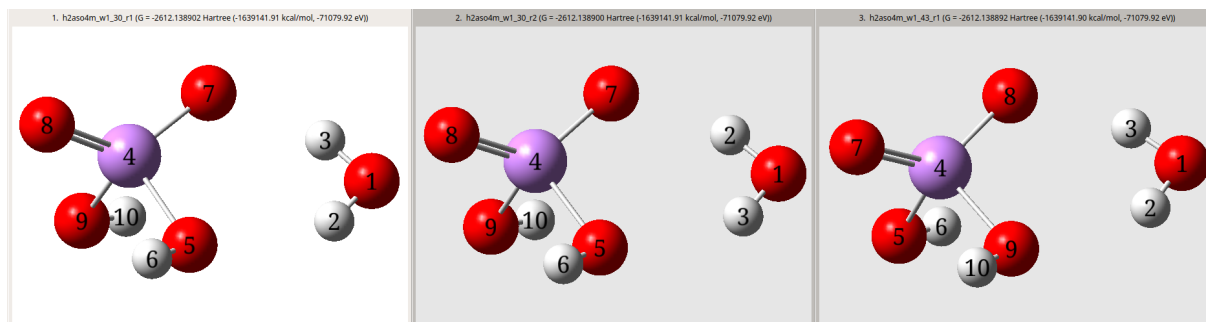


Figure 40: Correctly unified cluster identified by standard topological analysis.

However, applying RMSD refinement forced a split of this group. Due to atom indexing permutations inherent to the stochastic generation process, the RMSD algorithm flagged 8 structures as "distinct," creating a redundant artifact (`cluster_3_8`, Figure 41). These structures are chemically indistinguishable from the parent cluster.

Cluster 3 (8 configurations) | RMSD Validated from Cluster 2:

- h2aso4m_w1_12_r2.log (Gibbs Energy: -2612.138878 Hartree (-1639141.89 kcal/mol))
- h2aso4m_w1_38_r1.log (Gibbs Energy: -2612.138874 Hartree (-1639141.89 kcal/mol))

...

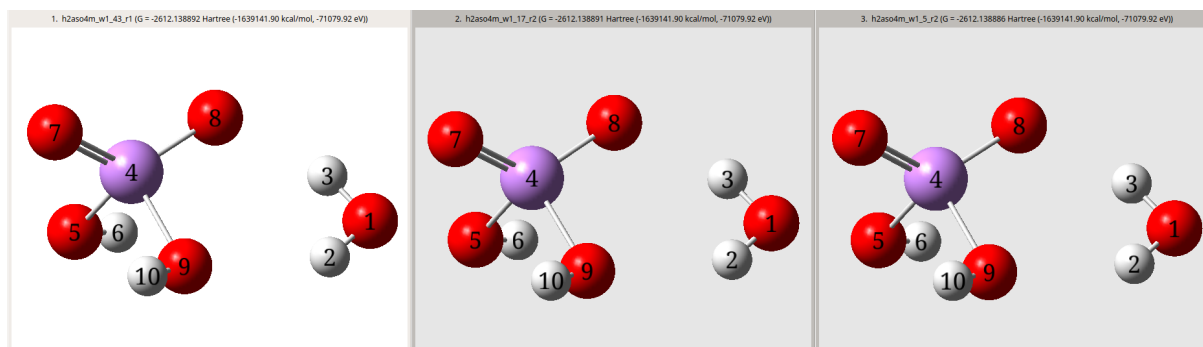


Figure 41: Redundant cluster artifact generated by RMSD sensitivity to atom indexing.

13.4.3 Clustering Overview

The RMSD refinement acts as a rigorous geometric filter, necessary for isolating specific stereoisomers (Case A). However, for general PES exploration, it risks inflating the ensemble size with false positives due to indexing permutations (Case B). Therefore, the ASCEC protocol defaults to topological vector clustering (Step I) for maximum efficiency, reserving RMSD (Step II) as an optional flag for systems where specific spatial orientation is critical.

14 Command Reference

14.1 Cheat Sheet

The commands below cover almost every run. Most users only need the last block — the automated protocol drives every stage from a single `.asc` file.

```
# --- Sampling -----
ascec input.asc                # single annealing run
ascec input.asc r5 --box20 --concurrent=5  # 5 replicas, 20% packing
ascec input.asc box            # box / packing analysis only

# --- Build QM decks from the results -----
ascec opt template.inp launcher.sh  # optimization inputs (+launcher)
ascec ref template.inp launcher.sh  # refinement inputs (+launcher)

# --- Collect & cluster -----
ascec sort                      # rank optimized structures
cosmic -j4                     # cluster (auto threshold, 4 cores)
cosmic --rmsd=1 -j4            # + geometric split at 1.0 Å
cosmic --th=opt -j4            # refinement stage: reuse postopt tau
cosmic --partialweights -j4     # preliminary xTB / semiempirical

# --- Reproducibility (seeds) -----
COSMIC_ASCEC_SEED=42 ascec input.asc      # one seed, all replicas
COSMIC_ASCEC_SEEDS=1,2,3 ascec input.asc r3  # one seed per replica
```

```

# --- Housekeeping -----
ascec status                # progress of the current run
ascec diagram               # energy plots from the tvse_*.dat files
ascec cleanup               # remove temp calc/cosmic folders
ascec input.asc --maxprint  # keep all intermediate files
ascec input.asc --review    # rehearse the protocol on the putative
global minimum

# --- One shot automated protocol (recommended) -----
ascec input.asc protocol    # run every stage from the .asc
ascec input.asc protocol 3  # force-restart from stage 3
ascec input.asc protocol 3 -i # resume stage 3 where it stopped

```

14.2 Sampling and Replication

Commands used to initiate Simulated Annealing runs.

```

# Basic Execution
ascec input.asc          # Single run

# Replication (rN)
ascec input.asc r3       # 3 Replicated runs (Default packing)
ascec input.asc r5 --box10 # 5 Replicated runs with 10% packing density

# Box Validation
ascec input.asc box      # Analyzes simulation box requirements

```

14.3 Automated Workflow (Protocol)

Controls the execution of the multistage pipeline.

Command Format	Description
... protocol	Starts (or resumes) the automated workflow defined.
... protocol N	Force Restart. Restarts the pipeline from stage <i>N</i> (wiping progress for that stage).
... protocol N -i	Resume Interactive. Resumes stage <i>N</i> from where it left off (incomplete state).

Workflow: Annealing → opt → COSMIC → ref → COSMIC [→ eref → COSMIC]

14.4 Protocol Thresholds

These commands generate input files for Quantum Chemistry packages (ORCA/Gaussian) and define the error handling logic.

Flag	Description
<code>--retry=N</code>	Execution Retries. Maximum attempts to restart a calculation that crashed or failed (e.g., walltime limit).
<code>--redo=N</code>	Topology Correction. Maximum attempts to fix imaginary frequencies by perturbing coordinates along the unstable mode.
<code>--critical=N</code>	Critical Threshold (Preopt). Max % of allowed "Critical" structures (isolated imaginary freqs). Set to 0 for strict filtering.
<code>--skipped=N</code>	Skipped Threshold (Final Opt). Max % of allowed "Skipped" structures. Set to 0 to ensure all final motifs are true minima.

14.5 COSMIC Analysis Module

Invoked via `ascec cosmic` or `cosmic`, this module handles clustering and Boltzmann analysis.

Flag	Description
<code>--th, --threshold</code>	COSMIC Threshold. Defines the clustering granularity. Default <code>auto</code> (knee detection, capped at $\tau = 2.0$; see Section 7.3.7); a float forces a fixed cut, lower values giving stricter clustering.
<code>--rmsd</code>	Geometric Validation. Enables RMSD checking in Ångströms (Default: 1.0 Å). Useful for distinguishing conformers with similar energies but different shapes.
<code>-j, --cores</code>	Parallelization. Number of CPU cores to use for parsing and matrix calculation (Default: Autodetect).
<code>-T, --temperature</code>	Boltzmann Temp. Temperature in Kelvin for Gibbs Free Energy ranking (Default: 298.15 K).
<code>--weights</code>	Custom Weights. Define specific feature importance (e.g., <code>'(energy=0.8)(dipole=0.15)'</code>).
<code>--group-hb</code>	H-Bond Grouping. Groups structures by hydrogen bond count before clustering. Produces separate dendrograms per HB family (<code>dendrogram_H{N}.png</code>). Useful for visual inspection of specific interaction networks.
<code>--reprocess-files</code>	Ignores the cached <code>.pkl</code> data and forces a reread of all output.

15 Quick ORCA 6.1.1 Installation for Linux

ORCA is optional: xTB is the default backend and is installed for you. Install ORCA when you want DFT or wavefunction refinement. Every command below can be pasted as written; nothing needs editing, because all paths are built from `$HOME`.

1. Check whether your CPU supports AVX2

AVX2 builds are faster. If this prints nothing, download the plain (non AVX2) build in the next step.

```
grep -c avx2 /proc/cpuinfo
```

2. Download the archives

Register on the ORCA forum and download the ORCA 6.1.1 shared library build together with the matching OpenMPI version. ORCA is linked against one specific OpenMPI release, so the pair must match.

- ORCA download portal: <https://orcaforum.kofo.mpg.de/app.php/portal>
- OpenMPI releases: <https://www.open-mpi.org/>

3. Put the downloads where the rest of these commands expect them

Create the working directory and move the two archives into it from wherever your browser saved them, then list it to confirm both are present.

```
mkdir -p "$HOME/software" && mv "$HOME/Downloads"/orca_6_1_1_linux*.tar.xz \
"$HOME/Downloads"/openmpi-4.1.8.tar.gz "$HOME/software/" \
&& cd "$HOME/software" && pwd && ls -lh
```

You should see both files listed. If your browser saved them elsewhere, change \$HOME/Downloads above to that directory.

4. Build and install OpenMPI

This compiles OpenMPI into \$HOME/software/openmpi-4.1.8. It takes several minutes; -j\$(nproc) uses every available core.

```
cd "$HOME/software" && mkdir -p openmpi-4.1.8-src \
&& tar -xf openmpi-4.1.8.tar.gz -C openmpi-4.1.8-src --strip-components=1 \
&& cd openmpi-4.1.8-src \
&& ./configure --prefix="$HOME/software/openmpi-4.1.8" \
&& make -j"$(nproc)" && make install
```

Confirm the build, which should report 4.1.8:

```
"$HOME/software/openmpi-4.1.8/bin/mpirun" --version
```

5. Unpack ORCA

Use the AVX2 archive if step 1 found AVX2 support, otherwise the plain one.

```
cd "$HOME/software" && mkdir -p orca_6_1_1 \
&& tar -xf orca_6_1_1_linux_x86-64_shared_openmpi418.tar.xz \
-C orca_6_1_1 --strip-components=1 \
&& ls orca_6_1_1/orca
```

6. Put ORCA and OpenMPI on your PATH

This appends the block to ~/.bashrc and reloads it, so there is no file to edit by hand. The quoted delimiter keeps the variable names literal.

```
cat >> ~/.bashrc <<'EOF'
```

```
# ORCA 6.1.1
export ORCA611_ROOT="$HOME/software/orca_6_1_1"
export OPENMPI418_ROOT="$HOME/software/openmpi-4.1.8"
export PATH="$ORCA611_ROOT:$OPENMPI418_ROOT/bin:$PATH"
export LD_LIBRARY_PATH="$ORCA611_ROOT:$OPENMPI418_ROOT/lib:$LD_LIBRARY_PATH"
EOF
source ~/.bashrc && which orca
```

which orca must print the path inside `$HOME/software/orca_6_1_1`.

7. Verify

ORCA has no dedicated version flag; it prints its version at the top of any run, and `orca -v` is enough to see it:

```
orca -v
```

If which orca points somewhere in /usr/bin, you have hit the GNOME screen reader, which is also called Orca. Either remove it (`sudo apt remove --purge orca && sudo apt autoremove`) or make sure the line you added in step 6 comes last in `~/.bashrc`, so your ORCA wins the PATH lookup.

16 AI Acknowledgement and Reproducibility

The development and documentation of the ASCEC protocol involved the use of large language models (LLMs) to improve code optimization, data synthesis, and technical writing. Specifically, **Claude Opus 4.5** [15] and **Gemini 3.0** [16] were used to streamline the programming process and refine the algorithmic logic.

In accordance with ethical research standards, all AI assisted results underwent rigorous human verification to ensure scientific accuracy and reproducibility. We encourage users who adapt this methodology to maintain transparency regarding the use of computational assistants and to verify all results.

17 Glossary

These terms recur throughout the manual and in the program's output.

Configuration vs conformation A *configuration* is an arrangement of whole molecules relative to one another; a *conformation* is the internal shape of one molecule, set by its dihedral angles. ASCEC samples both.

Motif One representative structure standing for a family of structures COSMIC judged equivalent. Reducing hundreds of candidates to a handful of motifs is what makes the expensive stages affordable.

Replica An independent annealing run of the same system with a different random seed. Written **rN** in a protocol, so **r3** means three replicas. More replicas cover the surface better.

Concurrent How many quantum chemistry jobs run at the same time (`--concurrent=N`). This is about throughput, not sampling, and is bounded by your cores and memory.

Threshold τ The distance at which the clustering tree is cut. A smaller τ splits structures into more, finer families; a larger one merges them. It can be chosen automatically.

Critical structure A refined structure that came back incomplete or with imaginary frequencies, so it is not a true minimum. COSMIC reports the percentage of these.

Redo An automatic repeat of a stage for the critical structures, nudging them off the saddle point and reoptimizing, until none are left or the attempt limit is reached.

miniprint / maxprint How much is kept on disk. Under **miniprint** (the default) only the final ensemble's individual quantum chemistry files survive; **--maxprint** keeps everything from every stage.

review run A protocol launched with **--review**: every stage executes, but only the lowest energy annealing structure (the putative global minimum) enters the per structure stages. Used to validate an expensive protocol before running it in full.

18 References

- [1] F. Neese. "Software Update: The ORCA Program System—Version 6.0". In: *WIREs Comput. Mol. Sci.* 15.2 (2025). DOI: [10.1002/wcms.70019](https://doi.org/10.1002/wcms.70019).
- [2] C. Bannwarth, S. Ehlert, and S. Grimme. "GFN2-xTB—An Accurate and Broadly Parametrized Self-Consistent Tight-Binding Quantum Chemical Method with Multipole Electrostatics and Density-Dependent Dispersion Contributions". In: *J. Chem. Theory Comput.* 15.3 (2019). DOI: [10.1021/acs.jctc.8b01176](https://doi.org/10.1021/acs.jctc.8b01176).
- [3] S. Kim et al. "An Update on PUG-REST: RESTful Interface for Programmatic Access to PubChem". In: *Nucleic Acids Res.* 46.W1 (2018). DOI: [10.1093/nar/gky294](https://doi.org/10.1093/nar/gky294).
- [4] N. Rego and D. Koes. "3Dmol.js: Molecular Visualization with WebGL". In: *Bioinformatics* 31.8 (2015). DOI: [10.1093/bioinformatics/btu829](https://doi.org/10.1093/bioinformatics/btu829). PMID: 25505090.
- [5] G. Hincapié et al. "Structural Studies of the Water Hexamer". In: *J. Phys. Chem. A* 114.29 (2010). DOI: [10.1021/jp103683m](https://doi.org/10.1021/jp103683m).
- [6] P. J. van Laarhoven and E. H. Aarts. *Simulated Annealing: Theory and Applications*. Springer Science & Business Media, 1987.
- [7] N. Metropolis et al. "Equation of State Calculations by Fast Computing Machines". In: *J. Chem. Phys.* 21.6 (1953). DOI: [10.1063/1.1699114](https://doi.org/10.1063/1.1699114).
- [8] J. F. Pérez, C. Z. Hadad, and A. Restrepo. "Structural Studies of the Water Tetramer". In: *Int. J. Quantum Chem.* 108.10 (2008). DOI: [10.1002/qua.21615](https://doi.org/10.1002/qua.21615).
- [9] T. M. Abramyan et al. "Cluster Analysis of Molecular Simulation Trajectories for Systems Where Both Conformation and Orientation of the Sampled States Are Important". In: *J. Comput. Chem.* 37.21 (2016). DOI: [10.1002/jcc.24416](https://doi.org/10.1002/jcc.24416). PMID: 27292100.
- [10] W. Kabsch. "A Solution for the Best Rotation to Relate Two Sets of Vectors". In: *Acta Crystallogr. Sect. A* 32.5 (1976). DOI: [10.1107/S0567739476001873](https://doi.org/10.1107/S0567739476001873).
- [11] R. R. Sokal and C. D. Michener. "A Statistical Method for Evaluating Systematic Relationships". In: *Univ. Kans. Sci. Bull.* 38.22 (1958).
- [12] V. Satopaa et al. "Finding a "Kneedle" in a Haystack: Detecting Knee Points in System Behavior". In: *Proc. 2011 31st Int. Conf. Distrib. Comput. Syst. Workshop. ICDCSW '11*. USA: IEEE Computer Society, 2011. DOI: [10.1109/ICDCSW.2011.20](https://doi.org/10.1109/ICDCSW.2011.20).
- [13] R. Mojena. "Hierarchical Grouping Methods and Stopping Rules: An Evaluation*". In: *Comput J* 20.4 (1977). DOI: [10.1093/comjnl/20.4.359](https://doi.org/10.1093/comjnl/20.4.359).

- [14] P. Farfán et al. “Dimers of Formic Acid: Structures, Stability, and Double Proton Transfer”. In: *J. Chem. Phys.* 147.4 (2017). DOI: [10.1063/1.4985880](https://doi.org/10.1063/1.4985880).
- [15] Anthropic. *System Card: Claude Opus 4.5*. <https://www.anthropic.com/claude-opus-4-5-system-card>. 2025.
- [16] Google DeepMind. *Gemini 3 Deep Think: Advancing Science, Research and Engineering*. <https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-deep-think/>. 2026.